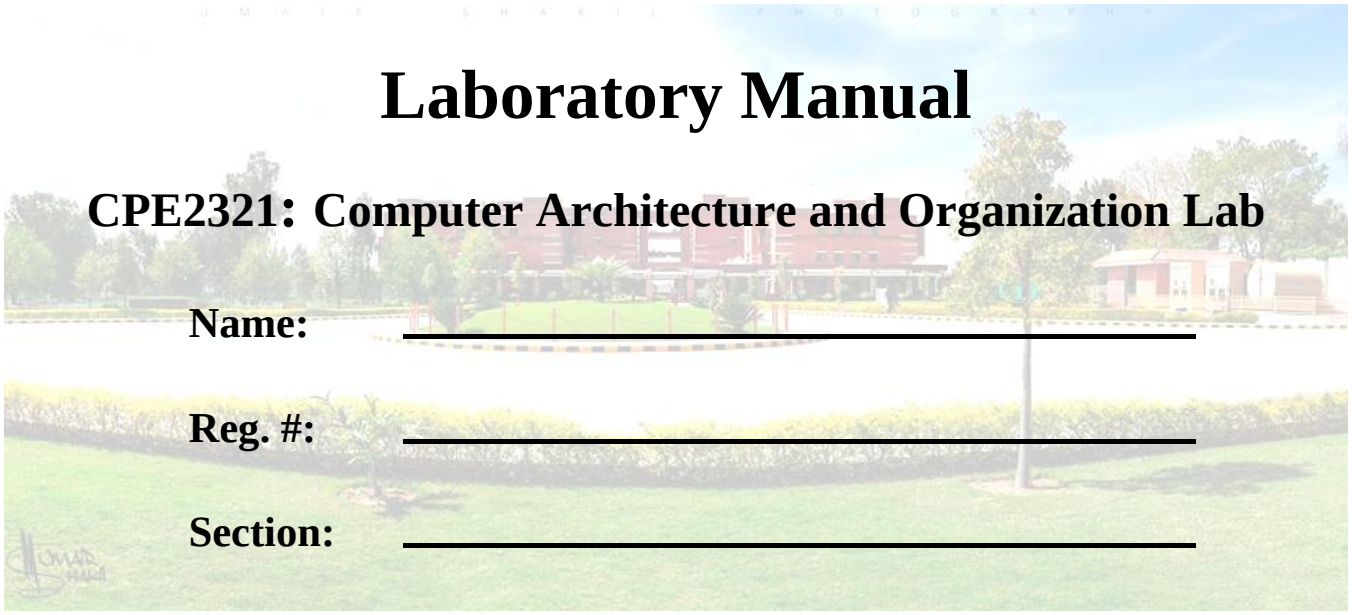


Laboratory Manual

CPE2321: Computer Architecture and Organization Lab

Name: _____

Reg. #: _____

Section: _____

Department of Electrical and Computer Engineering

Capital University of Science and Technology

Islamabad, Pakistan

© Copy Rights, Department of Electrical and Computer Engineering, CUST Islamabad

All Rights Reserved

Prepared By: Engr. Yasir Hussain

Revised By: Engr. Khurram Shahzad Khalid

List of Experiments

1. Introduction to Computer Architecture and Software Tools
2. Implementation of Arithmetic Logic Unit (ALU) in LogiSim
3. Implementation of RAM and ROM memory blocks in LogiSim
4. RISC-V assembly programming using Data Transfer and Arithmetic Instructions
5. RISC-V assembly programming using Logic and Jump instructions
6. Implementation of RISC-V assembler
7. Design of Single Cycle Processor Data path I
8. Design of Single Cycle Processor Data path II
9. Design of Single Cycle Processor Control-path
10. Integration of data and control path and Simulation of Single Cycle Processor
11. Pipelining of single cycle processor
12. Implementation of Cache memory hierarchy
13. Lab Project I
14. Lab Project II

Experiment No 1

Introduction to Computer Architecture and Software Tools

1.1 Objectives:

After completing this experiment, student will be able to:

- Understand the fundamentals of computer architecture and Organization.
- Understand the essential software tools including LogiSim and Venus.
- Explore LogiSim as a logic simulator for designing and simulating digital circuits.
- Explore Venus as a RISC-V simulator aiding in the comprehension of assembly language programming.

1.2 Background Theory

Basic computer architecture refers to the fundamental structure and organization of a computer system, outlining how its various components interact and function. At its core, computer architecture encompasses the central processing unit (CPU), memory, input/output devices, and the interconnection pathways. The CPU is the brain of the system, responsible for executing instructions, while memory stores data and instructions for quick access. The instruction set architecture defines the set of instructions the CPU can execute. The flow of data between the CPU and memory, as well as external devices, is managed through buses. The von Neumann architecture, a prevalent model, features a single shared memory space for both data and instructions. This architecture has evolved over time, with innovations like pipelining, parallel processing, and multiple cores enhancing computational capabilities. Understanding computer architecture is crucial for both hardware and software development, as it lays the foundation for efficient and effective computing systems.

LogiSim is a graphical, user-friendly logic simulator designed for educational purposes in the field of digital logic and computer architecture. It enables users, particularly students, to create and simulate digital circuits through a drag-and-drop interface. With a wide array of pre-built logic gates and components, LogiSim allows the design and analysis of both combinational and sequential circuits. Its real-time simulation feature aids in understanding the functionality of digital circuits, making it a valuable tool for hands-on learning in the realm of logic design.

Venus, on the other hand, is a RISC-V simulator geared towards facilitating the study of assembly language programming and computer architecture concepts. Supporting various RISC-V

instruction set extensions, Venus provides an environment for users to write, debug, and execute RISC-V assembly programs. It enables users to visualize the execution of each instruction, observe changes in registers and memory, and gain insights into the workings of a RISC-V processor. Venus serves as a powerful educational tool for students and enthusiasts seeking to delve into the intricacies of computer architecture and assembly programming.

1.2.1 Basic Computer Architecture Components

1. **Central Processing Unit (CPU)**
    - The brain of the computer responsible for executing instructions.
    - Comprises the control unit, arithmetic logic unit (ALU), and registers.
    - Interprets and executes instructions stored in the computer's memory.
2. **Memory**
    - Stores data and instructions that the CPU accesses during operations.
    - Divided into primary memory (RAM) for temporary storage and secondary memory (e.g., hard drives) for long-term storage.
3. **Input/Output (I/O) Devices**
    - Facilitate communication between the computer and the external world.
    - Examples include keyboards, mice, monitors, printers, and storage devices.
4. **Motherboard**
    - Main circuit board connecting all components.
    - Contains CPU socket, memory slots, expansion slots, and interfaces for peripheral devices.
5. **Bus System**
    - Communication pathways connecting various components.
    - Data bus transfers information between the CPU, memory, and peripherals.
    - Address bus specifies locations in memory, and the control bus manages data flow.
6. **Registers**
    - Small, high-speed storage locations within the CPU.
    - Hold data, addresses, and control information for quick access by the CPU.
7. **Arithmetic Logic Unit (ALU)**
    - Performs mathematical and logical operations.
    - Executes instructions, such as addition, subtraction, AND, OR, and NOT.
8. **Control Unit**
    - Manages the flow of data and instructions within the CPU.
    - Decodes instructions, initiates operations, and controls data movement.
9. **Clock**
    - Synchronizes operations within the computer.
    - The clock signal regulates the timing of instruction execution and data transfer.
10. **Cache Memory**
    - High-speed, small-sized memory that stores frequently used instructions and data.
    - Improves CPU performance by reducing the time needed to fetch information from main memory.

11. Power Supply

- Provides electrical power to all components.
- Converts electrical power from an external source to voltages suitable for the computer's operation.

12. Expansion Cards

- Additional circuit boards that enhance the computer's capabilities.
- Examples include graphics cards, sound cards, and network interface cards.

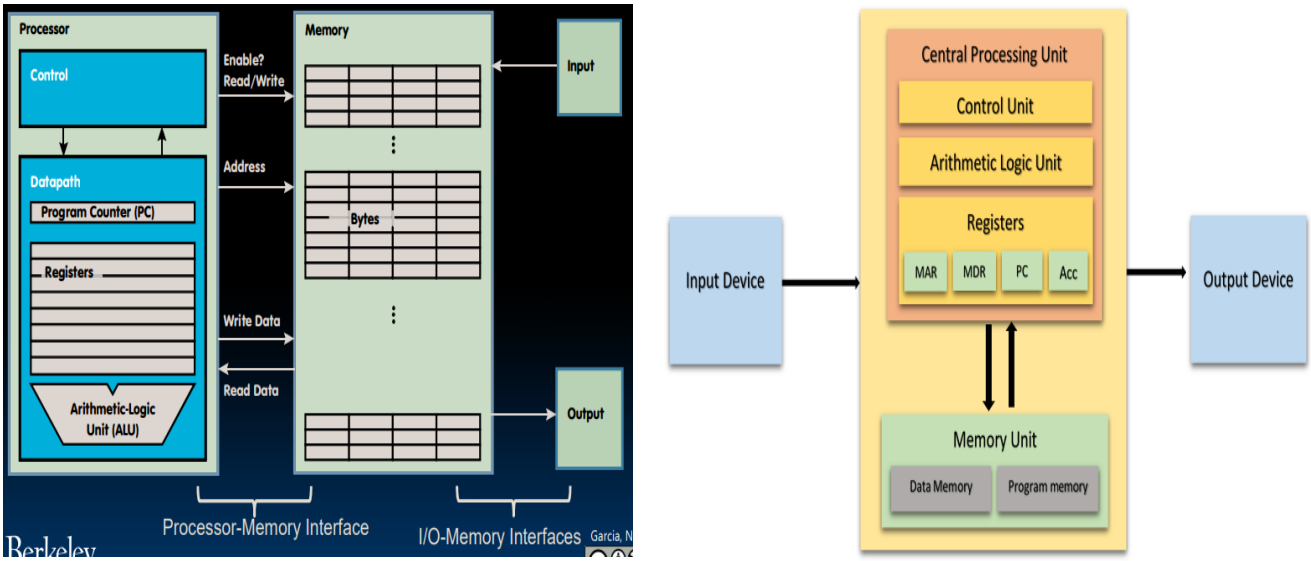


Figure 1.1: Basic Computer Architecture

1.2.2 LogiSim

Logisim is an educational tool for designing and simulating digital logic circuits. With its simple toolbar interface and simulation of circuits as they are built, it is simple enough to facilitate learning the most basic concepts related to logic circuits. With the capacity to build larger circuits from smaller subcircuits, and to draw bundles of wires with a single mouse drag, Logisim can be used (and is used) to design and simulate entire CPUs for educational purposes.

Students at colleges and universities around the world use Logisim for a variety of purposes, including:

1. A module in general-education computer science surveys
2. A unit in sophomore-level computer organization courses
3. Over a full semester in upper-division computer architecture courses

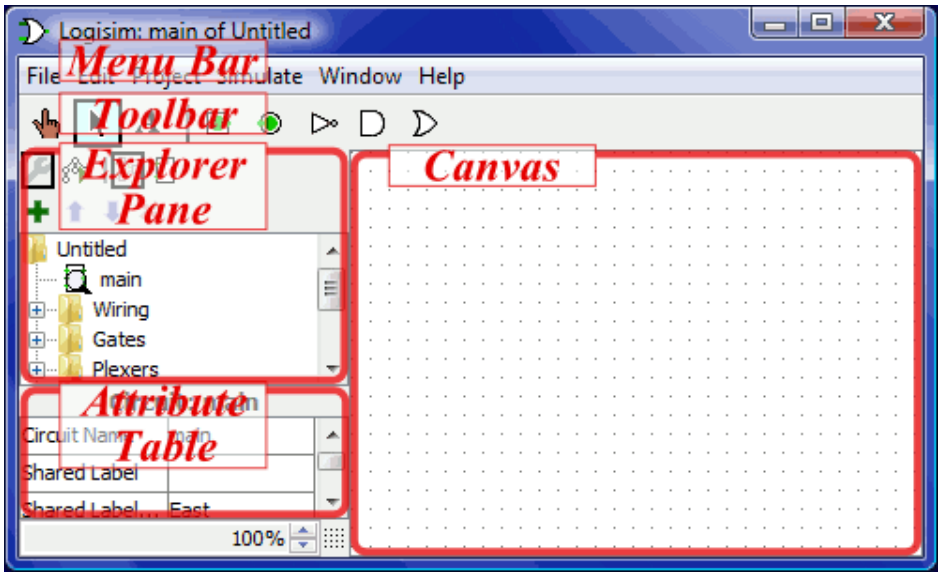


Figure 1.2: Logisim Interface

The explorer pane

Logisim organizes tools into libraries. They are displayed as folders in the explorer pane; to access a library's components, you have only to double-click the corresponding folder. Below, I have opened the Gates library and selected the NAND tool from it. You can see that Logisim now stands ready to add NAND gates into the circuit.

If you look through the choices in the Gates library, you'll notice that there was no need for us to develop a XOR circuit earlier: It's built into Logisim.

When you create a project, it automatically includes several libraries:

1. Wiring: Components that interact directly with wires.
2. Gates: Components that perform simple logic functions.
3. Plexers: More complex combinational components, like multiplexers and decoders.
4. Arithmetic: Components that perform arithmetic.
5. Memory: Components that remember data, like flip-flops, registers, and RAM.
6. I/O: Components that exist for the purpose of interacting with the user.

7. Base: Tools that are integral to using Logisim, though you probably won't need to dig into this library very often.

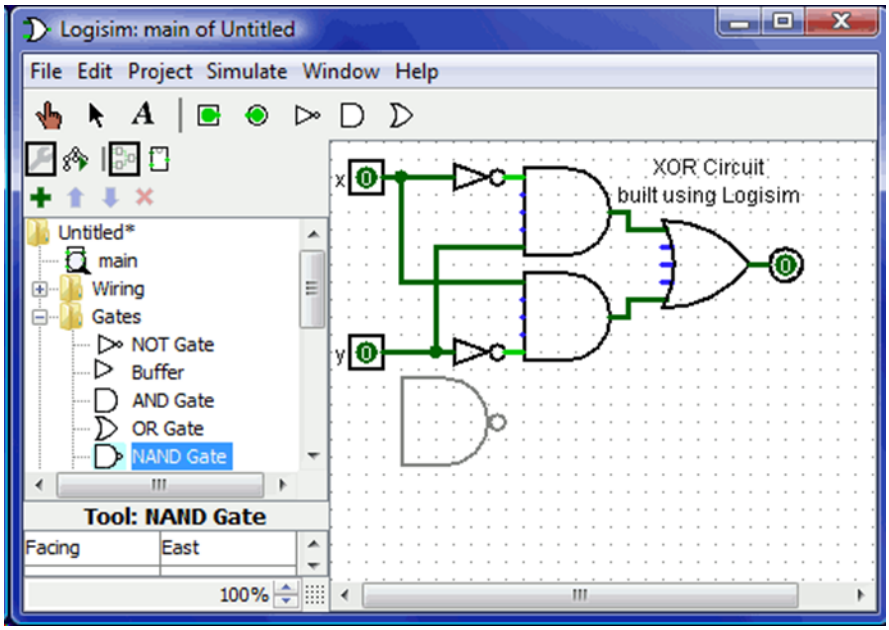


Figure 1.3: Explorer Pane

The attribute table

Many components have **attributes**, which are properties for configuring how the component behaves or appears. The **attribute table** is for viewing and displaying a component's attribute values.

To select which component's attributes you wish to view, click the component using the Edit tool ( ). (You can also right-click (or control-click) the component and choose Show Attributes from the popup menu. Also, manipulating a component via the Poke tool ( ) or the Text tool ( ) will display that component's attributes.)

The below screen shot demonstrates what things look like after selecting the upper input of our XOR circuit and scrolling down to view the Label Font attribute.

To modify an attribute value, click on the value. The interface for modifying the attribute will depend on which attribute you are changing; in the case of the Label Font attribute, a dialog box will appear for selecting the new font; but some attributes (like Label) will allow you to edit the value as a text field, while others (like Label Location) will display a drop-down menu from which to select the value.

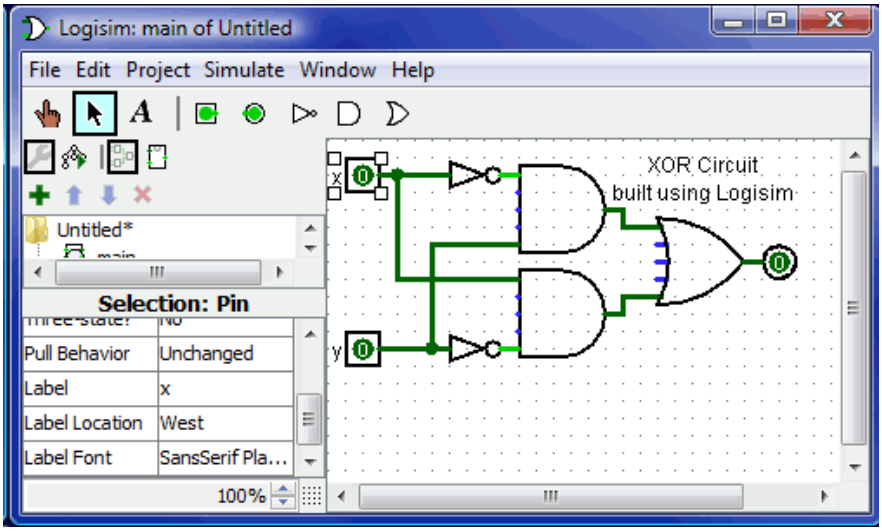


Figure 1.4: The Attribute Table

Tool attributes

Every tool for adding components to a circuit also has a set of attributes, which are imparted to the components created by the tool, although the components' attributes may be changed later without affecting the tool's attributes. When you select a tool, Logisim will change the attribute table to display that tool's attributes.

For example, suppose we want to create smaller AND gates. Right now, each time we select the AND tool, it creates a large AND gate. But if we edit the Gate Size attribute just after selecting the tool (before placing its AND gate into the circuit), we'll be changing the attributes for the tool, so that future AND gates added using the tool would be narrow instead.

With some tools, the tool's icon reflects some of the attributes' values. One example of this is the Pin tool, whose icon faces the same way as its Facing attribute says.

The tools in the toolbar each have a separate attribute set from the corresponding tools in the explorer pane. Thus, even though we changed the toolbar's AND tool to create narrow AND gates,

the AND tool in the Gates library will still create wide AND gates unless you change its attributes too.

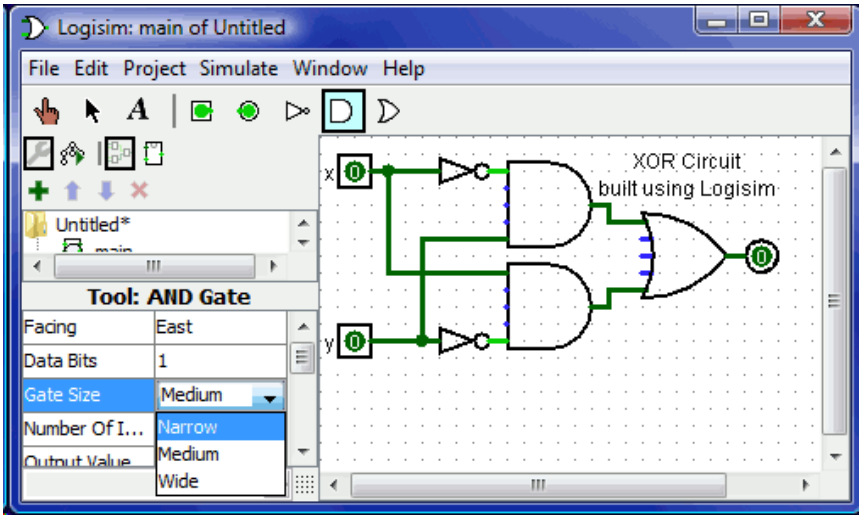


Figure 1.5: The Attribute Table

How to Get Started:

1. **Download and Install:**
    - Visit the Logisim website and download the latest version suitable for your operating system. Install it following the provided instructions.
2. **Create a New Circuit:**
    - Open Logisim and create a new project. Start designing your circuit by adding components from the toolbar.
3. **Wire Components:**
    - Connect components by dragging wires between them. Establish logical connections according to your design.
4. **Simulation:**
    - Use the simulation tool to test your circuit. Provide input values and observe the output behavior.
5. **Save and Export:**
    - Save your project for future editing. You can also export your circuit as a file or an image.

1.2.3 Venus Risc-V Simulator

The Visual Studio Code extension embeds the popular Venus RISC-V simulator. It provides a standalone learning environment as no other tools are needed. It runs RISC-V assembly code with the standard debugging capabilities of VS Code.

To use it as educational tool, further views are added as described below.

1. **Installation**

   You can easily install the extension via the marketplace by searching for "riscv":

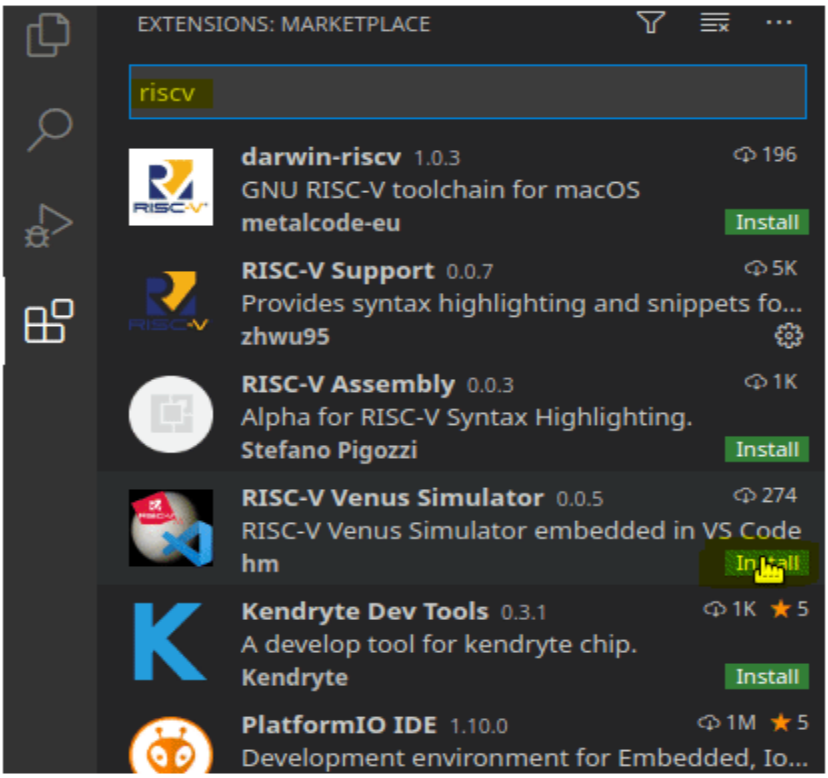


Figure 1.6: Venus Installation

Alternatively, you can install it via VS Code Quick Open (CTRL+P):

```
ext install hm.riscv-venus
```

2. Getting Started

You can debug any assembler file you are editing. We recommend using the RISC-V Support extension for syntax highlighting.

Just start debugging with "Run and Debug" to debug the current file:

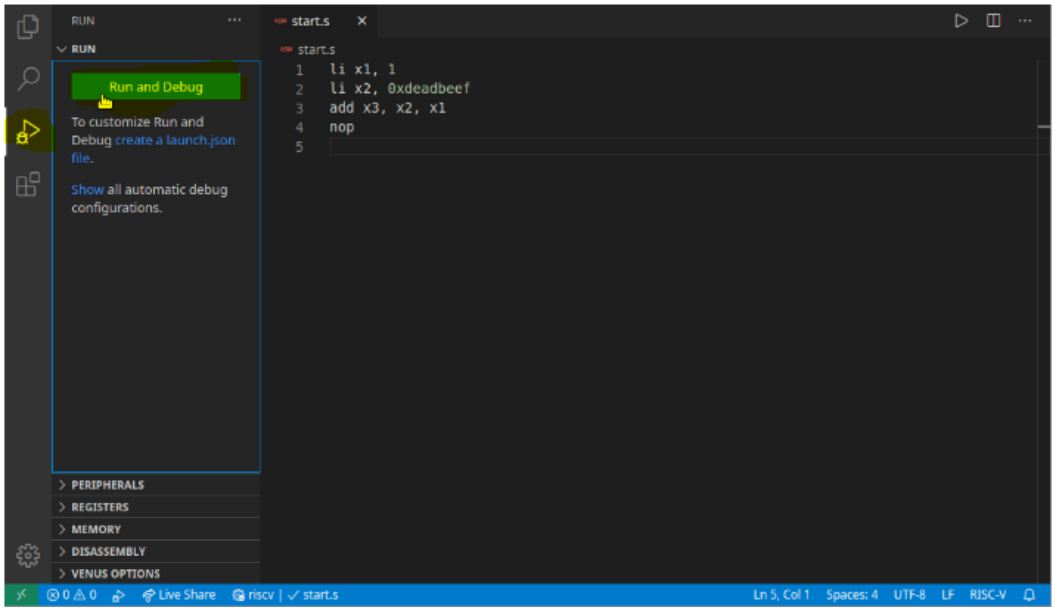


Figure 1.7: Run and Debug

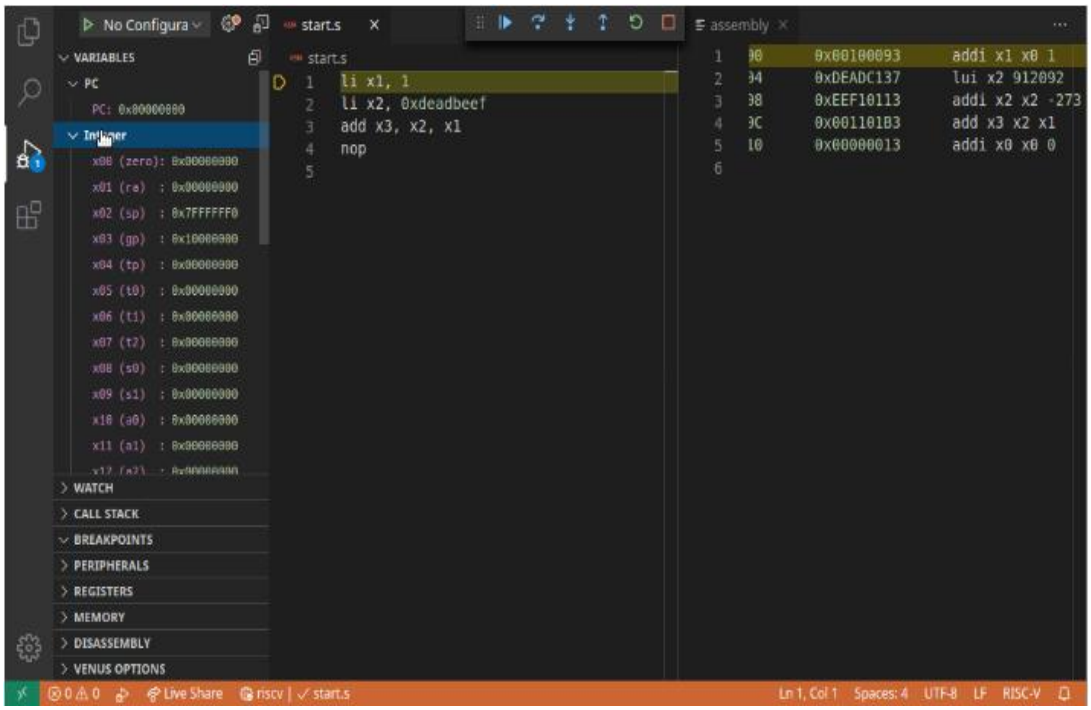


Figure 1.8: Instruction Execution

You can step through the code and pseudo-operations are expanded automatically. The actually executed instructions are displayed in an assembly view (auto-opens if set in extension-settings). Breakpoints are of course also supported.

The basic Venus environmental calls are also supported. Further environment calls are also supported for educational toy examples are described in the following.

3. UI and launch paramters

From, the Debugger you can open the "Venus Options". From there you can open settings, views, documentation and more. Also there are commands for all these actions which you can execute with CTRL+P and typing "Venus: ..."

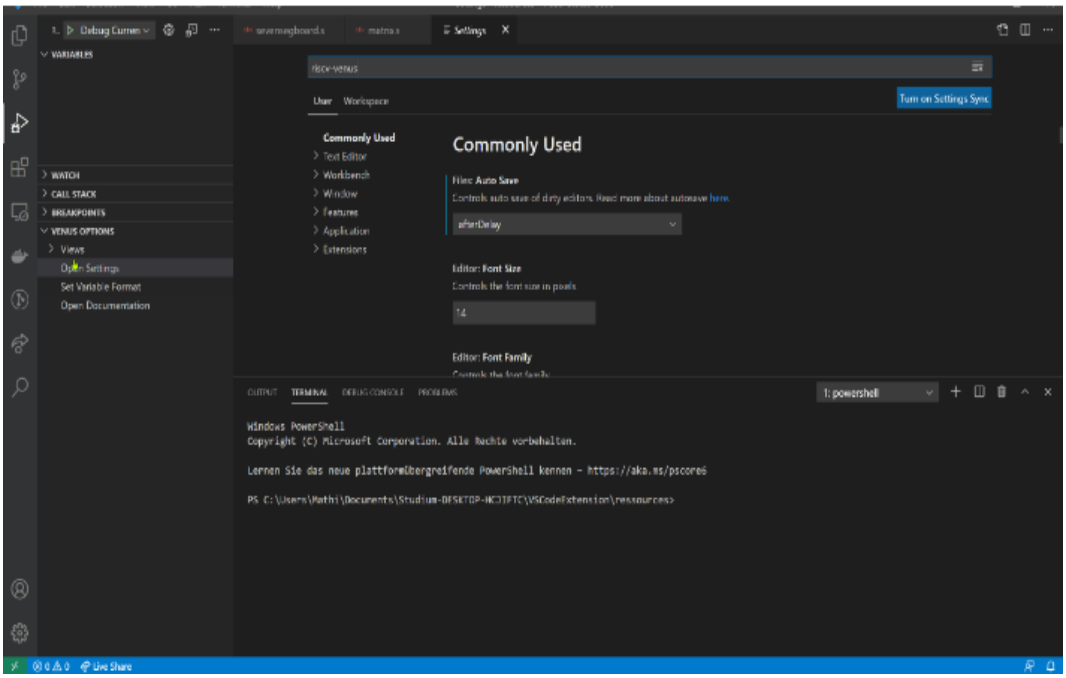


Figure 1.9: UI and Launch Parameters

4. View memory

You can view the memory contents by opening the memory view with CTRL+P and "Venus: Open Memory".

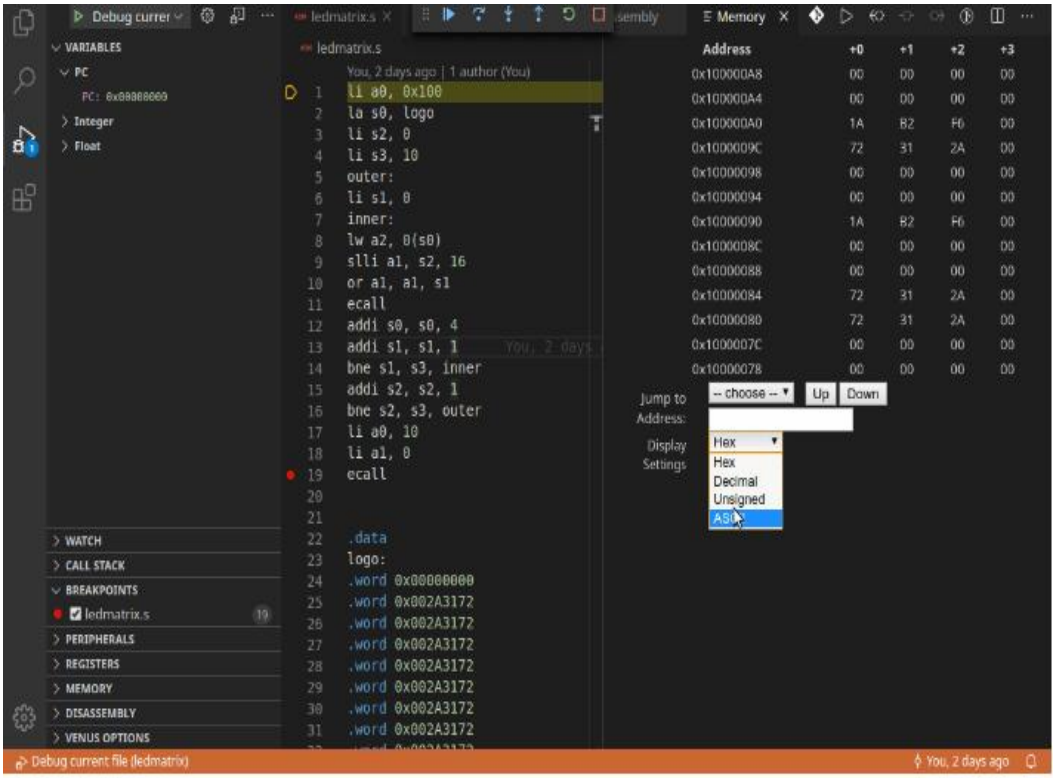


Figure 1.10: View Memory

You can select which segment to view and scroll through the memory words (as in the online Venus version).

5. Accessing registers

You can change the registers by clicking on their value, update the value and press enter.

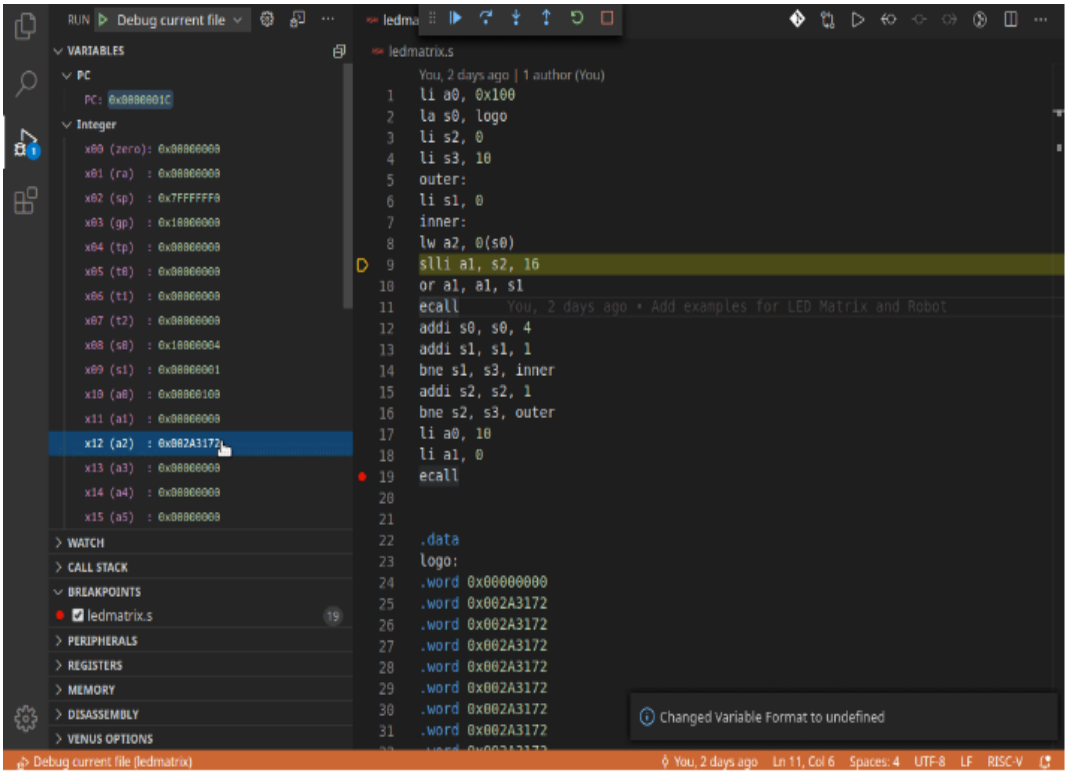


Figure 1.11: Accessing Registers

To change to format of the registers, click on "Venus Options" or press CTRL+P and enter "Venus: Set Variable Format".

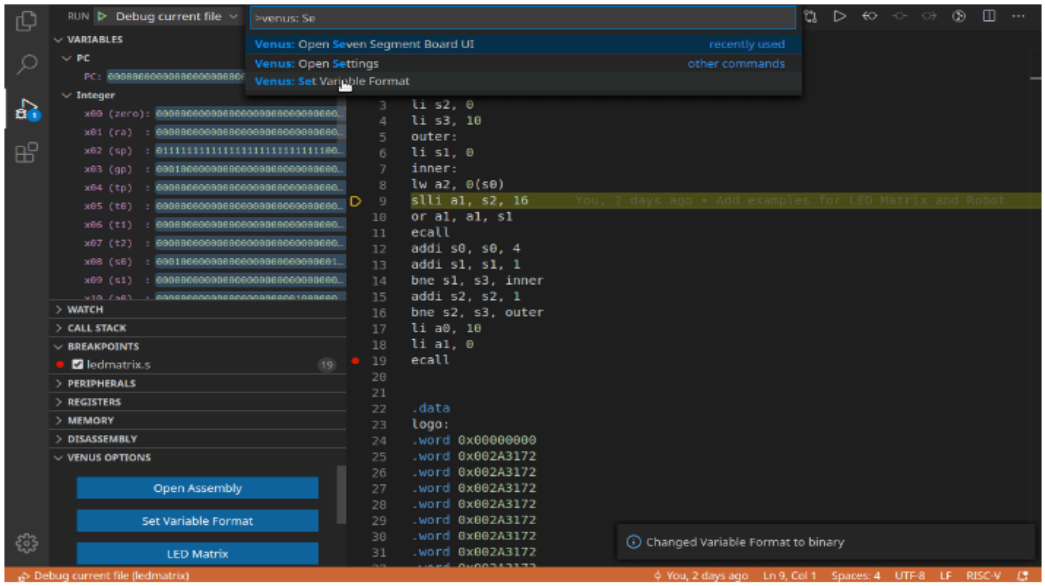
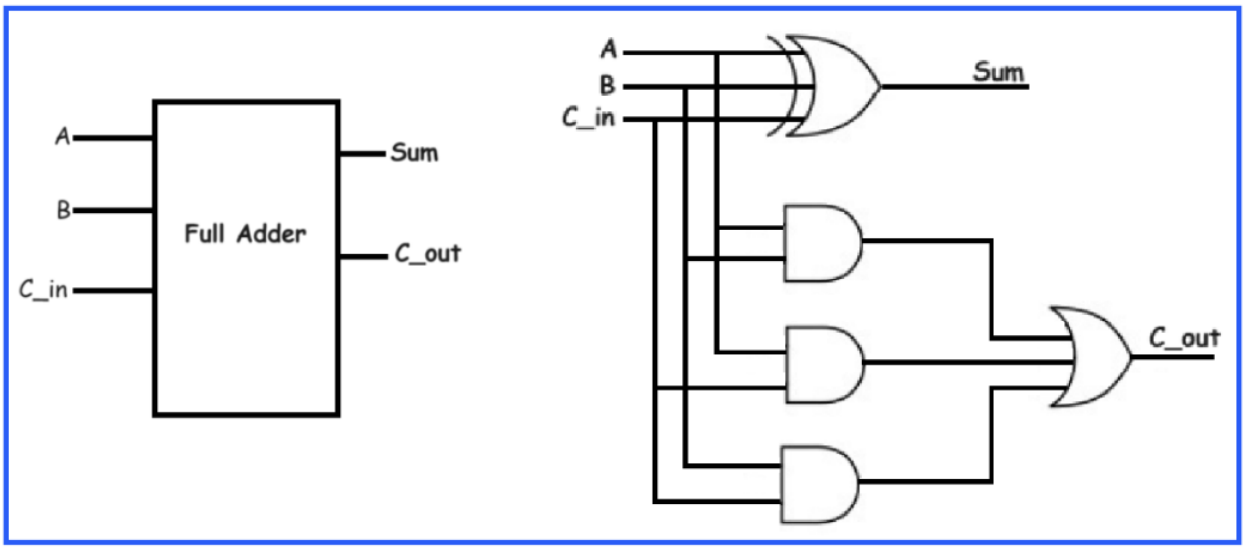


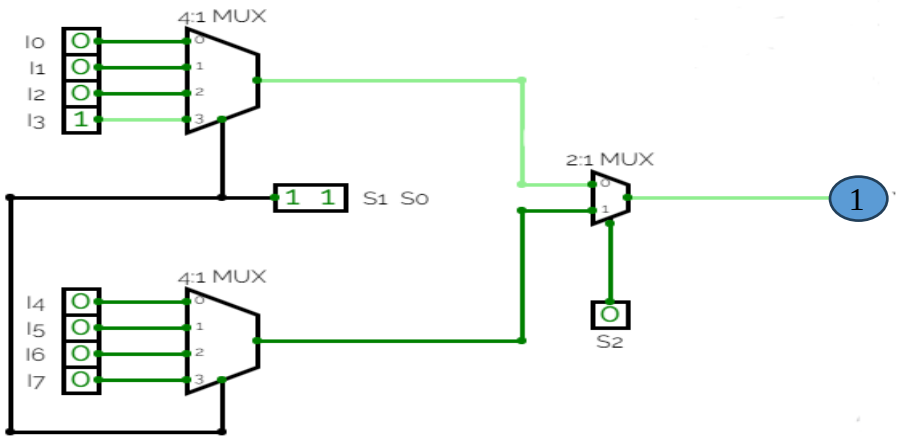
Figure 1.12: Formatting the Registers

1.3 Lab Activities

1.3.1 Lab Task-1: Implement and Verify the Following Full Adder in Logisim



1.3.2 Lab Task-2: Implement and Verify 8:1 Mux using 4:1 Mux using LogiSim



Sr. No	S0	S1	S2	I0	I1	I2	I3	I4	I5	I6	I7	Y
1	0	0	0	0	X	X	X	X	X	X	X	0
2	0	0	0	1	X	X	X	X	X	X	X	1
3	0	0	1	X	0	X	X	X	X	X	X	0
4	0	0	1	X	1	X	X	X	X	X	X	1
5	0	1	0	X	X	0	X	X	X	X	X	0
6	0	1	0	X	X	1	X	X	X	X	X	1
7	0	1	1	X	X	X	0	X	0	0	0	0
8	0	1	1	X	X	X	1	X	1	1	1	1
9	1	0	0	X	X	X	X	0	X	X	X	0
10	1	0	0	X	X	X	X	1	X	X	X	1
11	1	0	1	X	X	X	X	X	0	X	X	0
12	1	0	1	X	X	X	X	X	1	X	X	1
13	1	1	0	X	X	X	X	X	X	0	X	0
14	1	1	0	X	X	X	X	X	X	1	X	1
15	1	1	1	X	X	X	X	X	X	X	0	0
16	1	1	1	X	X	X	X	X	X	X	1	1

Cognitive Activity

Q1. You need an 8:1 MUX but only have 4:1 MUX and 2:1 MUX components available. Explain, step by step, how you would connect two 4:1 MUXes and one 2:1 MUX to build it, and state how many select lines are needed in total and what each one does. **(5)**

Ans: _____

_____

_____

_____

_____

Q2. A Full Adder is built from two Half Adders and one OR gate. Trace the circuit for inputs A=1, B=1, Cin=1: show the Sum and Carry produced by the first Half Adder, the Sum and Carry produced by the second Half Adder, and the final Sum and Cout of the Full Adder. **(5)**

Ans: _____

_____

_____

_____

_____

Obtained Marks	**/10**

Lab Exercise and Summary

Summary should cover Introduction, Procedure, Data Analysis and Evaluation.

Student's Signature: _____ **Date: _____**

Psychomotor Activity 1 — Experimental Setup

Criteria	Performance Level	Description	Marks
Tool Setup (4)	Level 4	Opens and configures LogiSim and/or Venus correctly and independently.	4
	Level 3	Configures the required tool with minor errors or assistance.	3
	Level 2	Completes the setup with considerable assistance.	2
	Level 1	Setup is incomplete and requires substantial assistance.	1
File/Design Setup (4)	Level 4	Correctly creates/selects the required LogiSim circuit/design and/or Venus assembly file and required components.	4
	Level 3	File/design setup is mostly correct with minor errors or omissions.	3
	Level 2	Setup is partially completed and requires assistance.	2
	Level 1	Major errors or omissions are present in the setup.	1
Parameter Setup (2)	Level 2	Correctly defines required inputs, data, registers, components and initial conditions.	2
	Level 1	Defines most required inputs/components with minor errors.	1
Total Obtained			/10

If a student is absent:

Criteria	Description	Marks
Absent	Student is absent from the laboratory activity.	**0**

Instructor Name: _____ Sign: _____

Psychomotor Activity 2 — Implementation + Results

Criteria	Performance	Description	Marks
Implementation (4)	Level 4	Implements the required Full Adder and 8:1 MUX using 4:1 MUX correctly and independently in LogiSim.	4
	Level 3	Implements the required design with minor errors or assistance.	3
	Level 2	Partially implements the design and requires considerable assistance.	2
	Level 1	Implementation is largely incomplete and requires substantial assistance.	1
Execution/Simulation (2)	Level 2	Runs/simulates the LogiSim designs correctly and appropriately handles basic execution errors.	2
	Level 1	Runs/simulates the work with minor errors or assistance.	1
Output/Results	Level 4	Obtains correct results and clearly demonstrates the required circuit behavior/output for the tested inputs.	4
	Level 3	Obtains mostly correct results with minor errors in output or presentation.	3
	Level 2	Obtains partially correct results with noticeable errors or omissions.	2
	Level 1	Results are largely incomplete or require substantial correction.	1
Total Obtained			/10

If a student is absent:

Criteria	Description	Marks
Absent	Student is absent from the laboratory activity.	0

Affective Activity — Lab Report + Viva Voce

Criteria	Performance	Description	Marks
Lab Report Completeness & Organization	Level 4	Report is complete, well organized, clearly presents procedure, observations, measurements, and discussion.	4
	Level 3	Report is complete and organized with minor omissions.	3
	Level 2	Report is partially complete and requires improvement in organization/detail.	2
	Level 1	Report is incomplete or poorly organized.	1
Results & Conclusion	Level 2	Correctly presents observations/results and provides an appropriate conclusion.	2
	Level 1	Results or conclusion are incomplete or contain minor errors.	1
Viva Voce	Level 4	Clearly explains oscilloscope operation, function generator settings, waveform measurements, and experimental observations.	4
	Level 3	Explains most concepts correctly with minor prompting.	3
	Level 2	Demonstrates basic understanding but requires prompting.	2
	Level 1	Demonstrates limited understanding of the experiment.	1

The student has not submitted the report and has not appeared in the Viva Voce.

Criteria	Description	Marks
Lab Report and Results Not Submitted	Student has not submitted the report and results file.	0
Absent in Viva Voce	Student is absent from the viva voce.	0

Instructor's Signature: _____ **Date: _____**

Experiment No 2

Implementation of Arithmetic Logic Unit (ALU) in LogiSim

2.1 Objectives:

After completing this experiment, student will be able to:

- Understand the basic functions of an ALU.
- Use different components like MUX, ADDER, SUBTRACTOR and basic Logical Operations in LogiSim.
- Design and Implement 1, 8, 16 and 32 bit ALU in LogiSim.

2.2 Background Theory

An Arithmetic Logic Unit (ALU) is a fundamental component in digital systems, playing a crucial role in performing arithmetic and logical operations. It serves as the computational heart of a microprocessor or a digital circuit, executing mathematical calculations and logical comparisons that are essential for processing data.

2.2.1 Functions of an ALU

2.2.1.1 Addition

The primary arithmetic function of an ALU is addition. It can add binary numbers, which is a fundamental operation in any digital computing system. The ALU takes two binary inputs and produces their sum as the output.

2.2.1.2 Subtraction

Subtraction is closely tied to addition in digital systems. An ALU can perform subtraction by employing a mechanism known as two's complement arithmetic. This allows for efficient addition of negative numbers, and the ALU can effectively subtract one binary number from another.

2.2.1.3 Logical AND Operations

ALUs also perform logical operations, including the AND operation. In the AND operation, each bit of the output is the logical AND of the corresponding bits of the input. It results in a '1' only if both input bits are '1'; otherwise, the output is '0'.

2.2.1.4 Logical OR Operations

Another essential logical operation is the OR operation. The OR operation produces a '1' if at least one of the corresponding input bits is '1'. It results in '0' only when both input bits are '0'.

2.2.1.5 Logical XOR Operations

The XOR operation is a logical operation where the output is '1' if the input bits are different and '0' if they are the same. XOR is frequently used in various applications, including data comparison and error detection.

2.2.2 Importance in Digital Systems

2.2.2.1 Central Processing Unit (CPU)

ALUs are a critical component of a CPU, where they perform the arithmetic and logical operations necessary for executing program instructions. The speed and efficiency of an ALU directly impact the overall performance of the CPU.

2.2.2.2 Data Processing

In digital systems, ALUs are responsible for manipulating data, making them indispensable in applications ranging from basic calculators to complex data processing units in computers and embedded systems.

2.2.2.3 Control Flow

ALUs play a key role in controlling the flow of operations within a digital system. They enable decision-making processes, comparisons, and conditional branching based on the results of arithmetic and logical operations.

2.2.2.4 Mathematical Computations

ALUs are crucial for performing mathematical computations, making them integral to applications such as scientific calculations, simulations, and numerical analysis.

So, the Arithmetic Logic Unit is a fundamental building block of digital systems, providing the computational capabilities necessary for a wide range of applications in the field of electronics and computing. Its ability to perform both arithmetic and logical operations makes it a versatile and indispensable component in the world of digital design and information processing.

2.2.4 1 Bit ALU Design

Designing a 1-bit Arithmetic Logic Unit (ALU) with basic operations like AND, OR, XOR, addition, subtraction, and multiplication involves careful arrangement of logic components. In the following Figure: 2.1, two multiplexers (MUX) play a crucial role in managing the output and carry-out signals. For each operation, the ALU utilizes logic gates such as AND, OR, and XOR to process the input bits. The addition and subtraction operations employ a 1-bit full-adder for the arithmetic computations. To facilitate the selection of the desired operation, a control line is connected to the input of the MUX responsible for generating the output. Another MUX is incorporated to manage the carry-out signal. During addition or subtraction, the carry-out from the full-adder is selected, while for logical operations, the carry-out is set to zero. By implementing this design, the 1-bit ALU effectively performs various operations, and the multiplexers contribute to the flexibility of choosing the appropriate results based on the operation selected.

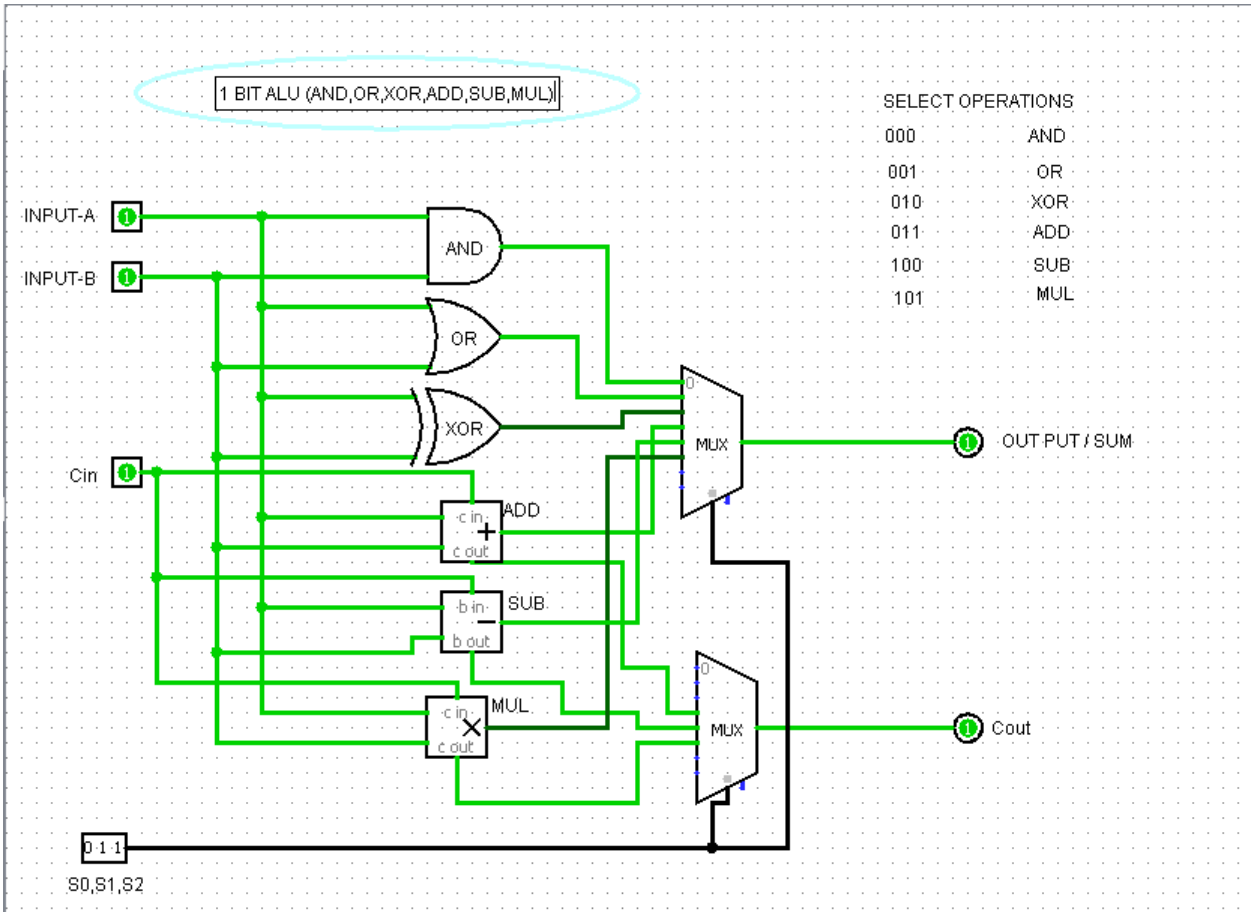


Figure 2.1: 1-Bit ALU Design

2.2.5 8 Bit ALU Design

Just change the input and output bit size to 8 bits and the rest of the circuit will be remain same as shown in Figure: 2.2.

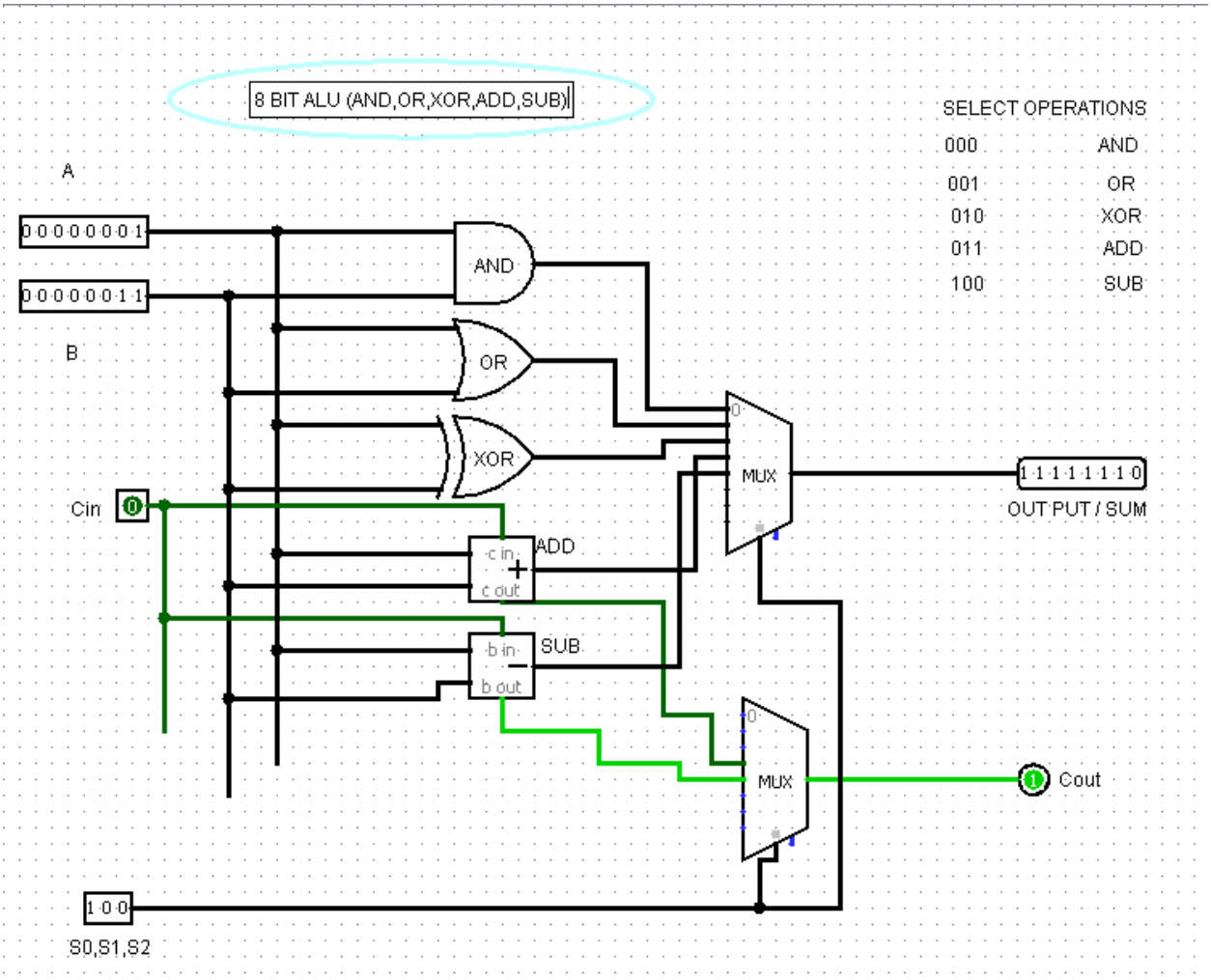


Figure 2.2: 8-Bit ALU Design

2.3 Lab Activities

2.3.1 Task-1: Design 16 Bits ALU in LogiSim having (AND, OR, XOR, ADD and SUB) functions.

2.3.2 Task-2: Design 32 Bits ALU in LogiSim having (AND, OR, NOT, XOR, ADD and SUB) functions.

Lab Exercise and Summary

Summary should cover Introduction, Procedure, Data Analysis and Evaluation.

Student's Signature: _____ **Date: _____**

Cognitive Activity

Q1. You are designing a 4-function ALU (ADD, SUB, AND, OR) selected by a 2-bit control signal. Describe, in words, the components you would use and how the 2-bit control signal decides which result reaches the output. **(5)**

Ans: _____

_____

_____

_____

_____

Q2. For inputs A = 1010 and B = 0011 (4-bit, unsigned), calculate the ALU output for each of: ADD, SUB (A-B, two's complement), AND, and OR. **(5)**

Ans: _____

_____

_____

_____

_____

Obtained Marks	**/10**

Psychomotor Activity 1 — Experimental Setup

S. No	Criteria	Performance	Description	Marks
1	Tool Setup (4)	Level 4	Opens and configures LogiSim and selects the required environment correctly and independently.	4
		Level 3	Configures the required tool with minor errors or assistance.	3
		Level 2	Completes the setup with considerable assistance.	2
		Level 1	Setup is incomplete and requires substantial assistance.	1
2	File/Design Setup (4)	Level 4	Correctly creates/selects the required assembly file, Logisim circuit/design and required components for the Logisim ALU circuit.	4
		Level 3	File/design setup is mostly correct with minor errors or omissions.	3
		Level 2	Setup is partially completed and requires assistance.	2
		Level 1	Major errors or omissions are present in the setup.	1
3	Parameter Setup (2)	Level 2	Correctly defines required data, registers, parameters, components and initial conditions.	2
		Level 1	Most required inputs/components are correctly configured with minor errors.	1
			Total Obtained	/10

If a student is absent:

Criteria	Description	Marks
Absent	Student is absent from the laboratory activity.	0

Psychomotor Activity 2 — Implementation + Results

S. No	Criteria	Performance	Description	Marks
1	**Implementation (4)**	Level 4	Implements the required Logisim ALU circuit correctly and independently.	**4**
		Level 3	Implements the required program/design with minor errors or assistance.	**3**
		Level 2	Partially implements the program/design and requires considerable assistance.	**2**
		Level 1	Implementation is largely incomplete and requires substantial assistance.	**1**
2	**Execution/Simulation (2)**	Level 2	Executes the RISC-V program or simulates the Logisim design correctly and handles basic errors appropriately.	**2**
		Level 1	Executes/simulates the work with minor errors or assistance.	**1**
3	**Results/Output (4)**	Level 4	Obtains correct results and clearly demonstrates the required program output or circuit behavior.	**4**
		Level 3	Obtains mostly correct results with minor errors.	**3**
		Level 2	Obtains partially correct results with noticeable errors or omissions.	**2**
		Level 1	Results are largely incorrect or incomplete.	**1**
			Total Obtained	**/10**

If a student is absent:

Criteria	Description	Marks
Absent	Student is absent from the laboratory activity.	**0**

Affective Activity — Lab Report + Viva Voce

S. No	Criteria	Performance	Description	Marks
1	Lab Report Completeness & Organization (4)	Level 4	Report is complete, well organized and demonstrates full responsibility for the assigned work.	4
		Level 3	Report is complete with minor omissions or organizational issues and demonstrates satisfactory responsibility for the assigned work.	3
		Level 2	Report is partially complete with several omissions and demonstrates limited responsibility for the assigned work.	2
		Level 1	Report has major omissions or poor organization and demonstrates minimal responsibility for the assigned work.	1
2	Code/Design, Results & Conclusion (2)	Level 2	Logisim ALU circuit, results, and conclusion are correctly presented and discussed, demonstrating ownership of the assigned work.	2
		Level 1	Logisim ALU circuit, results, or conclusion are presented with minor errors or omissions.	1
3	Viva Voce (4)	Level 4	Clearly explains the assigned Logisim ALU circuit, working, and results, demonstrating individual responsibility, participation, and effective communication.	4
		Level 3	Explains most of the assigned program/design and results correctly, with minor difficulty, and demonstrates good individual participation.	3
		Level 2	Explains basic aspects of the assigned program/design with some prompting and demonstrates basic participation.	2
		Level 1	Demonstrates limited understanding or involvement and requires substantial prompting to explain the assigned work.	1
			Total Obtained	/10

The student has not submitted the report and has not appeared in the Viva Voce.

Criteria	Description	Marks
Lab Report and Results Not Submitted	Student has not submitted the report and results file.	**0**
Absent in Viva Voce	Student is absent from the viva voce.	**0**

Instructor's Signature: _____ **Date:** _____

Experiment No 3

Implementation of RAM and ROM memory blocks in LogiSim

3.1 Objectives

After completing this experiment, student will be able to:

- Understand RAM and ROM basic operations.
- Implement RAM and ROM memory blocks in LogiSim.
- Store and Load data in RAM and ROM.
- Experience Counter, Bit Extender, TTY and Random Generator components of LogiSim.

3.2 Background Theory

RAM (Random Access Memory) is a type of volatile memory in a computer system that serves as a temporary workspace for actively running programs and data. It provides fast access to data for the CPU (Central Processing Unit), allowing it to quickly read and write information during program execution. RAM's volatile nature means that its contents are lost when the computer is powered off, necessitating regular saving of data to non-volatile storage like hard drives or SSDs. This temporary storage capability enables multitasking, as RAM can hold multiple programs and data simultaneously, facilitating smooth operation without constantly accessing slower storage mediums.

On the other hand, ROM (Read-Only Memory) is a type of non-volatile memory that stores essential software and firmware instructions required for booting up the computer and initializing hardware components. Unlike RAM, ROM's contents are permanent and cannot be easily modified or erased during normal operation. It holds critical instructions that are crucial for the system's basic functionality, such as the BIOS (Basic Input/Output System) or firmware for various hardware components. ROM plays a foundational role in the startup process, ensuring that the computer can begin executing instructions even before other forms of storage, like hard drives, become accessible.

3.2.1 RAM in LogiSim

In Logisim, RAM (Random Access Memory) refers to a component that can store binary data in a manner that allows for both reading and writing. You can use RAM in LogiSim as shown in Figure: 3.1.

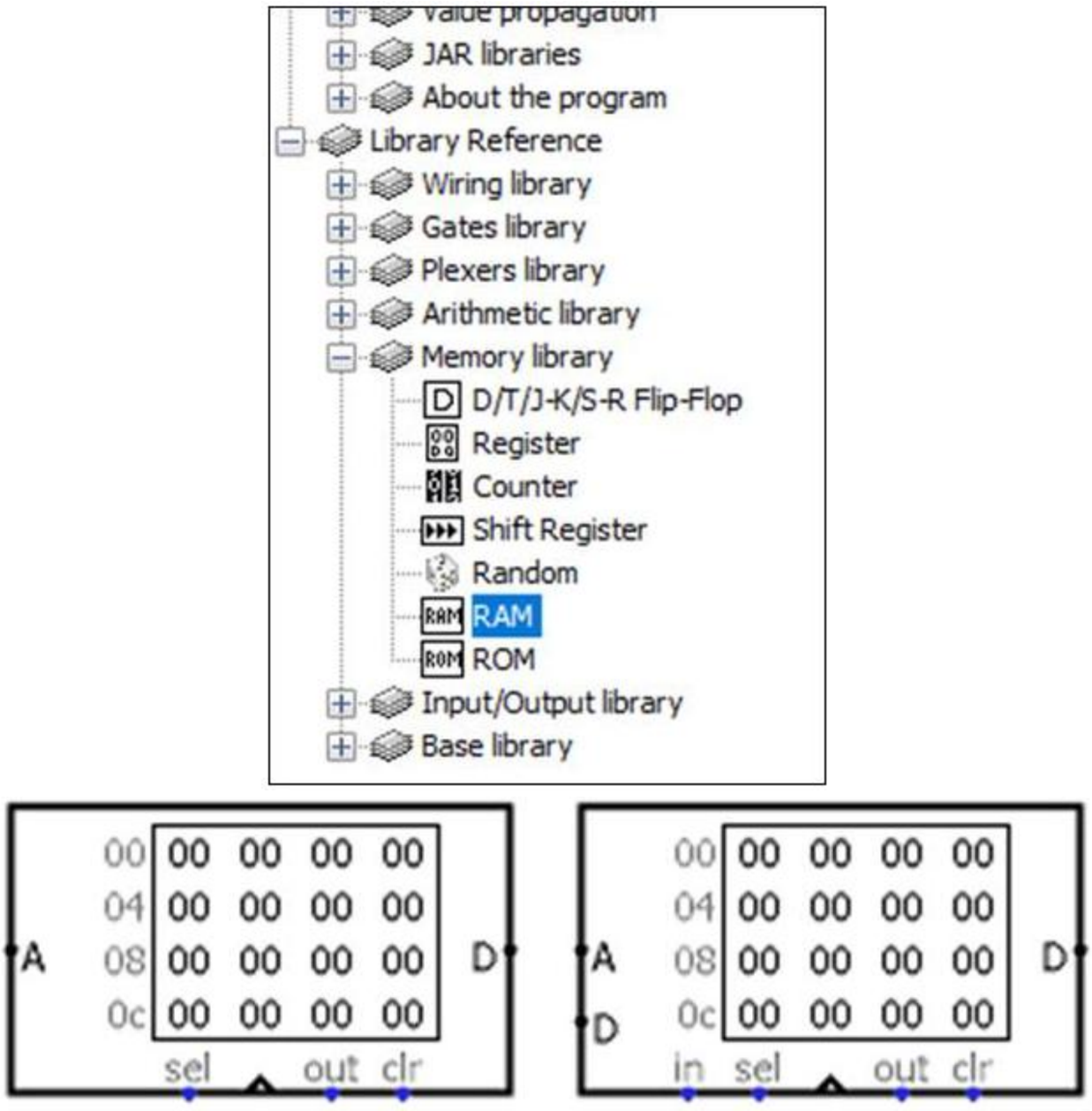


Figure 3.1: Appearance of RAM in LogiSim

The RAM component, easily the most complex component in Logisim's built-in libraries, stores up to 2^{24} = 16,777,216 values (specified in the Address Bit Width attribute), each of which can

include up to to 32 bits (specified in the Data Bit Width attribute). The circuit can load and store values in RAM. Also, the user can modify individual values interactively via the Poke Tool, or the user can modify the entire contents via the Menu Tool.

Current values are displayed in the component. Addresses displayed are listed in gray to the left of the display area. Inside, each value is listed using hexadecimal. The value at the currently selected address will be displayed in inverse text (white on black).

The RAM component supports three different interfaces, depending on the Data Interface attribute.

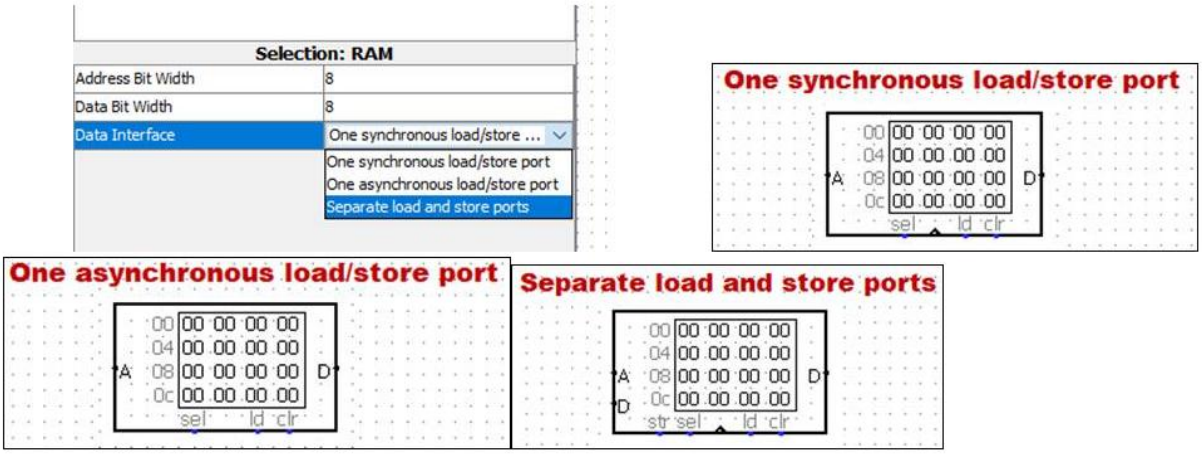


Figure 3.2: Different Interfaces of RAM in LogiSim

3.2.1.1 One synchronous load/store port (default)

The component includes a single port on its east side that serves for both loading and storing data. Which it performs depends on the input labeled *ld*: 1 (or floating) indicates to load the data at the address designated on the component's west side, and 0 indicates to store the data given on the

port. To transmit data into and out of the component, you will need to use a Controlled Buffer component, as illustrated below.

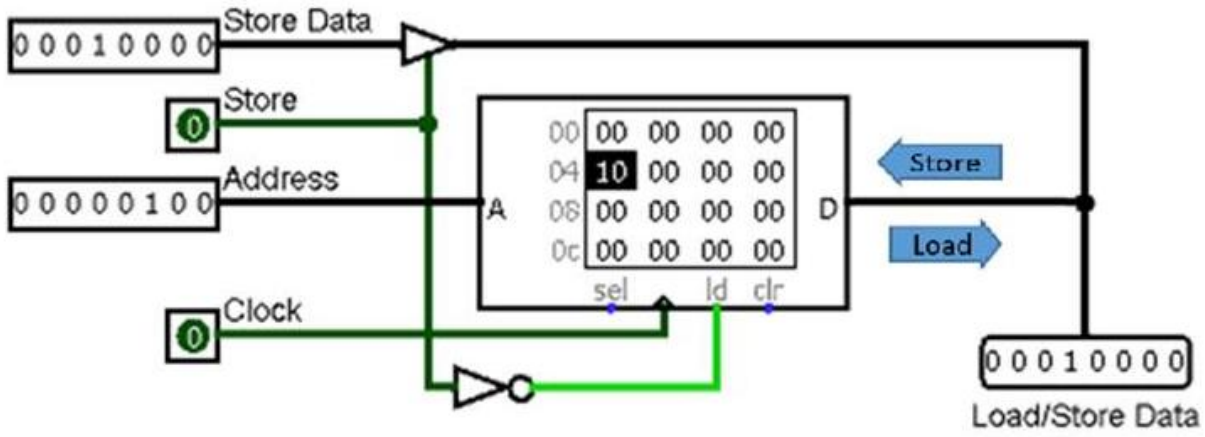


Figure 3.3: One synchronous load/store port (default) of RAM in LogiSim

3.2.1.2 One asynchronous load/store port

This is the same as above, except that there is no clock. The value found on the data bus is stored into memory whenever the *ld* input is 0. If, while the *ld* input is 0, the address or data changes, then an additional store occurs. This option is meant to more closely approximate the interface of many available random-access memories.

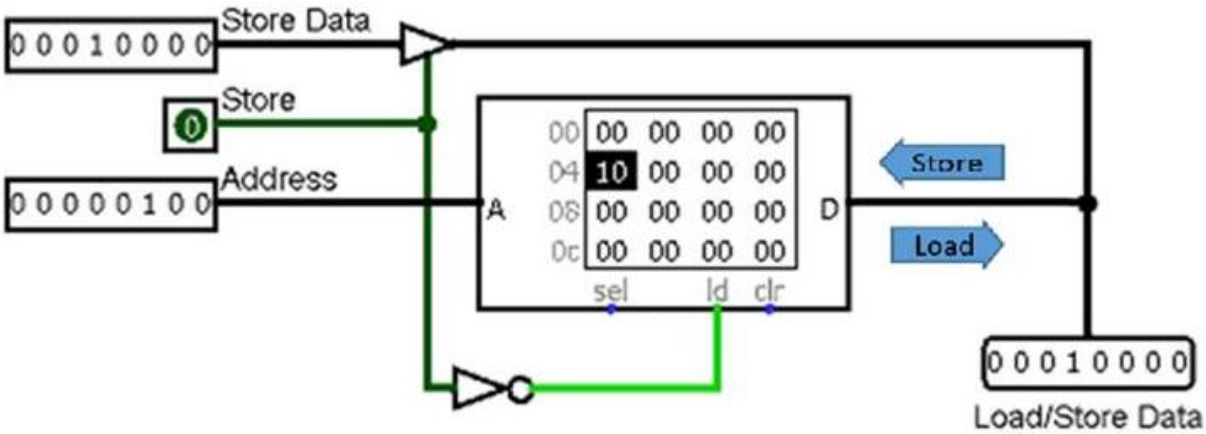


Figure 3.4: One synchronous load/store port of RAM in LogiSim

3.2.1.3 Separate load and store ports

Two data ports are provided - one on the west side for storing data, and another on the east side for loading data. This option removes the necessity of dealing with the Controlled Buffer and so it is easier to use.

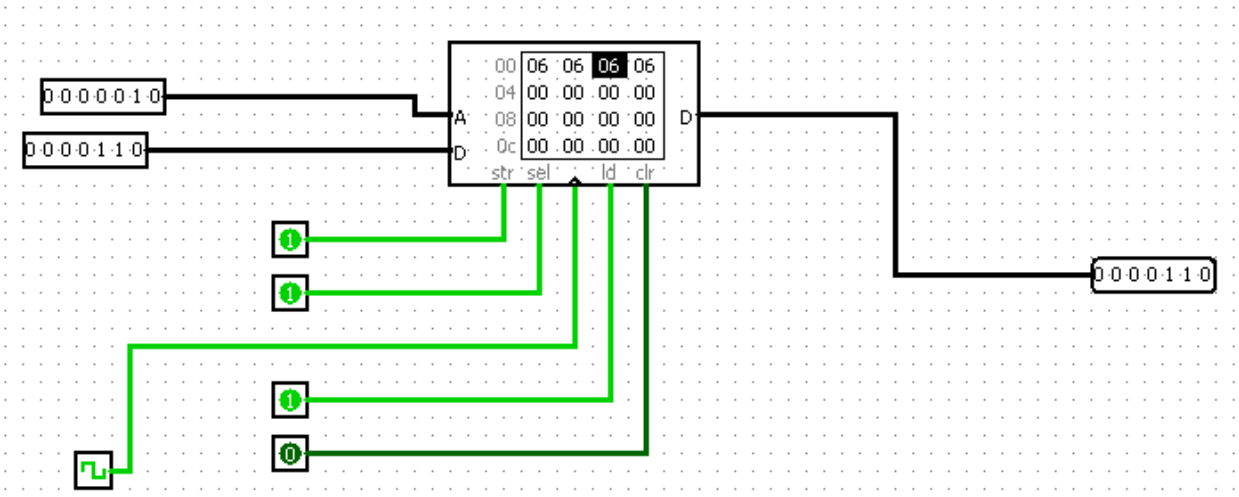


Figure 3.5: Separate load/store port of RAM in LogiSim

3.2.1.4 Pins

A on west edge (input, bit width matches Address Bit Width attribute)
Selects which of the values in memory is currently being accessed by the circuit.

D on west edge (input, bit width matches Data Bit Width attribute)
This input is present only if "separate load and store ports" is selected for the Data Interface attribute. When a store is requested (via the clock changing from 0 to 1 while *sel* and *str* are both 1 or floating), the value found at this port is stored into memory at the currently selected address.

D on east edge (input/output or output, bit width matches Data Bit Width attribute)
If *sel* and *ld* are 1 or floating, then the RAM component emits the value found at the currently selected address on this port. If there is a single load/store port, the value read from this port is stored whenever a store is requested.

str on south edge (input, bit width 1)
Store: This input is present only if "separate load and store ports" is selected for the Data Interface attribute. When it is 1 or floating, a clock pulse will result in storing the data found on the west edge into memory (provided the *sel* input is also 1 or floating).

sel on south edge (input, bit width 1)

Chip select: This input enables or disables the entire RAM module, based on whether the value is 1/floating or 0. The input is meant primarily for situations where you have multiple RAM units, only one of which would be enabled at any time.

Triangle on south edge (input, bit width 1)

Clock input: This is absent when the Data Interface attribute's value is "One asynchronous load/store port." In other circumstances, when *ld* is 0, and this input rises from 0 to 1 (and *sel* is 1/undefined and *clr* is 0), then the value at the currently selected address changes to whatever value is at the *D* pin. As long as the clock input remains 0 or 1, though, the *D* value will not be stored into memory.

ld on south edge (input, bit width 1)

Load: Selects whether the RAM should emit (on *D*) the value at the current address (*A*). This output behavior is enabled if *out* is 1 or undefined; if *out* is 0, then no value is pushed onto *D* - but if there is a combined load/store port, stores will be enabled.

clr on south edge (input, bit width 1)

Clear: When this is 1, all values in memory are pinned to 0, no matter what the other inputs are.

3.2.1.5 Attributes

When the component is selected or being added, the digits '0' through '9' alter its Address Bit Width attribute and Alt-0 through Alt-9 alter its Data Bit Width attribute.

Address Bit Width

The bit width of the address bits. The number of values stored in RAM is $2^{addrBitWidth}$.

Data Bit Width

The bit width of each individual value in memory.

Data Interface

Configures which of the three interfaces are used for communicating data into and out of the component.

3.2.2 ROM in LogiSim

The ROM component stores up to 16,777,216 values (specified in the Address Bit Width attribute), each of which can include up to to 32 bits (specified in the Data Bit Width attribute). A circuit can access the current values in ROM, but it cannot change them. The user can modify individual values interactively via the Poke Tool, or the user can modify the entire contents via the Menu Tool.

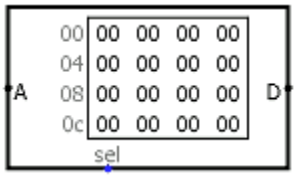


Figure 3.6: Appearance of ROM in LogiSim

Unlike the RAM component, the ROM component's current contents are stored as an attribute of the component. Thus, if a circuit containing a ROM component is used twice, then both ROM components will hold the same values. Also because of this behavior, the current ROM contents are stored in files created by Logisim.

Current values are displayed in the component. Addresses displayed are listed in gray to the left of the display area. Inside, each value is listed using hexadecimal. The value at the currently selected address will be displayed in inverse text (white on black).

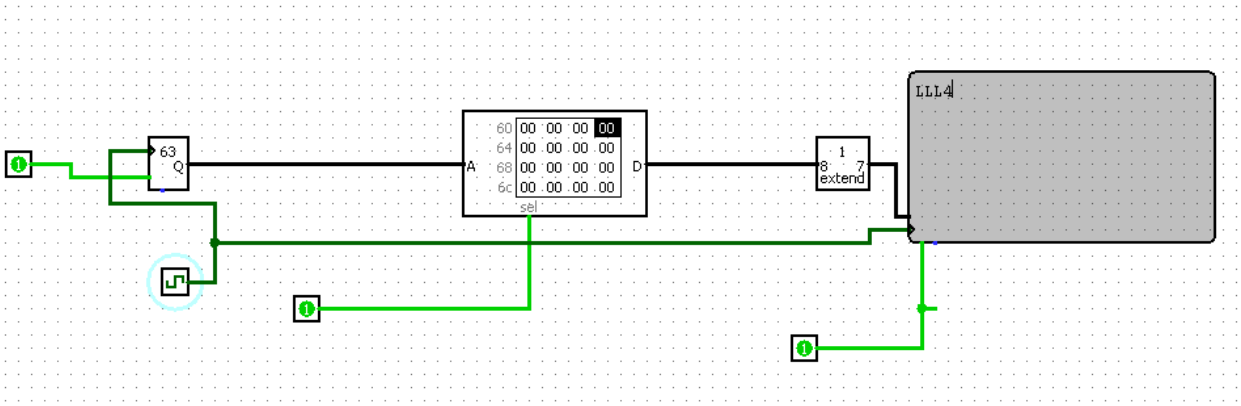


Figure 3.7: 8 Bit ROM in LogiSim

3.2.2.1 PINS

A on west edge (input, bit width matches Address Bit Width attribute)

Selects which of the values are currently being accessed by the circuit.

D on east edge (input/output, bit width matches Data Bit Width attribute)

Outputs the value at the currently selected address at the *D* pin if *sel* is 1 or floating. If *sel* is 0, then *D* will be floating.

sel on south edge (input, bit width 1)

If you have just one ROM module, ignore this input. If you have multiple ROM modules in parallel, you can use this input to enable or disable the entire ROM module, based on whether the value is 1 or 0. In other words, when this is 0, no value is emitted on the *D* output.

3.2.2.2 ATTRIBUTES

Address Bit Width

The bit width of the address bits. The number of values stored in RAM is $2^{addrBitWidth}$.

Data Bit Width

The bit width of each individual value in memory.

Contents

Stores the contents of memory.

3.3 Lab Activities

3.3.1 Task-1: Implement Figure: 3.5 in LogiSim

3.3.2 Task-2 Implement 32 Bit RAM in LogiSim using Random Generator and Counter Components for inputs and Bit Extender and TTY for output.

3.3.3 Task-3 Implement 32 Bit ROM in LogiSim using Random Generator and Counter Components for inputs and Bit Extender and TTY for output.

Lab Exercise and Summary

Summary should cover Introduction, Procedure, Data Analysis and Evaluation.

Student's Signature: _____ **Date: _____**

Cognitive Activity

Q1. A memory block has a 4-bit address bus. How many distinct locations can it address, and why? If you now need to store 64 locations, what is the minimum number of address bits required? **(5)**

Ans: _____

_____

_____

_____

_____

Q2. Explain, step by step, how you would connect a Counter and a Random Number Generator together with a RAM block in LogiSim so that a sequence of memory addresses is automatically filled with random values. **(5)**

Ans: _____

_____

_____

_____

_____

Obtained Marks	**/10**

Psychomotor Activity 1 — Experimental Setup

S. No	Criteria	Performance	Description	Marks
1	Tool Setup (4)	Level 4	Opens and configures LogiSim and selects the required environment correctly and independently.	4
		Level 3	Configures the required tool with minor errors or assistance.	3
		Level 2	Completes the setup with considerable assistance.	2
		Level 1	Setup is incomplete and requires substantial assistance.	1
2	File/Design Setup (4)	Level 4	Correctly creates/selects the required assembly file, Logisim circuit/design and required components for the Logisim RAM/ROM design.	4
		Level 3	File/design setup is mostly correct with minor errors or omissions.	3
		Level 2	Setup is partially completed and requires assistance.	2
		Level 1	Major errors or omissions are present in the setup.	1
3	Parameter Setup (2)	Level 2	Correctly defines required data, registers, parameters, components and initial conditions.	2
		Level 1	Most required inputs/components are correctly configured with minor errors.	1
			Total Obtained	/10

If a student is absent:

Criteria	Description	Marks
Absent	Student is absent from the laboratory activity.	0

Psychomotor Activity 2 — Implementation + Results

S. No	Criteria	Performance	Description	Marks
1	**Implementation (4)**	Level 4	Implements the required Logisim RAM/ROM design correctly and independently.	**4**
		Level 3	Implements the required program/design with minor errors or assistance.	**3**
		Level 2	Partially implements the program/design and requires considerable assistance.	**2**
		Level 1	Implementation is largely incomplete and requires substantial assistance.	**1**
2	**Execution/Simulation (2)**	Level 2	Executes the RISC-V program or simulates the Logisim design correctly and handles basic errors appropriately.	**2**
		Level 1	Executes/simulates the work with minor errors or assistance.	**1**
3	**Results/Output (4)**	Level 4	Obtains correct results and clearly demonstrates the required program output or circuit behavior.	**4**
		Level 3	Obtains mostly correct results with minor errors.	**3**
		Level 2	Obtains partially correct results with noticeable errors or omissions.	**2**
		Level 1	Results are largely incorrect or incomplete.	**1**
			Total Obtained	**/10**

If a student is absent:

Criteria	Description	Marks
Absent	Student is absent from the laboratory activity.	**0**

Affective Activity — Lab Report + Viva Voce

S. No	Criteria	Performance	Description	Marks
1	Lab Report Completeness & Organization (4)	Level 4	Report is complete, well organized and demonstrates full responsibility for the assigned work.	4
		Level 3	Report is complete with minor omissions or organizational issues and demonstrates satisfactory responsibility for the assigned work.	3
		Level 2	Report is partially complete with several omissions and demonstrates limited responsibility for the assigned work.	2
		Level 1	Report has major omissions or poor organization and demonstrates minimal responsibility for the assigned work.	1
2	Code/Design, Results & Conclusion (2)	Level 2	Logisim RAM/ROM design, results, and conclusion are correctly presented and discussed, demonstrating ownership of the assigned work.	2
		Level 1	Logisim RAM/ROM design, results, or conclusion are presented with minor errors or omissions.	1
3	Viva Voce (4)	Level 4	Clearly explains the assigned Logisim RAM/ROM design, working, and results, demonstrating individual responsibility, participation, and effective communication.	4
		Level 3	Explains most of the assigned program/design and results correctly, with minor difficulty, and demonstrates good individual participation.	3
		Level 2	Explains basic aspects of the assigned program/design with some prompting and demonstrates basic participation.	2
		Level 1	Demonstrates limited understanding or involvement and requires substantial prompting to explain the assigned work.	1
			Total Obtained	/10

The student has not submitted the report and has not appeared in the Viva Voce.

Criteria	Description	Marks
Lab Report and Results Not Submitted	Student has not submitted the report and results file.	0
Absent in Viva Voce	Student is absent from the viva voce.	0

Instructor's Signature: _____ **Date: _____**

Experiment No 4

RISC-V assembly programming using Data Transfer and Arithmetic Instructions

4.1 Objectives

After completing this experiment, student will be able to:

- Gain proficiency in using data transfer instructions (**lw** and **sw**) to move data between memory and registers.
- Understand the syntax and functionality of data transfer instructions in RISC-V assembly programming.
- Explore arithmetic instructions (**add**, **sub**, **mul**, and **div**) for performing basic arithmetic operations on registers.
- Learn how to specify memory addresses and register operands in data transfer and arithmetic instructions.
- Practice writing RISC-V assembly code to load data from memory, perform arithmetic operations, and store results back into memory.
- Develop skills in debugging RISC-V assembly programs using the Venus IDE, including setting breakpoints, inspecting register values, and verifying program correctness.
- Enhance understanding of memory organization, register manipulation, and program execution flow in RISC-V architecture.

4.2 Background Theory

This lab on "RISC-V assembly programming using Data Transfer and Arithmetic Instructions in Venus" aims to provide students with a foundational understanding of low-level programming concepts within the RISC-V architecture. RISC-V, as an open-source instruction set architecture, offers a versatile platform for learning assembly language programming. Through this lab, students will explore the fundamentals of data transfer and arithmetic operations, essential for manipulating data within a RISC-V environment. By leveraging instructions like **lw** and **sw** for memory operations and **add**, **sub**, **mul**, and **div** for arithmetic computations, students will gain hands-on experience in moving data between registers and memory, performing basic arithmetic operations, and storing results back into memory. This lab not only familiarizes students with the syntax and usage of these instructions but also cultivates a deeper understanding of memory organization, register manipulation, and the execution flow of RISC-V programs. Through practical exercises conducted in the Venus IDE, students will develop proficiency in writing efficient and effective

RISC-V assembly code, laying a solid groundwork for further exploration into computer architecture and systems programming.

4.2.1.1 RISC-V Architecture

RISC-V, an open-source instruction set architecture (ISA), embodies the principles of simplicity and modularity, making it highly adaptable across a wide range of computing devices. It features a clean and elegant design with a small set of fixed-length instructions, categorized into base and optional extensions, allowing for flexibility and scalability. The architecture defines a set of registers, including general-purpose registers and special-purpose registers, providing a foundation for efficient data manipulation and control flow. RISC-V supports various addressing modes and instruction formats, facilitating ease of programming and efficient code generation. Its modular design allows for the inclusion of custom extensions tailored to specific application domains, promoting innovation and specialization. Overall, RISC-V architecture stands as a versatile and extensible framework, poised to drive innovation in computing across diverse applications and industries.

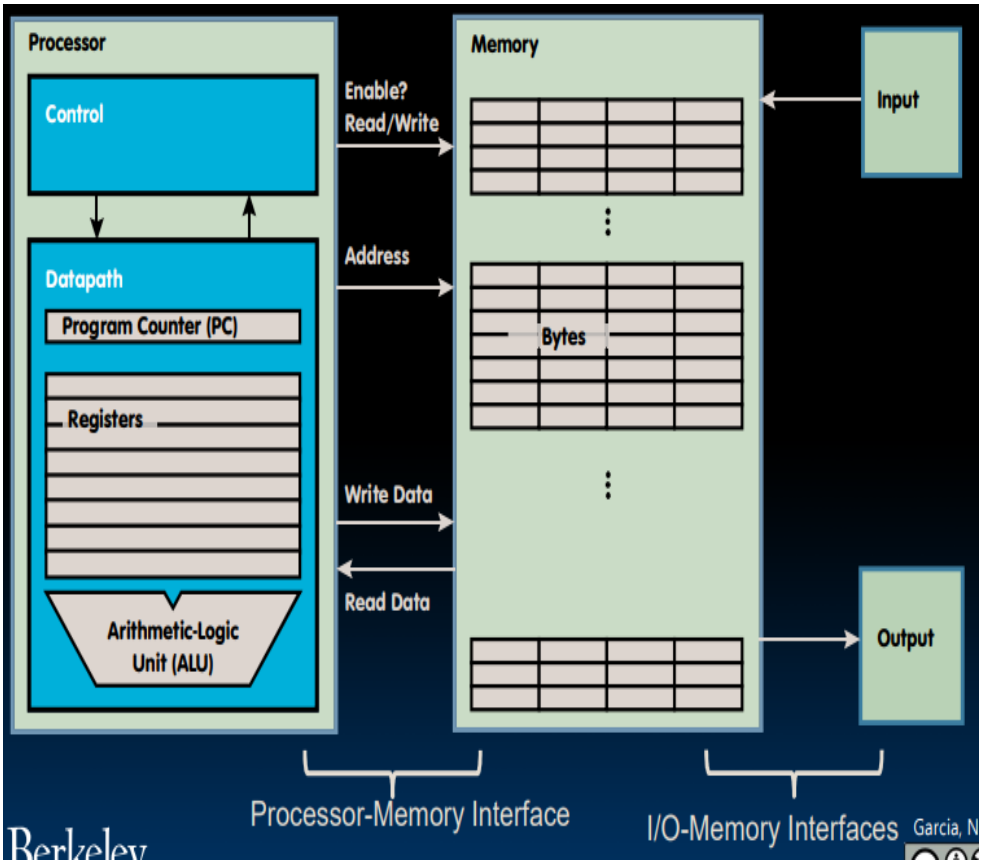


Figure 4.1: RISC-V Architecture

4.2.1.2 Basics of RISC-V assembly programming

Following are the foundational concepts and instructions for writing RISC-V assembly programs. By combining these elements, you can create programs to perform various computational tasks, manipulate data, control program flow, and interact with memory. Practice with these basics will help you gain proficiency in RISC-V assembly programming.

4.2.1.2.1 Registers

- RISC-V architecture includes 32 general-purpose registers (**x0** to **x31**) and a few special-purpose registers like the program counter (**pc**) and the stack pointer (**sp**).
- Example: Loading a value into a register and storing it in memory:

```assembly
li x1, 42      # Load immediate value 42 into register x1
sw x1, 0(x2)   # Store value in x1 into memory at address stored in x2
```

4.2.1.2.2 Data Transfer Instructions

- **lw** (load word) loads a 32-bit value from memory into a register.
- **sw** (store word) stores a 32-bit value from a register into memory.
- Example:

```
lw x3, 100(x4)   # Load value from memory into register x3
sw x5, 200(x6)   # Store value from register x5 into memory
```

4.2.1.2.3 Arithmetic Instructions

- **add** adds two registers and stores the result in another register.
- **sub** subtracts one register from another and stores the result in another register.
- Example:

```
add x7, x8, x9   # Add values in x8 and x9, store result in x7
sub x10, x11, x12 # Subtract value in x12 from x11, store result in x10
```

Multiplication:

- Load the first operand (**operand1**) into register **t0** using the **lw** instruction.
- Load the second operand (**operand2**) into register **t1** using the **lw** instruction.
- Multiply the values in registers **t0** and **t1** using the **mul** instruction, and store the result in register **t2**.
- Store the result from register **t2** into memory using the **sw** instruction.
- Example:

```
.data
    operand1:   .word 5      # Define operand1 as 5
    operand2:   .word 8      # Define operand2 as 8
    result:     .word 0      # Define a space to store the result

.text
    lw  t0, operand1   # Load operand1 into register t0
    lw  t1, operand2   # Load operand2 into register t1
    mul t2, t0, t1     # Multiply operand1 and operand2, store result in t2
    sw  t2, result     # Store the result in memory
```

Division:

- Load the dividend (**dividend**) into register **a0** using the **lw** instruction.
- Load the divisor (**divisor**) into register **a1** using the **lw** instruction.
- Divide the value in register **a0** by the value in register **a1** using the **div** instruction. The quotient is stored in the special register **lo**.
- Move the quotient from the special register **lo** to a general-purpose register **t3** using the **mflo** instruction.
- Store the quotient from register **t3** into memory using the **sw** instruction.
- Example:

```
.data
    dividend:    .word 20      # Define dividend as 20
    divisor:     .word 5       # Define divisor as 5
    quotient:    .word 0       # Define a space to store the quotient

.text
    lw  a0, dividend    # Load dividend into register a0
    lw  a1, divisor     # Load divisor into register a1
    div a0, a1          # Divide dividend by divisor, quotient stored in a0
    mflo t3             # Move the quotient from lo register to t3
    sw  t3, quotient    # Store the quotient in memory
```

4.2.1.2.4 Labeling and Comments

- Labels are used to mark specific locations in the code, facilitating branching and jumping.
- Comments provide explanatory notes within the code.
- Example:

```
loop_start:
    addi x18, x18, 1    # Increment x18
    bne x18, x19, loop_start    # Branch back to loop_start if x18 != x19
```

4.2.1.2.5 Data Directives

- **.data** directive marks the beginning of the data section, where data is initialized.
- **.word** directive allocates memory for a 32-bit word and initializes it with specified values.
- Example:

```
.data
my_data: .word 42, 56, -10   # Define an array of three 32-bit integers
```

RISC-V operands

Name	Example	Comments
32 registers	x0-x31	Fast locations for data. In RISC-V, data must be in registers to perform arithmetic. Register x0 always equals 0.
2^{30} memory words	Memory[0], Memory[4], ..., Memory[4,294,967,292]	Accessed only by data transfer instructions. RISC-V uses byte addresses, so sequential word accesses differ by 4. Memory holds data structures, arrays, and spilled registers.

Figure 4.2: RISC-V Operands

RISC-V assembly language

Category	Instruction	Example	Meaning	Comments
Arithmetic	Add	add x5, x6, x7	x5 = x6 + x7	Three register operands; add
	Subtract	sub x5, x6, x7	x5 = x6 - x7	Three register operands; subtract
	Add immediate	addi x5, x6, 20	x5 = x6 + 20	Used to add constants
Data transfer	Load word	lw x5, 40(x6)	x5 = Memory[x6 + 40]	Word from memory to register
	Load word, unsigned	lwu x5, 40(x6)	x5 = Memory[x6 + 40]	Unsigned word from memory to register
	Store word	sw x5, 40(x6)	Memory[x6 + 40] = x5	Word from register to memory
	Load halfword	lh x5, 40(x6)	x5 = Memory[x6 + 40]	Halfword from memory to register
	Load halfword, unsigned	lhu x5, 40(x6)	x5 = Memory[x6 + 40]	Unsigned halfword from memory to register
	Store halfword	sh x5, 40(x6)	Memory[x6 + 40] = x5	Halfword from register to memory
	Load byte	lb x5, 40(x6)	x5 = Memory[x6 + 40]	Byte from memory to register
	Load byte, unsigned	lbu x5, 40(x6)	x5 = Memory[x6 + 40]	Byte unsigned from memory to register
	Store byte	sb x5, 40(x6)	Memory[x6 + 40] = x5	Byte from register to memory
	Load reserved	lr.d x5, (x6)	x5 = Memory[x6]	Load; 1st half of atomic swap
	Store conditional	sc.d x7, x5, (x6)	Memory[x6] = x5; x7 = 0/1	Store; 2nd half of atomic swap
	Load upper immediate	lui x5, 0x12345	x5 = 0x12345000	Loads 20-bit constant shifted left 12 bits
Logical	And	and x5, x6, x7	x5 = x6 & x7	Three reg. operands; bit-by-bit AND
	Inclusive or	or x5, x6, x8	x5 = x6 | x8	Three reg. operands; bit-by-bit OR
	Exclusive or	xor x5, x6, x9	x5 = x6 ^ x9	Three reg. operands; bit-by-bit XOR
	And immediate	andi x5, x6, 20	x5 = x6 & 20	Bit-by-bit AND reg. with constant
	Inclusive or immediate	ori x5, x6, 20	x5 = x6 | 20	Bit-by-bit OR reg. with constant
	Exclusive or immediate	xori x5, x6, 20	x5 = x6 ^ 20	Bit-by-bit XOR reg. with constant
Shift	Shift left logical	sll x5, x6, x7	x5 = x6 << x7	Shift left by register
	Shift right logical	srl x5, x6, x7	x5 = x6 >> x7	Shift right by register
	Shift right arithmetic	sra x5, x6, x7	x5 = x6 >> x7	Arithmetic shift right by register
	Shift left logical immediate	slli x5, x6, 3	x5 = x6 << 3	Shift left by immediate
	Shift right logical immediate	srli x5, x6, 3	x5 = x6 >> 3	Shift right by immediate
	Shift right arithmetic immediate	srai x5, x6, 3	x5 = x6 >> 3	Arithmetic shift right by immediate
Conditional branch	Branch if equal	beq x5, x6, 100	if (x5 == x6) go to PC+100	PC-relative branch if registers equal
	Branch if not equal	bne x5, x6, 100	if (x5 != x6) go to PC+100	PC-relative branch if registers not equal
	Branch if less than	blt x5, x6, 100	if (x5 < x6) go to PC+100	PC-relative branch if registers less
	Branch if greater or equal	bge x5, x6, 100	if (x5 >= x6) go to PC+100	PC-relative branch if registers greater or equal
	Branch if less, unsigned	bltu x5, x6, 100	if (x5 < x6) go to PC+100	PC-relative branch if registers less, unsigned
	Branch if greater or equal, unsigned	bgeu x5, x6, 100	if (x5 >= x6) go to PC+100	PC-relative branch if registers greater or equal, unsigned
Unconditional branch	Jump and link	jal x1, 100	x1 = PC+4; go to PC+100	PC-relative procedure call
	Jump and link register	jalr x1, 100(x5)	x1 = PC+4; go to x5+100	Procedure return; indirect call

Figure 4.3: RISC-V Assembly Language

4.2.2 Venus IDE

Venus IDE provides a user-friendly environment for learning and practicing RISC-V assembly programming, including data transfer and arithmetic instructions. Here's an explanation of how Venus IDE facilitates RISC-V programming using these instructions:

4.2.2.1 Code Editor

Venus offers a built-in code editor where users can write RISC-V assembly code. The editor provides syntax highlighting, auto-indentation, and line numbering features, enhancing code readability and ease of writing.

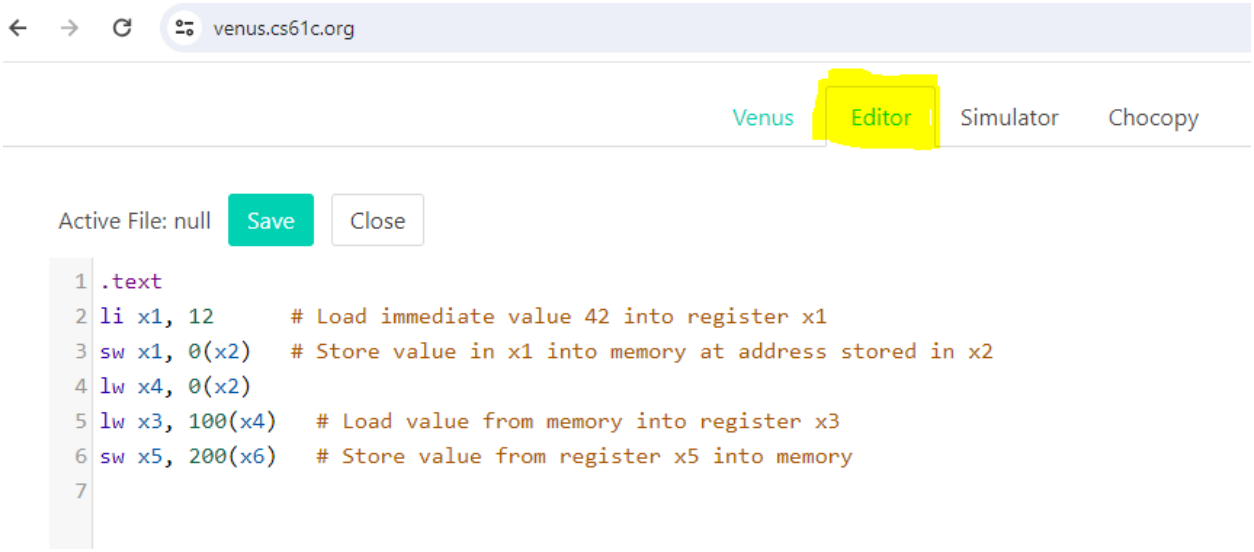


Figure 4.4: Editor

4.2.2.2 Assembler

Venus includes an assembler that translates RISC-V assembly code written in the editor into machine code. When the user clicks on the "Assemble" button, the assembler processes the code, checks for syntax errors, and generates the corresponding machine code.

4.2.2.3 Simulator

After assembling the code, Venus provides a simulation environment where users can execute their RISC-V programs step-by-step. The simulator allows users to observe the effects of each instruction on the CPU state, including register values, memory contents, and program counter (PC) changes.

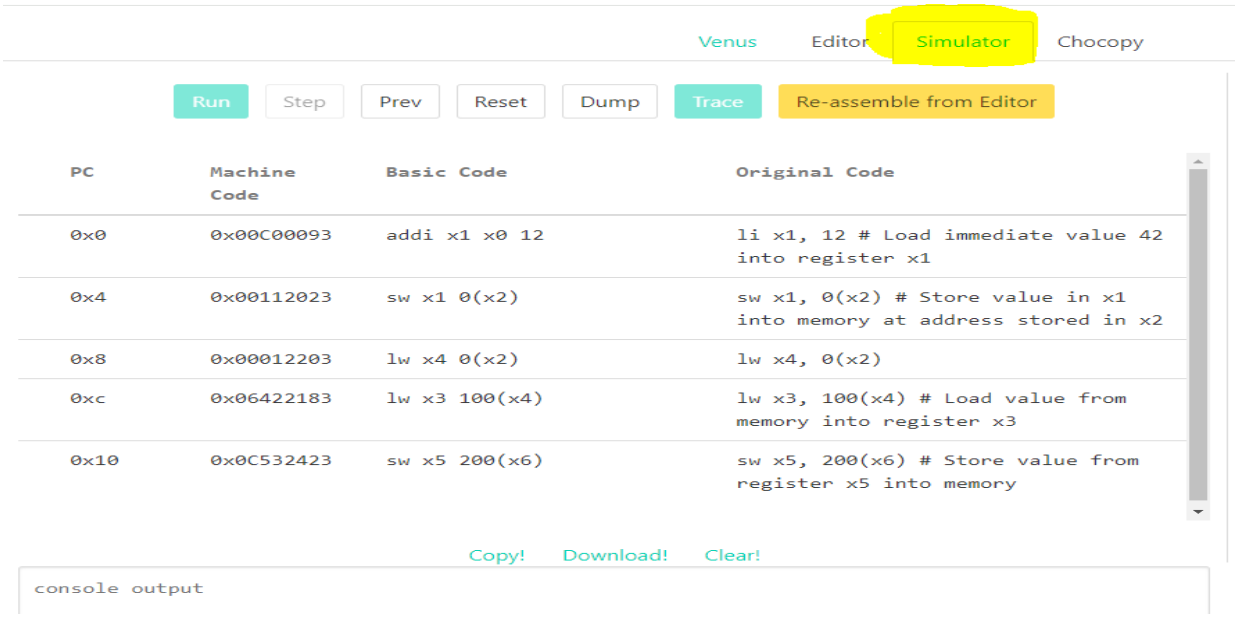


Figure 4.5: Simulator

4.2.2.4 Debugger

Venus comes equipped with a debugger that helps users troubleshoot and analyze their RISC-V programs. Users can set breakpoints, inspect register and memory values, and step through the code to identify logic errors or unexpected behavior.

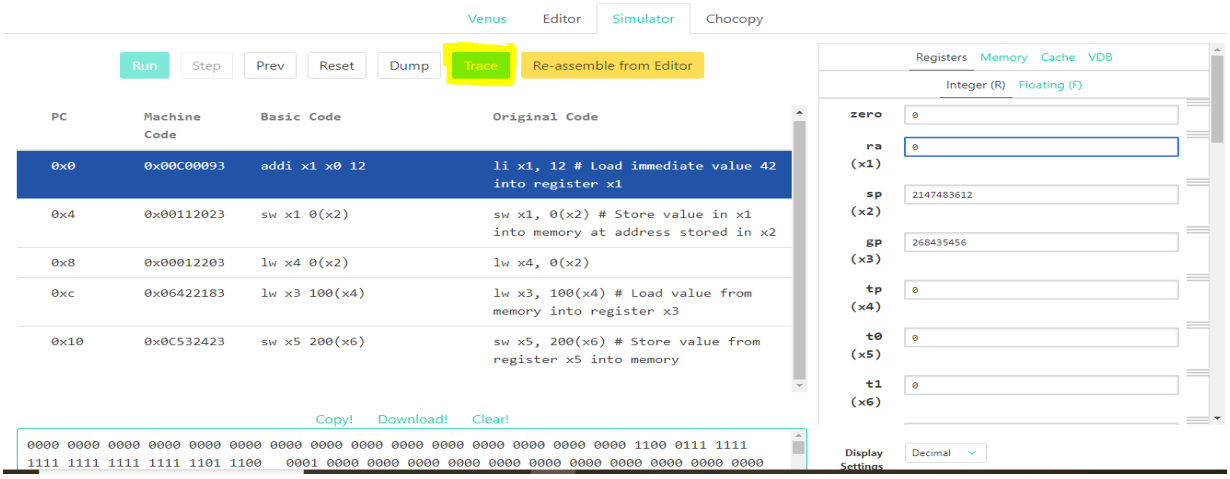


Figure 4.6: Trace

4.2.2.5 Memory Visualization

Venus provides a memory visualization feature that allows users to view the contents of memory locations as they execute their programs. This feature helps users track the data being transferred between memory and registers during data transfer instructions.

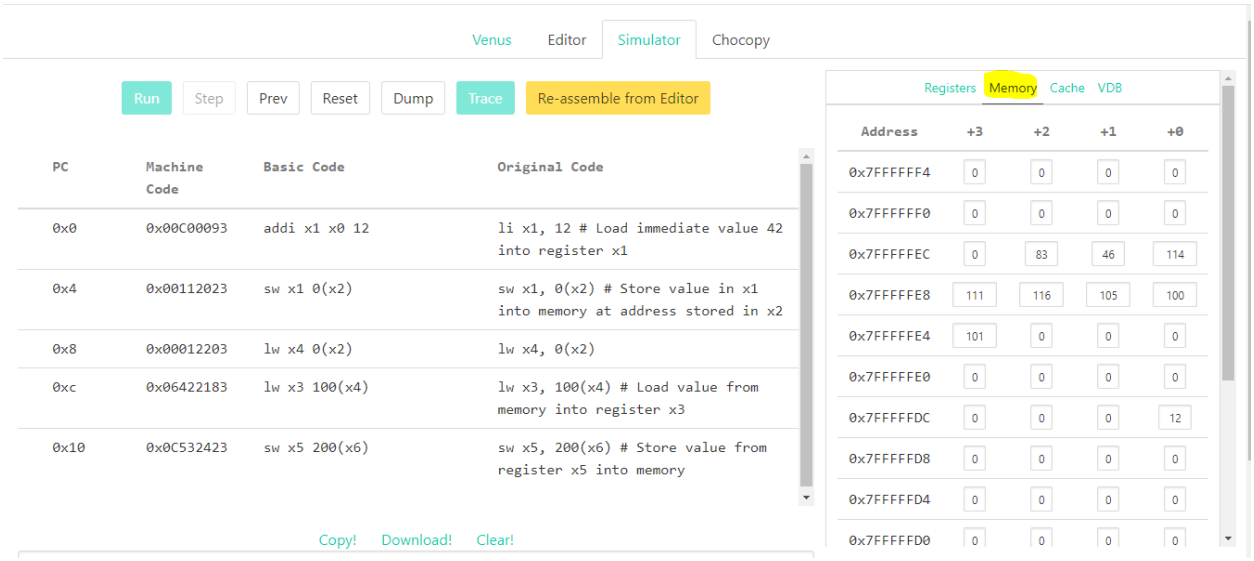


Figure 4.7: Memory Visualization

4.2.2.6 Register Display

Venus displays the current values of CPU registers, including general-purpose registers (x0 to x31) and special-purpose registers (pc, sp, etc.). Users can monitor register values to verify the correctness of arithmetic operations and data transfers.

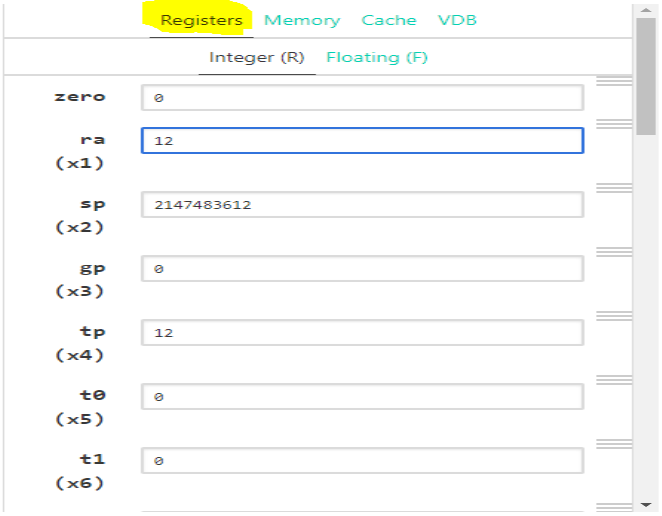


Figure 4.1: Register Display

4.2.2.7 Example Programs

Venus includes a collection of example programs demonstrating various RISC-V assembly programming concepts, including data transfer and arithmetic instructions. Users can study these examples to understand how to use these instructions effectively in their own programs.

4.3 Lab Activities

4.3.1 Task-1 Write a RISC-V assembly program to manipulate data using data transfer instructions in the Venus IDE. Your program should perform the following tasks:

1. Load an array of integers from memory into registers using the **lw** instruction.
2. Calculate the sum of all integers in the array and store the result in a designated register.
3. Store the computed sum back into memory at a specific memory location using the **sw** instruction.

Your task is to write the RISC-V assembly code to accomplish the above objectives and test it using the Venus IDE. Execute the program in the Venus simulator, and observe the values loaded from memory, the computed sum, and the stored result in registers.

Note: Add Screen shots of outputs and show it to instructor.

4.3.2 Task-2 Develop a RISC-V assembly program to demonstrate the use of arithmetic instructions in the Venus IDE. The program should accomplish the following objectives:

1. Perform addition and subtraction operations on predetermined values stored in registers using the **add** and **sub** instructions.
2. Implement multiplication and division operations between specified registers using the **mul** and **div** instructions.
3. Verify the correctness of the arithmetic operations by inspecting the results stored in registers.

4.3.3 Task-3 Develop a RISC-V assembly program in the Venus IDE that combines arithmetic and data transfer instructions to perform a simple computational task. The program should achieve the following objectives:

1. Load two integers from memory into registers using the lw instruction.
2. Perform an arithmetic operation (addition, subtraction, multiplication, or division) between the two integers using the corresponding arithmetic instruction (add, sub, mul, or div).
3. Store the result of the arithmetic operation back into memory at a specified memory location using the sw instruction.

Note: Show your outputs to your instructor.

Lab Exercise and Summary

Summary should cover Introduction, Procedure, Data Analysis and Evaluation.

Student's Signature: _____ **Date: _____**

Cognitive Activity

Answer both questions. Each question carries 5 marks.

Q1. Trace the following RISC-V code by hand and give the final value in register t2:

```
li t0, 5
li t1, 10
add t2, t0, t1
sw t2, 0(sp)
lw t3, 0(sp)
sub t2, t3, t0
```

(5)

Ans: _____

_____

_____

_____

_____

Q2. Write a short RISC-V assembly snippet (using lw, mul and sw) that loads two numbers from memory addresses 0(s0) and 4(s0), multiplies them, and stores the result back at address 8(s0). Explain each line briefly. **(5)**

Ans: _____

_____

_____

_____

_____

Obtained Marks	**/10**

Psychomotor Activity 1 — Experimental Setup

S. No	Criteria	Performance	Description	Marks
1	Tool Setup (4)	Level 4	Opens and configures Venus and selects the required environment correctly and independently.	4
		Level 3	Configures the required tool with minor errors or assistance.	3
		Level 2	Completes the setup with considerable assistance.	2
		Level 1	Setup is incomplete and requires substantial assistance.	1
2	File/Design Setup (4)	Level 4	Correctly creates/selects the required assembly file, Logisim circuit/design and required components for the RISC-V assembly program.	4
		Level 3	File/design setup is mostly correct with minor errors or omissions.	3
		Level 2	Setup is partially completed and requires assistance.	2
		Level 1	Major errors or omissions are present in the setup.	1
3	Parameter Setup (2)	Level 2	Correctly defines required data, registers, parameters, components and initial conditions.	2
		Level 1	Most required inputs/components are correctly configured with minor errors.	1
			Total Obtained	/10

If a student is absent:

Criteria	Description	Marks
Absent	Student is absent from the laboratory activity.	0

Psychomotor Activity 2 — Implementation + Results

S. No	Criteria	Performance	Description	Marks
1	**Implementation (4)**	Level 4	Implements the required RISC-V assembly program correctly and independently.	**4**
		Level 3	Implements the required program/design with minor errors or assistance.	**3**
		Level 2	Partially implements the program/design and requires considerable assistance.	**2**
		Level 1	Implementation is largely incomplete and requires substantial assistance.	**1**
2	**Execution/Simulation (2)**	Level 2	Executes the RISC-V program or simulates the Logisim design correctly and handles basic errors appropriately.	**2**
		Level 1	Executes/simulates the work with minor errors or assistance.	**1**
3	**Results/Output (4)**	Level 4	Obtains correct results and clearly demonstrates the required program output or circuit behavior.	**4**
		Level 3	Obtains mostly correct results with minor errors.	**3**
		Level 2	Obtains partially correct results with noticeable errors or omissions.	**2**
		Level 1	Results are largely incorrect or incomplete.	**1**
			Total Obtained	**/10**

If a student is absent:

Criteria	Description	Marks
Absent	Student is absent from the laboratory activity.	**0**

Affective Activity — Lab Report + Viva Voce

S. No	Criteria	Performance	Description	Marks
1	Lab Report Completeness & Organization (4)	Level 4	Report is complete, well organized and demonstrates full responsibility for the assigned work.	4
		Level 3	Report is complete with minor omissions or organizational issues and demonstrates satisfactory responsibility for the assigned work.	3
		Level 2	Report is partially complete with several omissions and demonstrates limited responsibility for the assigned work.	2
		Level 1	Report has major omissions or poor organization and demonstrates minimal responsibility for the assigned work.	1
2	Code/Design, Results & Conclusion (2)	Level 2	RISC-V assembly program, results, and conclusion are correctly presented and discussed, demonstrating ownership of the assigned work.	2
		Level 1	RISC-V assembly program, results, or conclusion are presented with minor errors or omissions.	1
3	Viva Voce (4)	Level 4	Clearly explains the assigned RISC-V assembly program, working, and results, demonstrating individual responsibility, participation, and effective communication.	4
		Level 3	Explains most of the assigned program/design and results correctly, with minor difficulty, and demonstrates good individual participation.	3
		Level 2	Explains basic aspects of the assigned program/design with some prompting and demonstrates basic participation.	2
		Level 1	Demonstrates limited understanding or involvement and requires substantial prompting to explain the assigned work.	1
			Total Obtained	/10

The student has not submitted the report and has not appeared in the Viva Voce.

Criteria	Description	Marks
Lab Report and Results Not Submitted	Student has not submitted the report and results file.	**0**
Absent in Viva Voce	Student is absent from the viva voce.	**0**

Instructor's Signature: _____ **Date:** _____

Experiment No 5

RISC-V assembly programming using Logic and Jump instructions in venus

5.1 Objectives:

After completing this experiment, student will be able:

- To know the RISC-V architecture and its significance in contemporary computer systems by providing an overview of its simple yet powerful design principles.
- To know fundamental concepts in RISC-V assembly programming, focusing specifically on logic and jump instructions, which are essential for controlling program flow and performing bitwise operations.
- To gain practical experience in using logic instructions such as AND, OR, XOR, and NOT to manipulate data at the bitwise level, thereby enhancing their understanding of low-level programming concepts.
- Hands-on learning through the utilization of Venus, a RISC-V simulator, allowing students to experiment with logic and jump instructions in a simulated environment, reinforcing theoretical knowledge with practical application and promoting deeper comprehension of computer architecture principles.

5.2 Background Theory

Understanding RISC-V assembly programming is foundational for comprehending computer architecture and low-level system programming. Logic and jump instructions play crucial roles in controlling program flow and performing bitwise operations in RISC-V assembly. Logic instructions such as AND, OR, XOR, and NOT facilitate bitwise manipulation of data, while jump instructions like unconditional jumps and conditional branches enable the execution of code

Conditional branch	Branch if equal	`beq x5, x6, 100`	`if (x5 == x6) go to PC+100`	PC-relative branch if registers equal
	Branch if not equal	`bne x5, x6, 100`	`if (x5 != x6) go to PC+100`	PC-relative branch if registers not equal
	Branch if less than	`blt x5, x6, 100`	`if (x5 < x6) go to PC+100`	PC-relative branch if registers less
	Branch if greater or equal	`bge x5, x6, 100`	`if (x5 >= x6) go to PC+100`	PC-relative branch if registers greater or equal
	Branch if less, unsigned	`bltu x5, x6, 100`	`if (x5 < x6) go to PC+100`	PC-relative branch if registers less, unsigned
	Branch if greater or equal, unsigned	`bgeu x5, x6, 100`	`if (x5 >= x6) go to PC+100`	PC-relative branch if registers greater or equal, unsigned
Unconditional branch	Jump and link	`jal x1, 100`	`x1 = PC+4; go to PC+100`	PC-relative procedure call
	Jump and link register	`jalr x1, 100(x5)`	`x1 = PC+4; go to x5+100`	Procedure return; indirect call

Figure 5.1: Conditional and Unconditional jumps/branch

segments based on specific conditions. Venus, a popular RISC-V simulator, provides a conducive environment for learning and experimenting with RISC-V assembly programming concepts,

allowing students and researchers to gain practical insights into computer organization and programming paradigms. This lab aims to provide hands-on experience with logic and jump instructions in RISC-V assembly using Venus, thereby fostering a deeper understanding of fundamental concepts in computer architecture and assembly programming.

5.2.1 Overview of Logical Instructions

In RISC-V assembly programming, logical instructions perform bitwise operations on binary data. These instructions include AND, OR, XOR, and NOT. Below, I'll explain each type of logical instruction with proper examples in RISC-V assembly programming using the Venus simulator:

5.2.1.1 AND (Bitwise AND)

The AND instruction performs a bitwise AND operation between corresponding bits of two operands.

```
# RISC-V Assembly code for bitwise AND operation

# Initialize registers
li a0, 5     # Load immediate value 5 into register a0 (101 in binary)
li a1, 3     # Load immediate value 3 into register a1 (011 in binary)

# Perform AND operation
and a2, a0, a1  # Perform bitwise AND of a0 and a1, store result in a2

# a2 will contain the result: 101 AND 011 = 001 (decimal 1)
```

Figure 5.2: AND Logic

5.2.1.2 OR (Bitwise OR)

The OR instruction performs a bitwise OR operation between corresponding bits of two operands.

```
# RISC-V Assembly code for bitwise OR operation

# Initialize registers
li a0, 5     # Load immediate value 5 into register a0 (101 in binary)
li a1, 3     # Load immediate value 3 into register a1 (011 in binary)

# Perform OR operation
or a2, a0, a1  # Perform bitwise OR of a0 and a1, store result in a2

# a2 will contain the result: 101 OR 011 = 111 (decimal 7)
```

Figure 5.3: OR Logic

5.2.1.3 XOR (Bitwise XOR)

The XOR instruction performs a bitwise XOR (exclusive OR) operation between corresponding bits of two operands.

```
# RISC-V Assembly code for bitwise XOR operation

# Initialize registers
li a0, 5     # Load immediate value 5 into register a0 (101 in binary)
li a1, 3     # Load immediate value 3 into register a1 (011 in binary)

# Perform XOR operation
xor a2, a0, a1  # Perform bitwise XOR of a0 and a1, store result in a2

# a2 will contain the result: 101 XOR 011 = 110 (decimal 6)
```

Figure 5.4: XOR Logic

5.2.1.4 NOT (Bitwise NOT)

The NOT instruction performs a bitwise negation (complement) operation on a single operand.

```
# RISC-V Assembly code for bitwise NOT operation

# Initialize registers
li a0, 5     # Load immediate value 5 into register a0 (101 in binary)

# Perform NOT operation
not a1, a0  # Perform bitwise NOT of a0, store result in a1

# a1 will contain the result: NOT 101 = 010 (decimal 2)
```

Figure 5.5: NOT Logic

5.2.2 Overview of Jump and Branch Instructions

In RISC-V assembly programming, jump and branch instructions are used to control the flow of execution within a program. Let's explain each type of jump and branch instruction separately with examples in RISC-V assembly programming using Venus.

5.2.2.1 Unconditional Jump

Unconditional jump instructions cause the control flow to transfer to a specific target address unconditionally.

Example:

```
# RISC-V Assembly code for unconditional jump

# Jump to label 'target'
j target

# Some code that will be skipped due to the jump
li a1, 42

target:
# Code continues here
li a0, 10     # Load immediate value 10 into register a0

# Exit
li a7, 10     # Load immediate value 10 (exit syscall number)
ecall         # Perform exit syscall
```

Figure 5.6: Unconditional Jump

5.2.2.2 Conditional Branch

Conditional branch instructions allow the program to change the flow of execution based on specified conditions.

Example:

```
# Initialize registers
li a0, 5     # Load immediate value 5 into register a0
li a1, 3     # Load immediate value 3 into register a1

# Compare values
bge a0, a1, greater_equal  # Branch if a0 >= a1

# If a0 < a1
li a0, 1     # Load immediate value 1 into register a0
j end        # Jump to end

greater_equal:
li a0, 0     # Load immediate value 0 into register a0

end:
# Code continues here

# Exit
li a7, 10    # Load immediate value 10 (exit syscall number)
ecall        # Perform exit syscall
```

Figure 5.7: Conditional Branch

5.2.2.3 Jump and Link (jal)

The **jal** instruction jumps to a target address and saves the address of the instruction following the jump (return address) into a specified register.

Example:

```
# RISC-V Assembly code for jump and link

# Jump to label 'target' and save return address in ra
jal ra, target

# Some code that will be skipped due to the jump
li a1, 42

target:
# Code continues here
li a0, 10     # Load immediate value 10 into register a0

# Exit
li a7, 10     # Load immediate value 10 (exit syscall number)
ecall         # Perform exit syscall
```

Figure 5.8: Jump and Link (jal)

5.2.2.4 Jump Register (jalr)

The **jalr** instruction jumps to an address stored in a register and saves the address of the instruction following the jump (return address) into a specified register.

Example:

```
# RISC-V Assembly code for jump register

# Load target address into t0
lui t0, %hi(target)
addi t0, t0, %lo(target)

# Jump to address in t0 and save return address in ra
jalr ra, t0, 0

# Some code that will be skipped due to the jump
li a1, 42

target:
# Code continues here
li a0, 10     # Load immediate value 10 into register a0

# Exit
li a7, 10     # Load immediate value 10 (exit syscall number)
ecall         # Perform exit syscall
```

Figure 5.9: Jump Register

These examples demonstrate the usage of various jump and branch instructions in RISC-V assembly programming using the Venus simulator. Each instruction type serves specific purposes in controlling program flow and enabling conditional execution based on runtime conditions.

5.3 Lab Activities

5.3.2 Task-1: Write a RISC V Assembly code to implement and verify the logical operations of OR, AND, XOR and NOT:

5.3.3 Task-2: Write a RISC-V assembly code that incorporates unconditional jump instructions and observe their effects on program execution.

1) Write a RISC-V assembly program that demonstrates the usage of the unconditional jump instruction (**j**).

2) Include at least two sections of code separated by an unconditional jump instruction. The first section should contain some initialization or computation, and the second section should be the target of the jump instruction.

3) Execute the program in the simulator and observe the flow of execution. Pay attention to how the control flow transfers to the target section upon encountering the unconditional jump instruction.

4) Modify the program to include multiple unconditional jump instructions and observe how each jump affects the flow of execution.

5.3.4 Task-3: Write a RISC-V assembly code that implements conditional instructions to achieve specific tasks. For example:

1) Implement a program that compares two numbers and prints a message indicating which number is greater.
2) Write a program that checks if a given value is positive, negative, or zero and outputs the result accordingly.

Lab Exercise and Summary

Summary should cover Introduction, Procedure, Data Analysis and Evaluation.

Student's Signature: _____ **Date: _____**

Cognitive Activity

Q1. Trace this snippet by hand, given t2 starts at 0:

```
li t0, 4
li t1, 4
beq t0, t1, label
addi t2, t2, 1
label:
addi t2, t2, 5
```

What is the final value of t2, and why is the 'addi t2,t2,1' line skipped or not skipped? **(5)**

Ans: _____

_____

_____

_____

_____

Q2. Using AND, OR/branch-type instructions, describe (in RISC-V pseudo-code or plain steps) how you would check whether the value in register t0 is even or odd, and store 1 in t1 if even or 0 if odd. Explain your logic. **(5)**

Ans: _____

_____

_____

_____

_____

Obtained Marks	**/10**

Psychomotor Activity 1 — Experimental Setup

S. No	Criteria	Performance	Description	Marks
1	Tool Setup (4)	Level 4	Opens and configures Venus and selects the required environment correctly and independently.	4
		Level 3	Configures the required tool with minor errors or assistance.	3
		Level 2	Completes the setup with considerable assistance.	2
		Level 1	Setup is incomplete and requires substantial assistance.	1
2	File/Design Setup (4)	Level 4	Correctly creates/selects the required assembly file, Logisim circuit/design and required components for the RISC-V assembly program.	4
		Level 3	File/design setup is mostly correct with minor errors or omissions.	3
		Level 2	Setup is partially completed and requires assistance.	2
		Level 1	Major errors or omissions are present in the setup.	1
3	Parameter Setup (2)	Level 2	Correctly defines required data, registers, parameters, components and initial conditions.	2
		Level 1	Most required inputs/components are correctly configured with minor errors.	1
			Total Obtained	/10

If a student is absent:

Criteria	Description	Marks
Absent	Student is absent from the laboratory activity.	0

Psychomotor Activity 2 — Implementation + Results

S. No	Criteria	Performance	Description	Marks
1	**Implementation (4)**	Level 4	Implements the required RISC-V assembly program correctly and independently.	**4**
		Level 3	Implements the required program/design with minor errors or assistance.	**3**
		Level 2	Partially implements the program/design and requires considerable assistance.	**2**
		Level 1	Implementation is largely incomplete and requires substantial assistance.	**1**
2	**Execution/Simulation (2)**	Level 2	Executes the RISC-V program or simulates the Logisim design correctly and handles basic errors appropriately.	**2**
		Level 1	Executes/simulates the work with minor errors or assistance.	**1**
3	**Results/Output (4)**	Level 4	Obtains correct results and clearly demonstrates the required program output or circuit behavior.	**4**
		Level 3	Obtains mostly correct results with minor errors.	**3**
		Level 2	Obtains partially correct results with noticeable errors or omissions.	**2**
		Level 1	Results are largely incorrect or incomplete.	**1**
			Total Obtained	**/10**

If a student is absent:

Criteria	Description	Marks
Absent	Student is absent from the laboratory activity.	**0**

Affective Activity — Lab Report + Viva Voce

S. No	Criteria	Performance	Description	Marks
1	Lab Report Completeness & Organization (4)	Level 4	Report is complete, well organized and demonstrates full responsibility for the assigned work.	4
		Level 3	Report is complete with minor omissions or organizational issues and demonstrates satisfactory responsibility for the assigned work.	3
		Level 2	Report is partially complete with several omissions and demonstrates limited responsibility for the assigned work.	2
		Level 1	Report has major omissions or poor organization and demonstrates minimal responsibility for the assigned work.	1
2	Code/Design, Results & Conclusion (2)	Level 2	RISC-V assembly program, results, and conclusion are correctly presented and discussed, demonstrating ownership of the assigned work.	2
		Level 1	RISC-V assembly program, results, or conclusion are presented with minor errors or omissions.	1
3	Viva Voce (4)	Level 4	Clearly explains the assigned RISC-V assembly program, working, and results, demonstrating individual responsibility, participation, and effective communication.	4
		Level 3	Explains most of the assigned program/design and results correctly, with minor difficulty, and demonstrates good individual participation.	3
		Level 2	Explains basic aspects of the assigned program/design with some prompting and demonstrates basic participation.	2
		Level 1	Demonstrates limited understanding or involvement and requires substantial prompting to explain the assigned work.	1
			Total Obtained	/10

The student has not submitted the report and has not appeared in the Viva Voce.

Criteria	Description	Marks
Lab Report and Results Not Submitted	Student has not submitted the report and results file.	**0**
Absent in Viva Voce	Student is absent from the viva voce.	**0**

Instructor's Signature: _____ **Date:** _____

Experiment No 6

Implementation of RISC-V Assembler

6.1 Objectives:

After completing this experiment, student will be able to:

- Convert Assembly language into Machine code.
- Explore different Instruction formats of RISC-V.
- Describe different fields i.e Fuc3, Func7, Opcode, rs1, and rs2 etc.

6.2 Background Theory

In the RISC-V architecture, machine code consists of binary instructions that the CPU can directly execute. Each instruction typically corresponds to a single operation or action performed by the CPU. Understanding the basics of RISC-V machine code involves knowing its instruction format, instruction types, and how assembly language instructions are translated into machine code.

Basics of RISC-V Machine Code:

6.2.1 Instruction Format

RISC-V instructions have a fixed-length encoding, which simplifies instruction decoding and execution. The instruction format includes fields for opcode, registers, immediate values, and other necessary information. The most common instruction formats in RISC-V are R-type (register), I-type (immediate), S-type (store), B-type (branch), U-type (upper immediate), and J-type (jump).

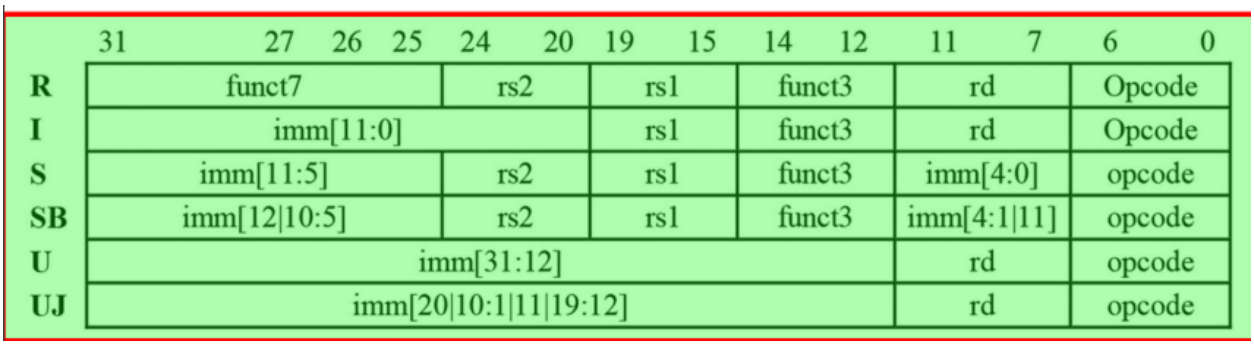
6.2.2 Instruction Types

RISC-V instructions can be categorized into various types based on their operations:

- Arithmetic and logical instructions (e.g., add, sub, and, or)
- Data transfer instructions (e.g., load, store)
- Control transfer instructions (e.g., branch, jump)
- System instructions (e.g., trap, syscall)

6.2.2 Instruction Encoding

Each RISC-V instruction is encoded into a fixed-length binary representation. The opcode field specifies the operation to be performed, while other fields encode register operands, immediate values, or memory addresses.



	31 27	26 25	24 20	19 15	14 12	11 7	6 0
R	funct7		rs2	rs1	funct3	rd	Opcode
I	imm[11:0]			rs1	funct3	rd	Opcode
S	imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode
SB	imm[12|10:5]		rs2	rs1	funct3	imm[4:1|11]	opcode
U	imm[31:12]					rd	opcode
UJ	imm[20|10:1|11|19:12]					rd	opcode

Figure 6.1: 4-Instruction Encoding

6.2.4 Conversion from Assembly Language to Machine Code:

The process of converting assembly language instructions into machine code involves several steps:

6.2.4.1 Parsing Assembly Instructions

The assembler parses assembly language instructions to identify the mnemonic operation (e.g., add, lw) and its operands (registers, immediate values, memory addresses).

6.2.4.2 Mapping Mnemonics to OpCodes

Each assembly language mnemonic corresponds to a specific opcode in the RISC-V instruction set. The assembler maps these mnemonics to their respective opcodes to determine the operation to be performed.

6.2.4.3 Encoding Registers and Immediate Values

Registers and immediate values specified in assembly instructions are encoded into binary representations according to the instruction format. Registers are typically represented by their register numbers, while immediate values are encoded directly or using sign-extension for signed values.

6.2.4.4 Generating Machine Code

Using the opcode and encoded operands, the assembler generates the binary machine code for each assembly instruction. It constructs the binary instruction by arranging the opcode and operand fields according to the instruction format.

6.2.4.5 Outputting Machine Code

Once all assembly instructions are translated into machine code, the assembler outputs the binary instructions in a format suitable for execution by the CPU. This could be in the form of a binary file, hexadecimal representation, or directly loaded into memory.

By following these steps, the assembler converts human-readable assembly language code into machine-executable binary instructions, allowing the CPU to execute the desired operations defined by the programmer.

6.2.5 Opcode

In computer architecture, an opcode, short for "operation code," is a fundamental component of machine language instructions. It specifies the operation to be performed by the CPU or processor. The opcode field of an instruction determines the type of operation or instruction that the CPU should execute.

Here are some key points about opcodes:

1. **Instruction Type:** Opcodes define the type of operation to be performed, such as arithmetic operations (add, subtract), logical operations (AND, OR), data transfer (load, store), control transfer (branch, jump), and more.

2. **Binary Representation:** Opcodes are represented as binary numbers in the machine code instructions. Each opcode corresponds to a unique binary pattern, which the CPU recognizes and interprets as a specific operation.

3. **Fixed Length:** In many instruction sets, including RISC-V, opcodes have fixed lengths. This simplifies the decoding process, as the CPU can quickly determine the type of operation without needing to analyze variable-length opcode fields.

4. **Instruction Set Architecture (ISA):** The opcode space is defined by the instruction set architecture (ISA) of a processor. Each ISA specifies a set of opcodes along with their corresponding operations and formats.

5. **Example:** In the RISC-V instruction set, the opcode field typically occupies the most significant bits of an instruction. For example, the opcode 0110011 corresponds to R-type instructions, which include arithmetic and logical operations.

So, the opcode is a crucial part of machine language instructions, as it directs the CPU to perform specific operations according to the program's requirements. It serves as a fundamental building block in the execution of programs on a computer system.

In the RISC-V instruction set architecture (ISA), different operations are represented by distinct opcodes. Here are some examples of opcodes for various operations in RISC-V:

1. **Arithmetic and Logical Operations (R-Type Instructions):**

- **add**: **0110011** (Binary)
- **sub**: **0110011**
- **and**: **0110011**
- **or**: **0110011**
- **xor**: **0110011**
- **slt** (Set Less Than): **0110011**
- **sltu** (Set Less Than Unsigned): **0110011**
- **sll** (Shift Left Logical): **0110011**
- **srl** (Shift Right Logical): **0110011**
- **sra** (Shift Right Arithmetic): **0110011**

2. **Data Transfer Operations (I-Type Instructions):**
    - **lw** (Load Word): **0000011** (Binary)
    - **sw** (Store Word): **0100011**
    - **lb** (Load Byte): **0000011**
    - **sb** (Store Byte): **0100011**
3. **Branch and Jump Operations (B-Type and J-Type Instructions):**
    - **beq** (Branch if Equal): **1100011** (Binary)
    - **bne** (Branch if Not Equal): **1100011**
    - **blt** (Branch if Less Than): **1100011**
    - **bge** (Branch if Greater Than or Equal): **1100011**
    - **jal** (Jump and Link): **1101111**
    - **jalr** (Jump and Link Register): **1100111**
4. **Immediate-Type Operations (I-Type Instructions):**
    - **addi** (Add Immediate): **0010011** (Binary)
    - **andi** (AND Immediate): **0010011**

- **ori** (OR Immediate): **0010011**
- **xori** (XOR Immediate): **0010011**
- **slti** (Set Less Than Immediate): **0010011**
- **sltiu** (Set Less Than Immediate Unsigned): **0010011**
- **slli** (Shift Left Logical Immediate): **0010011**
- **srli** (Shift Right Logical Immediate): **0010011**
- **srai** (Shift Right Arithmetic Immediate): **0010011**

5. **System Operations (System-Type Instructions):**
    - **ecall**: **1110011** (Binary)
    - **ebreak**: **1110011**
    - System calls and other privileged instructions have opcodes within the **1110011** opcode space.

These are just some examples of RISC-V opcodes for different operations. Each opcode corresponds to a specific operation or instruction, and they play a crucial role in the decoding and execution of instructions by the CPU in a RISC-V processor.

6.3 Understanding how to convert an assembly code into machine code

Let's walk through an example of converting a simple RISC-V assembly language instruction into machine code.

Example: Assembly to Machine Code Conversion

Consider the following RISC-V assembly instruction:

add x1, x2, x3

This instruction adds the values in registers x2 and x3 and stores the result in register x1.

Step 1: Parsing Assembly Instruction

- Operation: **add**
- Operands: **x1**, **x2**, **x3**

Step 2: Mapping Mnemonics to OpCodes

The **add** operation corresponds to the **add** opcode in the RISC-V instruction set.

Step 3: Encoding Registers

- Register x1: Encoded as **001** (binary representation of register number)
- Register x2: Encoded as **010**
- Register x3: Encoded as **011**

Step 4: Generating Machine Code

The RISC-V R-type instruction format for **add** instructions is as follows:

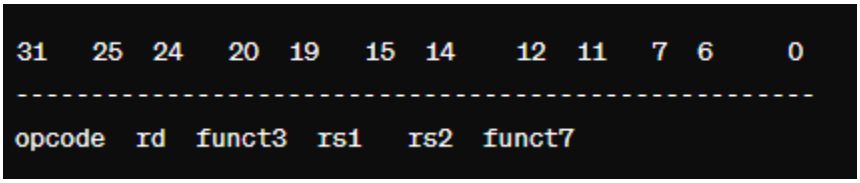


Figure 6.2: Machine Code Generation

- Opcode for **add**: **0110011**
- **rd** (destination register): Encoded register x1 (binary **001**)
- **rs1** (source register 1): Encoded register x2 (binary **010**)
- **rs2** (source register 2): Encoded register x3 (binary **011**)
- **funct3** for **add** instruction: **000**
- **funct7** for **add** instruction: **0000000**

Concatenating these fields together, the machine code instruction becomes:

0110011 00001 000 00010 00011 0000000

Step 5: Outputting Machine Code

The assembled machine code for the **add x1, x2, x3** instruction is **0x00A32333** in hexadecimal representation.

In this example, we converted the RISC-V assembly instruction **add x1, x2, x3** into its corresponding machine code **0x00A32333**. This process involves parsing the assembly instruction, mapping mnemonics to opcodes, encoding registers, generating the machine code according to the instruction format, and outputting the final machine code representation.

6.4 Conversion of Machine Code into Binary

The hexadecimal number 0x00A32333 represents a machine code instruction in RISC-V architecture. Let's break down this hexadecimal number into its binary representation and interpret its components:

Hexadecimal Representation:

0x00A32333

Binary Representation:

0000 0000 1010 0011 0010 0011 0011 0011

Interpretation:

The RISC-V instruction set architecture (ISA) typically divides machine code instructions into fields representing different parts of the instruction. Let's interpret the binary representation based on the standard RISC-V instruction formats.

Instruction Format:

RISC-V instructions can have different formats, such as R-type (register), I-type (immediate), S-type (store), B-type (branch), U-type (upper immediate), and J-type (jump). We'll need to identify the instruction format to interpret the binary representation accurately.

Interpretation Based on Opcode:

The opcode is the most significant part of the instruction and determines the operation to be performed. Let's extract the opcode from the binary representation.

In this case, the opcode appears to be 0000011. This opcode corresponds to the lw (Load Word) instruction in the RISC-V ISA.

Remaining Fields:

After the opcode, the binary representation contains fields for various components of the instruction, such as source and destination registers, immediate values, etc.

Since lw is an I-type instruction, it typically includes fields for the destination register (rd), base register (rs1), and immediate value (imm).

Interpreting the Load Word (lw) Instruction:

The lw instruction loads a 32-bit word from memory into a register.

The destination register (rd) is encoded as 00001.

The base register (rs1) is encoded as 00010.

The immediate value (imm) is encoded as 101000110010011.

Based on the interpretation, the machine code instruction 0x00A32333 corresponds to the RISC-V lw x1, 848(x2) instruction, where it loads a word from memory at address 848 plus the value in register x2 and stores it in register x1.

This breakdown illustrates how to interpret a hexadecimal machine code instruction in RISC-V and understand its components based on the RISC-V instruction format.

6.5 Lab Activities

6.5.4 Task-1: Convert at last three R-Type, S-Type and I-Type instructions into Machine code as well as explore its binary, Opcode, Functions, Operands and Destination Registers.

6.5.5 Task-2: Convert at least three B-Type and J-Type instructions into Machine code as well as explore its binary, Opcode, Functions, Operands and Destination Registers.

Lab Exercise and Summary

Summary should cover Introduction, Procedure, Data Analysis and Evaluation.

Student's Signature: _____ **Date: _____**

Cognitive Activity

Q1. Show how the R-type instruction 'add x3, x1, x2' would be broken into its instruction fields. List the fields in order (opcode, rd, funct3, rs1, rs2, funct7) and state what each one identifies for this instruction. **(5)**

Ans: _____

_____

_____

_____

_____

Q2. The instructions 'add' and 'sub' share the same opcode and the same funct3 value in RISC-V. Which field actually distinguishes them, and why do you think the ISA is designed this way instead of giving every instruction its own unique opcode? **(5)**

Ans: _____

_____

_____

_____

_____

Obtained Marks	**/10**

Psychomotor Activity 1 — Experimental Setup

S. No	Criteria	Performance	Description	Marks
1	Tool Setup (4)	Level 4	Opens and configures Venus and selects the required environment correctly and independently.	4
		Level 3	Configures the required tool with minor errors or assistance.	3
		Level 2	Completes the setup with considerable assistance.	2
		Level 1	Setup is incomplete and requires substantial assistance.	1
2	File/Design Setup (4)	Level 4	Correctly creates/selects the required assembly file, Logisim circuit/design and required components for the RISC-V machine code translation.	4
		Level 3	File/design setup is mostly correct with minor errors or omissions.	3
		Level 2	Setup is partially completed and requires assistance.	2
		Level 1	Major errors or omissions are present in the setup.	1
3	Parameter Setup (2)	Level 2	Correctly defines required data, registers, parameters, components and initial conditions.	2
		Level 1	Most required inputs/components are correctly configured with minor errors.	1
			Total Obtained	/10

If a student is absent:

Criteria	Description	Marks
Absent	Student is absent from the laboratory activity.	0

Psychomotor Activity 2 — Implementation + Results

S. No	Criteria	Performance	Description	Marks
1	**Implementation (4)**	Level 4	Implements the required RISC-V machine code translation correctly and independently.	**4**
		Level 3	Implements the required program/design with minor errors or assistance.	**3**
		Level 2	Partially implements the program/design and requires considerable assistance.	**2**
		Level 1	Implementation is largely incomplete and requires substantial assistance.	**1**
2	**Execution/Simulation (2)**	Level 2	Executes the RISC-V program or simulates the Logisim design correctly and handles basic errors appropriately.	**2**
		Level 1	Executes/simulates the work with minor errors or assistance.	**1**
3	**Results/Output (4)**	Level 4	Obtains correct results and clearly demonstrates the required program output or circuit behavior.	**4**
		Level 3	Obtains mostly correct results with minor errors.	**3**
		Level 2	Obtains partially correct results with noticeable errors or omissions.	**2**
		Level 1	Results are largely incorrect or incomplete.	**1**
			Total Obtained	**/10**

If a student is absent:

Criteria	Description	Marks
Absent	Student is absent from the laboratory activity.	**0**

Affective Activity — Lab Report + Viva Voce

S. No	Criteria	Performance	Description	Marks
1	Lab Report Completeness & Organization (4)	Level 4	Report is complete, well organized and demonstrates full responsibility for the assigned work.	4
		Level 3	Report is complete with minor omissions or organizational issues and demonstrates satisfactory responsibility for the assigned work.	3
		Level 2	Report is partially complete with several omissions and demonstrates limited responsibility for the assigned work.	2
		Level 1	Report has major omissions or poor organization and demonstrates minimal responsibility for the assigned work.	1
2	Code/Design, Results & Conclusion (2)	Level 2	RISC-V machine code translation, results, and conclusion are correctly presented and discussed, demonstrating ownership of the assigned work.	2
		Level 1	RISC-V machine code translation, results, or conclusion are presented with minor errors or omissions.	1
3	Viva Voce (4)	Level 4	Clearly explains the assigned RISC-V machine code translation, working, and results, demonstrating individual responsibility, participation, and effective communication.	4
		Level 3	Explains most of the assigned program/design and results correctly, with minor difficulty, and demonstrates good individual participation.	3
		Level 2	Explains basic aspects of the assigned program/design with some prompting and demonstrates basic participation.	2
		Level 1	Demonstrates limited understanding or involvement and requires substantial prompting to explain the assigned work.	1
			Total Obtained	**/10**

The student has not submitted the report and has not appeared in the Viva Voce.

Criteria	Description	Marks
Lab Report and Results Not Submitted	Student has not submitted the report and results file.	**0**
Absent in Viva Voce	Student is absent from the viva voce.	**0**

Instructor's Signature: _____ **Date:** _____

Experiment No 7

Design of Single Cycle Processor Data path I

7.1 Objectives:

After completing this experiment, student will be able to:

- Understand the basic functions of Data Path.
- Design a Register File in LogiSim.
- Design a complete ALU in LogiSim.

7.2 Background Theory

A data path is the collection of hardware components within a CPU that performs the actual data processing operations, including arithmetic, logic, memory access, and data transfer. To design a data path for executing RISC-V instructions, begin by identifying the major components required for each instruction type and their control signals, starting with the instruction memory, program counter (PC), and an adder to increment the PC. The memory unit stores program instructions, and the PC holds the address of the current instruction, incremented by the adder for the next instruction. These components form the initial part of the data path for fetching instructions. For R-format instructions, which perform arithmetic or logical operations on two registers and write the result to a third, we need a register file and an ALU. The register file, which stores the processor's 32 general-purpose registers, allows reading and writing by specifying register numbers. The ALU performs operations like addition or subtraction on the values read from the registers. Each R-format instruction requires reading two data words from the register file and writing one data word back, managed by specific control signals. The register file's inputs specify the registers to read and write, while the outputs carry the data. The ALU takes two 32-bit inputs, produces a 32-bit result, and outputs a signal if the result is zero, controlled by a 4-bit signal. Integrating these components ensures the data path can fetch, decode, and execute instructions correctly.

7.2.1 Designing of a Register File

Designing a register file in Logisim for a 16-bit single-cycle RISC-V processor involves creating a set of registers that can be read from and written to, controlled by the appropriate control signals.

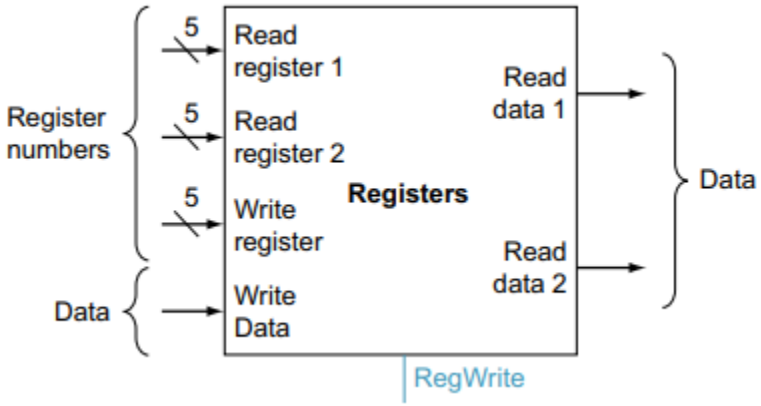


Figure 7.1: 16-Bit Register File Block Diagram

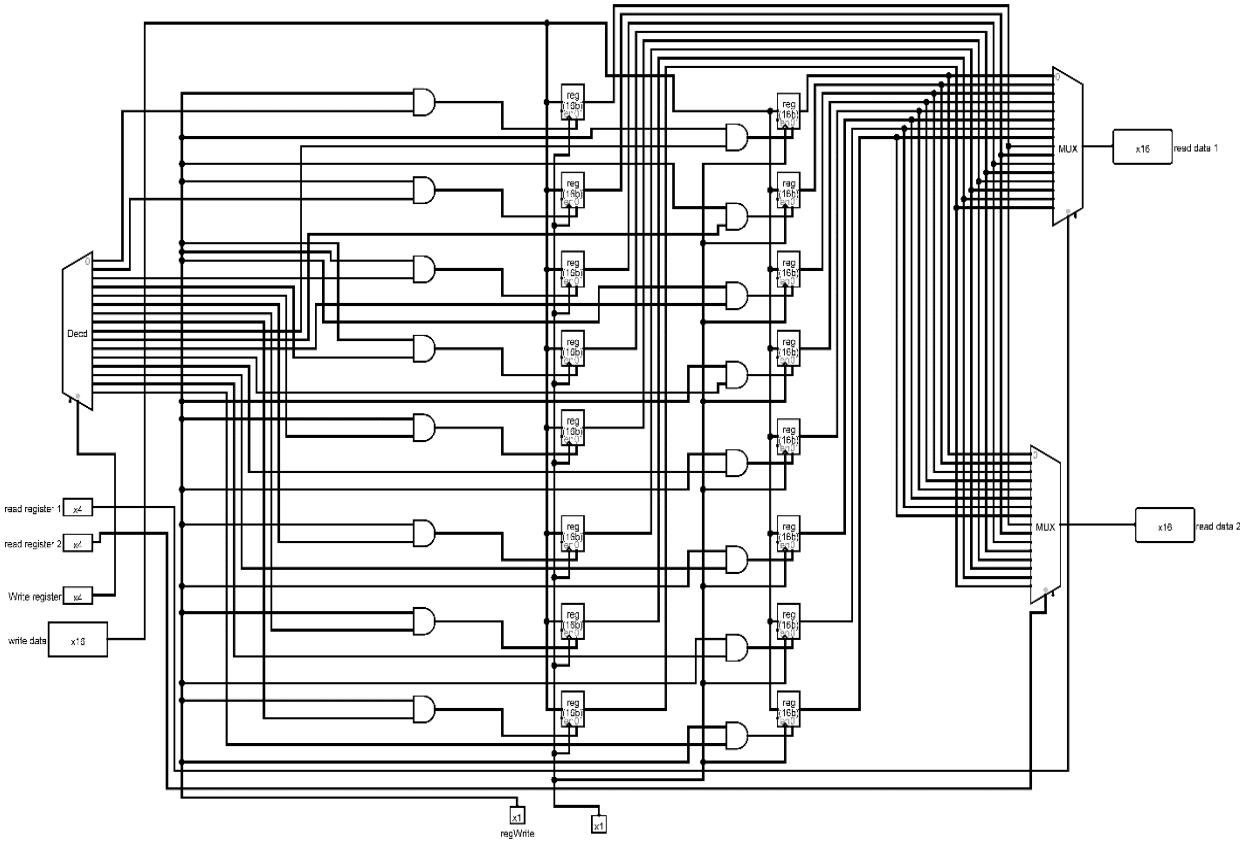


Figure 7.2: 16-Bit Register File Design in LogiSim

7.2.1.1 Step-by-Step Guide to Designing a Register File in Logisim.

1. **Open Logisim:**

- Launch Logisim and create a new project.

2. **Set Up the Circuit:**

- Create a new circuit within your project and name it `RegisterFile`.

3. **Add Registers:**

- Use the "Memory" component library.

- Select the "Register" component.

- Place 16 registers on the canvas to represent the 16-bit registers required for the RISC-V processor.

4. **Configure Registers:**

- Each register should be 16 bits wide. Double-click on each register to set its data width to 16 bits.

5. **Arrange the Registers:**

- Arrange the 16 registers in a column or row to make the design organized and clear.

6. **Create Input and Output Buses:**

- Use "Splitter" components to create input and output buses for addressing and data lines.

- Splitters will help manage the 16-bit data inputs and outputs and the 4-bit register address lines (since 2^4 = 16).

7. **Address Inputs:**

- Add two input pins for the read addresses (`ReadAddr1` and `ReadAddr2`) and one input pin for the write address (`WriteAddr`).

- These address inputs should be 4 bits wide each.

8. **Data Inputs and Outputs:**

- Add one input pin for the write data (`WriteData`), 16 bits wide.

- Add two output pins for the read data (`ReadData1` and `ReadData2`), each 16 bits wide.

9. **Control Signals:**

- Add an input pin for the write enable signal (`WriteEnable`), 1 bit wide.

10. **Connecting Components:**

- Connect the address input pins (`ReadAddr1`, `ReadAddr2`, and `WriteAddr`) to the splitters, and from the splitters to the address inputs of the respective registers.

- Connect the `WriteData` input pin to another splitter, and from there to the data inputs of all registers.

- Connect the `WriteEnable` input to each register's enable pin.

11. **Multiplexers for Read Outputs:**

- Use two 16-bit multiplexers (MUX) to select the output from the appropriate register based on the read addresses.

- Connect the outputs of all 16 registers to the inputs of the multiplexers.

- Connect the `ReadAddr1` and `ReadAddr2` inputs to the selection lines of the multiplexers.

- Connect the outputs of the multiplexers to the `ReadData1` and `ReadData2` output pins.

12. **Final Adjustments:**

- Ensure all connections are correct and labels are clear for easy understanding.

- Test the register file by creating a simple test circuit that writes and reads values from the registers.

7.2.2.2 Example Visual Layout:

- **Address Inputs:** Three input pins (`ReadAddr1`, `ReadAddr2`, `WriteAddr`), each connected to 4-bit splitters.

- **Write Data Input:** One 16-bit input pin (`WriteData`), connected to a 16-bit splitter.

- **Write Enable:** One 1-bit input pin (`WriteEnable`), connected to the enable pins of all registers.

- **Registers:** 16 registers, each 16 bits wide, connected to the address and data splitters.

- **Read Outputs:** Two 16-bit multiplexers, each selecting one of the 16 registers based on the read address inputs, and connected to `ReadData1` and `ReadData2` output pins.

By following these steps, you will create a functional register file in Logisim for a 16-bit single-cycle RISC-V processor, capable of reading and writing data as required by the processor's operations.

7.2.3 16 Bit ALU Design

Designing a 16-bit Arithmetic Logic Unit (ALU) with basic operations like AND, OR, XOR, addition, subtraction, and multiplication involves careful arrangement of logic components. In the following Figure: 2.2, two multiplexers (MUX) play a crucial role in managing the output and carry-out signals. For each operation, the ALU utilizes logic gates such as AND, OR, and XOR to process the input bits. The addition and subtraction operations employ a 16-bit full-adder for the arithmetic computations. To facilitate the selection of the desired operation, a control line is connected to the input of the MUX responsible for generating the output. Another MUX is incorporated to manage the carry-out signal. During addition or subtraction, the carry-out from the full-adder is selected, while for logical operations, the carry-out is set to zero. By implementing this design, the 16-bit ALU effectively performs various operations, and the multiplexers contribute to the flexibility of choosing the appropriate results based on the operation selected.

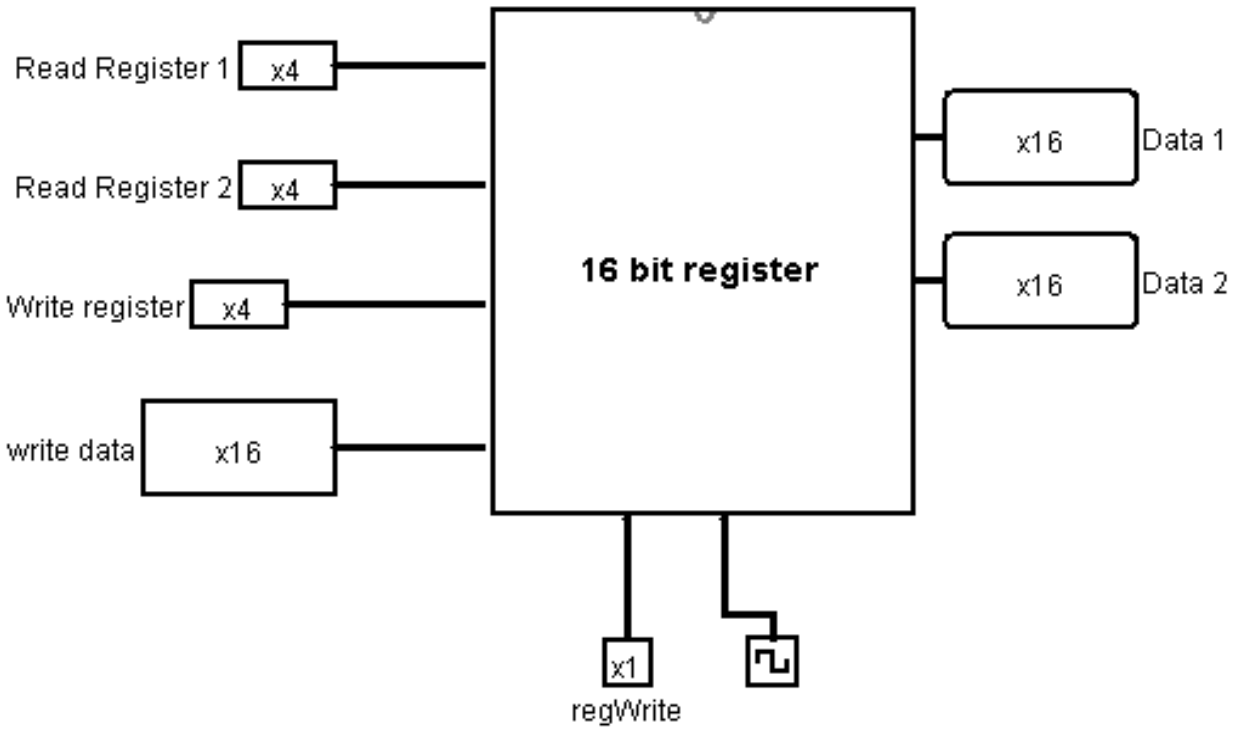


Figure 7.3: 16-Bit ALU Design in LogiSim

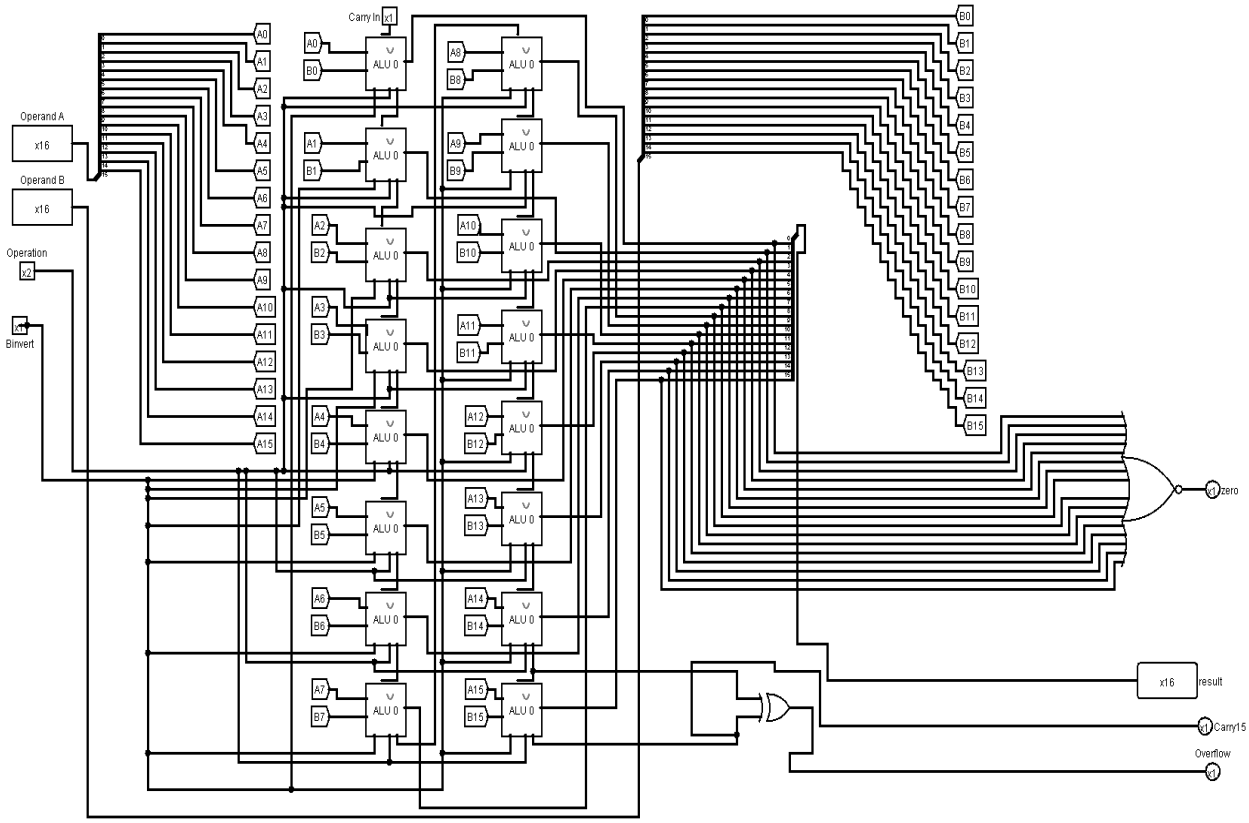


Figure 7.4: 16-Bit ALU Internal Design

7.3 Lab Activities

7.3.1 Task-1: Design and Implement a complete 16-Bit ALU in LogiSim.

7.3.2 Task-2: Design and Implement 16-Bit Register File in LogiSim.

Lab Exercise and Summary

Summary should cover Introduction, Procedure, Data Analysis and Evaluation.

Student's Signature: _____ **Date: _____**

Cognitive Activity

Q1. For the instruction 'add x5, x6, x7' passing through your data path, trace step by step: which register-file read ports are used, what values come out of them, what the ALU does with those values, and which port is used to write back the result. **(5)**

Ans: _____

_____

_____

_____

_____

Q2. Explain why the register file needs two read ports but only one write port to correctly execute R-type instructions in a single-cycle RISC-V processor. **(5)**

Ans: _____

_____

_____

_____

_____

Obtained Marks	**/10**

Psychomotor Activity 1 — Experimental Setup

S. No	Criteria	Performance	Description	Marks
1	Tool Setup (4)	Level 4	Opens and configures LogiSim and selects the required environment correctly and independently.	4
		Level 3	Configures the required tool with minor errors or assistance.	3
		Level 2	Completes the setup with considerable assistance.	2
		Level 1	Setup is incomplete and requires substantial assistance.	1
2	File/Design Setup (4)	Level 4	Correctly creates/selects the required assembly file, Logisim circuit/design and required components for the Logisim data path design.	4
		Level 3	File/design setup is mostly correct with minor errors or omissions.	3
		Level 2	Setup is partially completed and requires assistance.	2
		Level 1	Major errors or omissions are present in the setup.	1
3	Parameter Setup (2)	Level 2	Correctly defines required data, registers, parameters, components and initial conditions.	2
		Level 1	Most required inputs/components are correctly configured with minor errors.	1
			Total Obtained	/10

If a student is absent:

Criteria	Description	Marks
Absent	Student is absent from the laboratory activity.	0

Psychomotor Activity 2 — Implementation + Results

S. No	Criteria	Performance	Description	Marks
1	**Implementation (4)**	Level 4	Implements the required Logisim data path design correctly and independently.	**4**
		Level 3	Implements the required program/design with minor errors or assistance.	**3**
		Level 2	Partially implements the program/design and requires considerable assistance.	**2**
		Level 1	Implementation is largely incomplete and requires substantial assistance.	**1**
2	**Execution/Simulation (2)**	Level 2	Executes the RISC-V program or simulates the Logisim design correctly and handles basic errors appropriately.	**2**
		Level 1	Executes/simulates the work with minor errors or assistance.	**1**
3	**Results/Output (4)**	Level 4	Obtains correct results and clearly demonstrates the required program output or circuit behavior.	**4**
		Level 3	Obtains mostly correct results with minor errors.	**3**
		Level 2	Obtains partially correct results with noticeable errors or omissions.	**2**
		Level 1	Results are largely incorrect or incomplete.	**1**
			Total Obtained	**/10**

If a student is absent:

Criteria	Description	Marks
Absent	Student is absent from the laboratory activity.	**0**

Affective Activity — Lab Report + Viva Voce

S. No	Criteria	Performance	Description	Marks
1	**Lab Report Completeness & Organization (4)**	Level 4	Report is complete, well organized and demonstrates full responsibility for the assigned work.	**4**
		Level 3	Report is complete with minor omissions or organizational issues and demonstrates satisfactory responsibility for the assigned work.	**3**
		Level 2	Report is partially complete with several omissions and demonstrates limited responsibility for the assigned work.	**2**
		Level 1	Report has major omissions or poor organization and demonstrates minimal responsibility for the assigned work.	**1**
2	**Code/Design, Results & Conclusion (2)**	Level 2	Logisim data path design, results, and conclusion are correctly presented and discussed, demonstrating ownership of the assigned work.	**2**
		Level 1	Logisim data path design, results, or conclusion are presented with minor errors or omissions.	**1**
3	**Viva Voce (4)**	Level 4	Clearly explains the assigned Logisim data path design, working, and results, demonstrating individual responsibility, participation, and effective communication.	**4**
		Level 3	Explains most of the assigned program/design and results correctly, with minor difficulty, and demonstrates good individual participation.	**3**
		Level 2	Explains basic aspects of the assigned program/design with some prompting and demonstrates basic participation.	**2**
		Level 1	Demonstrates limited understanding or involvement and requires substantial prompting to explain the assigned work.	**1**
			Total Obtained	**/10**

The student has not submitted the report and has not appeared in the Viva Voce.

Criteria	Description	Marks
Lab Report and Results Not Submitted	Student has not submitted the report and results file.	**0**
Absent in Viva Voce	Student is absent from the viva voce.	**0**

Instructor's Signature: _____ Date: _____

Experiment No 8

Design of Single Cycle Processor Data path II

8.1 Objectives:

After completing this experiment, student will be able to:

- Understand the basic functions of Data Path.
- Design Program Counter and Memory for Instructions.
- Integrate PC, ROM, Register File, ALU and RAM to build a simple Data Path.

8.2 Background Theory

A data path is the collection of hardware components within a CPU that performs the actual data processing operations, including arithmetic, logic, memory access, and data transfer. To design a data path for executing RISC-V instructions, begin by identifying the major components required for each instruction type and their control signals, starting with the instruction memory, program counter (PC), and an adder to increment the PC. The memory unit stores program instructions, and the PC holds the address of the current instruction, incremented by the adder for the next instruction. These components form the initial part of the data path for fetching instructions. For R-format instructions, which perform arithmetic or logical operations on two registers and write the result to a third, we need a register file and an ALU. The register file, which stores the processor's 32 general-purpose registers, allows reading and writing by specifying register numbers. The ALU performs operations like addition or subtraction on the values read from the registers. Each R-format instruction requires reading two data words from the register file and writing one data word back, managed by specific control signals. The register file's inputs specify the registers to read and write, while the outputs carry the data. The ALU takes two 32-bit inputs, produces a 32-bit result, and outputs a signal if the result is zero, controlled by a 4-bit signal. Integrating these components ensures the data path can fetch, decode, and execute instructions correctly.

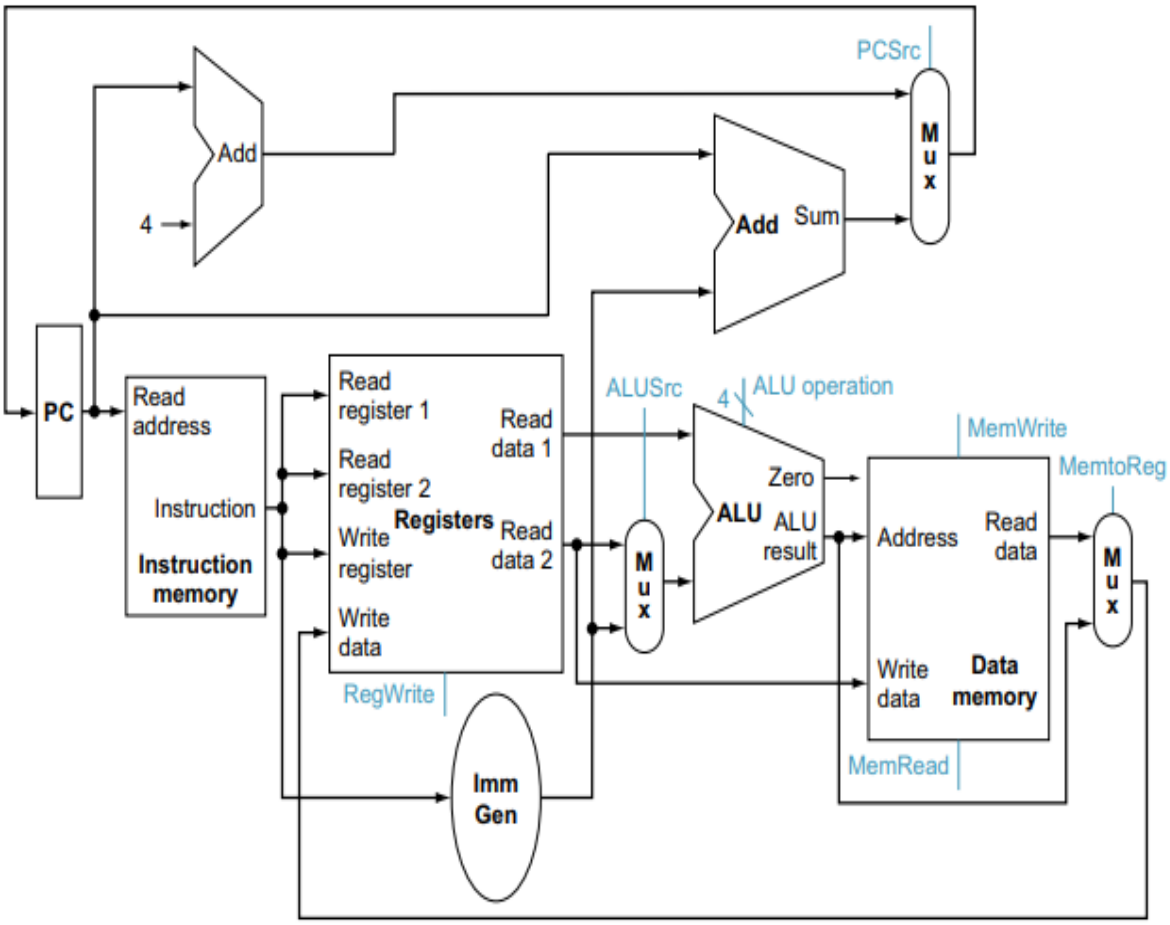
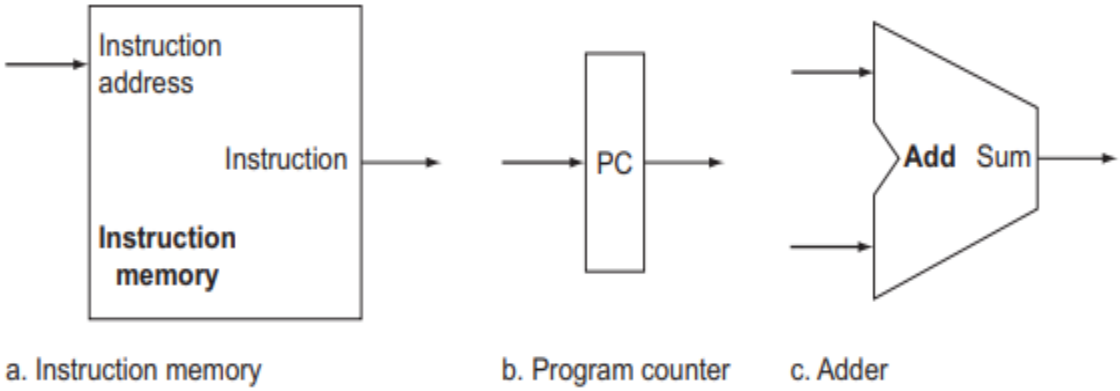


Figure 8.1: Simple Data Path

8.2.1 Designing a Program Counter with Memory

To execute instructions in a program, we need a memory unit to store and supply instructions, a program counter (PC) to hold the current instruction's address, and an adder to increment the PC to the next instruction's address. The adder can be constructed from an ALU set to always perform addition. Fetching an instruction involves retrieving it from memory and incrementing the PC to point to the next instruction. Figure 8.2 shows how these elements work together to fetch instructions and update the PC.



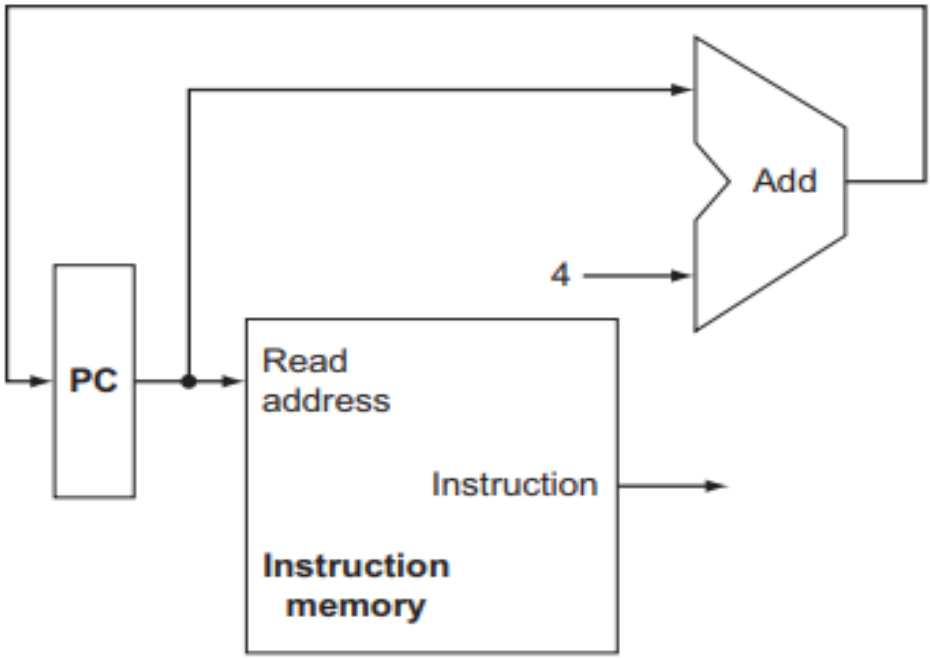


Figure 8.2: PC with Adder and Memory Block

9.2.2 Integration of Complete Data Path without Control Path

Designing a complete data path for a RISC-V core architecture in Logisim involves several steps, including understanding the RISC-V instruction set, the components needed, and how these components interact to execute instructions. Here's a step-by-step guide to designing a basic RISC-V data path in Logisim as mentioned in Figure: 8.1:

8.2.2.1. Understand the RISC-V ISA

The RISC-V instruction set architecture (ISA) defines a set of instructions that the processor can execute. For a simple implementation, you might focus on the RV32I base integer instruction set, which includes instructions for arithmetic, logical operations, control flow (branches and jumps), and memory access.

8.2.2.2. Components of the Data Path

A RISC-V data path typically includes the following components:

- Program Counter (PC): Holds the address of the current instruction.

- Instruction Memory: Stores the instructions to be executed.

- Register File: Contains the registers (e.g., x0-x31) used by the instructions.

- ALU (Arithmetic Logic Unit): Performs arithmetic and logical operations.

- Data Memory: Stores data that is loaded and stored by load/store instructions.

- Control Unit: Generates control signals based on the current instruction (will be coverd in Lab no 9).

- Multiplexers (MUX): Used to select inputs for the ALU, register file, etc.

- Immediate Generator: Generates immediate values from instruction fields.

8.2.2.3. Designing the Data Path in Logisim

Logisim is a graphical tool for designing and simulating digital circuits. Here's how to design each component and assemble the data path:

8.2.2.3.1 Program Counter (PC)

1. Add a Register for the PC: Use a register component to hold the PC value.

2. PC Increment Logic: Use an adder to increment the PC by 4 (since RISC-V instructions are 4 bytes).

8.2.2.3.2 Instruction Memory

1. Memory Component: Use a ROM component to store the instructions. Connect the address input to the PC register.

8.2.2.3.3 Register File

1. Register File Component: Use a 16\32-register file component. Connect the appropriate read and write ports.

2. Read Ports: Connect read ports to the inputs of the ALU.

3. Write Port: Connect the write data and address from the ALU or data memory based on the instruction type.

8.2.2.3.4 ALU

1. ALU Component: Add an ALU component capable of performing necessary operations (add, sub, and, or, etc.).

2. ALU Control Logic: Connect control signals to the ALU to select the operation based on the instruction.

8.2.2.3.5 Data Memory

1. Memory Component: Use a RAM component for data memory.

2. Address and Data Ports: Connect the address and data ports appropriately to the register file and ALU.

8.2.2.3.6 Control Unit

1. Control Logic: Implement control logic using combinational circuits or a ROM that decodes the instruction and generates control signals for the ALU, register file, data memory, and MUXs.

8.2.2.3.7 Multiplexers (MUX)

1. ALU Input Selection: Use MUXs to select between register values and immediate values for ALU inputs.

2. Write-Back Selection: Use a MUX to select the data to write back to the register file (from ALU result or data memory).

8.2.2.3.8 Immediate Generator

1. Immediate Generator Circuit: Design a circuit to extract and sign-extend immediate values from instruction fields.

8.2.2.4 Assembling the Data Path

Connect the components as follows:

8.2.2.4.1 PC to Instruction Memory: Connect the PC output to the address input of the instruction memory.

8.2.2.4.2 Instruction Memory to Control Unit and Immediate Generator: Connect the output of the instruction memory to the control unit and immediate generator.

8.2.2.4.3 Register File to ALU: Connect the read ports of the register file to the ALU inputs.

8.2.2.4.4 Immediate Generator to MUX: Connect the immediate value to one input of the ALU input MUX.

8.2.2.4.5 ALU to Data Memory and Register File: Connect the ALU output to the data memory address input and to the write-back MUX.

8.2.2.4.6 Data Memory to Register File: Connect the data memory output to the write-back MUX.

8.2.2.4.7 MUX to Register File Write Port: Connect the output of the write-back MUX to the register file write port.

8.2.2.4.8 Control Signals: Connect the control signals from the control unit to the MUXs, ALU, register file, and data memory (will be performed in lab no 9).

8.2.2.5 Simulation and Testing

Once the data path is assembled, load a simple RISC-V program into the instruction memory and simulate the execution. Verify that the data path correctly executes each instruction by checking the values in the registers and memory.

9.3 Lab Activities

8.3.1 Task-1: Design and Implement a PC with Adder and Memory Instruction Block.

8.3.2 Task-2: Integrate and assemble all components to design a simple data path.

Lab Exercise and Summary

Summary should cover Introduction, Procedure, Data Analysis and Evaluation.

Student's Signature: _____ **Date: _____**

Cognitive Activity

Q1. List, in order, the sequence of hardware blocks an 'lw' instruction passes through in your integrated data path, from fetching the instruction to writing the loaded value into the register file. **(5)**

Ans: _____

_____

_____

_____

_____

Q2. If the adder that increments the Program Counter were removed from your data path, what specific problem would occur when running a program with more than one instruction? Explain why. **(5)**

Ans: _____

_____

_____

_____

_____

Obtained Marks	**/10**

Psychomotor Activity 1 — Experimental Setup

S. No	Criteria	Performance	Description	Marks
1	Tool Setup (4)	Level 4	Opens and configures LogiSim and selects the required environment correctly and independently.	4
		Level 3	Configures the required tool with minor errors or assistance.	3
		Level 2	Completes the setup with considerable assistance.	2
		Level 1	Setup is incomplete and requires substantial assistance.	1
2	File/Design Setup (4)	Level 4	Correctly creates/selects the required assembly file, Logisim circuit/design and required components for the Logisim data path design.	4
		Level 3	File/design setup is mostly correct with minor errors or omissions.	3
		Level 2	Setup is partially completed and requires assistance.	2
		Level 1	Major errors or omissions are present in the setup.	1
3	Parameter Setup (2)	Level 2	Correctly defines required data, registers, parameters, components and initial conditions.	2
		Level 1	Most required inputs/components are correctly configured with minor errors.	1
			Total Obtained	/10

If a student is absent:

Criteria	Description	Marks
Absent	Student is absent from the laboratory activity.	0

Psychomotor Activity 2 — Implementation + Results

S. No	Criteria	Performance	Description	Marks
1	**Implementation (4)**	Level 4	Implements the required Logisim data path design correctly and independently.	**4**
		Level 3	Implements the required program/design with minor errors or assistance.	**3**
		Level 2	Partially implements the program/design and requires considerable assistance.	**2**
		Level 1	Implementation is largely incomplete and requires substantial assistance.	**1**
2	**Execution/Simulation (2)**	Level 2	Executes the RISC-V program or simulates the Logisim design correctly and handles basic errors appropriately.	**2**
		Level 1	Executes/simulates the work with minor errors or assistance.	**1**
3	**Results/Output (4)**	Level 4	Obtains correct results and clearly demonstrates the required program output or circuit behavior.	**4**
		Level 3	Obtains mostly correct results with minor errors.	**3**
		Level 2	Obtains partially correct results with noticeable errors or omissions.	**2**
		Level 1	Results are largely incorrect or incomplete.	**1**
			Total Obtained	**/10**

If a student is absent:

Criteria	Description	Marks
Absent	Student is absent from the laboratory activity.	**0**

Affective Activity — Lab Report + Viva Voce

S. No	Criteria	Performance	Description	Marks
1	**Lab Report Completeness & Organization (4)**	Level 4	Report is complete, well organized and demonstrates full responsibility for the assigned work.	**4**
		Level 3	Report is complete with minor omissions or organizational issues and demonstrates satisfactory responsibility for the assigned work.	**3**
		Level 2	Report is partially complete with several omissions and demonstrates limited responsibility for the assigned work.	**2**
		Level 1	Report has major omissions or poor organization and demonstrates minimal responsibility for the assigned work.	**1**
2	**Code/Design, Results & Conclusion (2)**	Level 2	Logisim data path design, results, and conclusion are correctly presented and discussed, demonstrating ownership of the assigned work.	**2**
		Level 1	Logisim data path design, results, or conclusion are presented with minor errors or omissions.	**1**
3	**Viva Voce (4)**	Level 4	Clearly explains the assigned Logisim data path design, working, and results, demonstrating individual responsibility, participation, and effective communication.	**4**
		Level 3	Explains most of the assigned program/design and results correctly, with minor difficulty, and demonstrates good individual participation.	**3**
		Level 2	Explains basic aspects of the assigned program/design with some prompting and demonstrates basic participation.	**2**
		Level 1	Demonstrates limited understanding or involvement and requires substantial prompting to explain the assigned work.	**1**
			Total Obtained	**/10**

The student has not submitted the report and has not appeared in the Viva Voce.

Criteria	Description	Marks
Lab Report and Results Not Submitted	Student has not submitted the report and results file.	**0**
Absent in Viva Voce	Student is absent from the viva voce.	**0**

Instructor's Signature: _____ **Date: _____**

Experiment No 9

Design of Single Cycle Processor Control-Path

9.1 Objectives:

After completing this experiment, student will be able to:

- Describe Control Unit/Control path of Single Cycle Processor.
- Design ALU Control.
- Design Main Control Unit.

9.2 Background Theory

The control path in a single-cycle RISC-V processor is a crucial component responsible for generating the necessary signals that direct the operation of the processor's data path during each clock cycle. It interprets the instruction fetched from memory and produces control signals that manage how data moves through the processor. The control unit consists of an instruction decoder and control signal generator. The instruction decoder identifies the type of instruction (such as arithmetic, memory access, or branch) and sets the appropriate control signals. These signals dictate actions like selecting the correct registers, determining ALU operations, enabling memory reads or writes, and controlling the program counter for branching. For example, in an R-type instruction, the control path ensures that the ALU performs the correct arithmetic operation, and the result is written back to the specified register. In a load instruction, it directs data from memory to the register. By efficiently coordinating these actions, the control path ensures that the processor executes each instruction correctly within a single clock cycle, maintaining the simplicity and efficiency characteristic of RISC-V architecture. Here we will discuss two control paths:

- ALU Control Path
- Main Control Path

9.2.1 ALU Control Path

The ALU control path is a vital component of the processor's control unit, responsible for directing the operations of the Arithmetic Logic Unit (ALU) based on the instruction being executed. When an instruction is fetched, the main control unit decodes the opcode and sends it to the ALU control unit. The ALU control unit then interprets the ALUOp code and the function code (funct field)

from the instruction, which together specify the exact operation the ALU needs to perform, such as addition, subtraction, AND, OR, or a comparison.

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract

Instruction opcode	ALUOp	Operation	Funct7 field	Funct3 field	Desired ALU action	ALU control input
lw	00	load word	XXXXXXX	XXX	add	0010
sw	00	store word	XXXXXXX	XXX	add	0010
beq	01	branch if equal	XXXXXXX	XXX	subtract	0110
R-type	10	add	0000000	000	add	0010
R-type	10	sub	0100000	000	subtract	0110
R-type	10	and	0000000	111	AND	0000
R-type	10	or	0000000	110	OR	0001

Figure 9.1 Instruction Set of Control Path.

For example, in an R-type instruction, the main control unit sends an ALUOp code that indicates an arithmetic operation, and the funct field further specifies the exact operation (like add or subtract). The ALU control unit combines these inputs to generate a specific control signal for the ALU. This signal determines the operation performed on the operands provided by the register file.

The control path ensures that the ALU performs the correct computation required by the instruction. For instance, for an add instruction, the ALU control signal will direct the ALU to perform an addition operation on the two input values. The result is then forwarded to the destination register or memory, depending on the type of instruction. This mechanism allows the ALU to execute a variety of operations correctly and efficiently based on the instruction set

architecture.

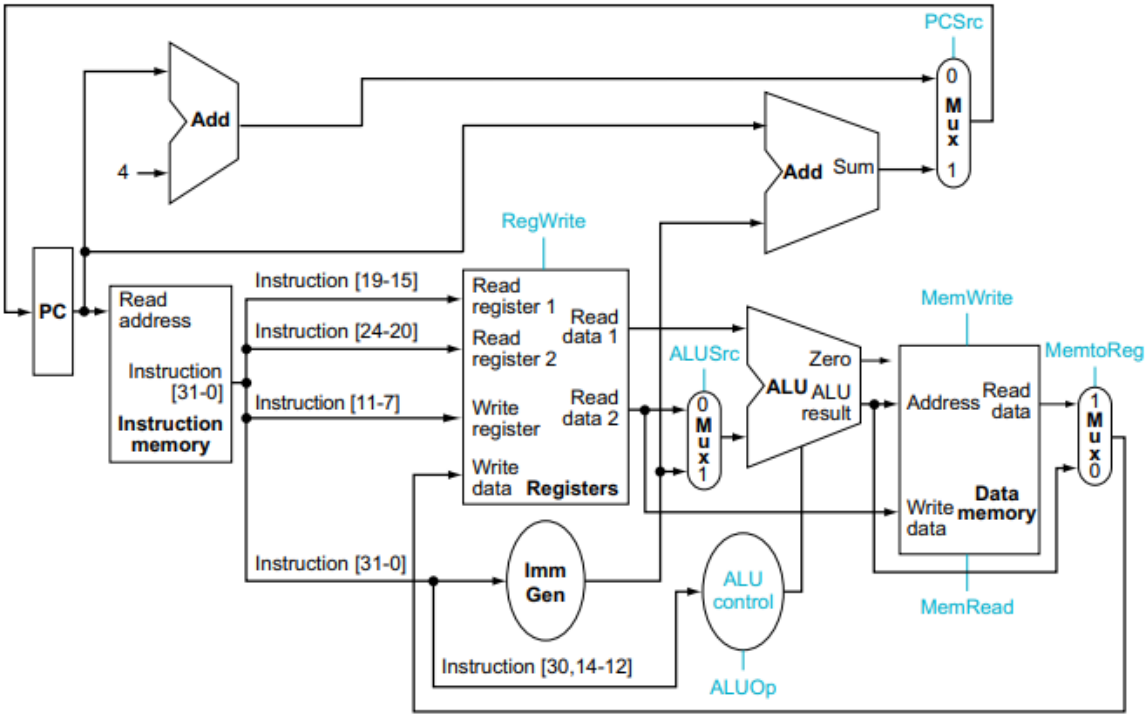


Figure 9.2 Block Diagram with Control Path.

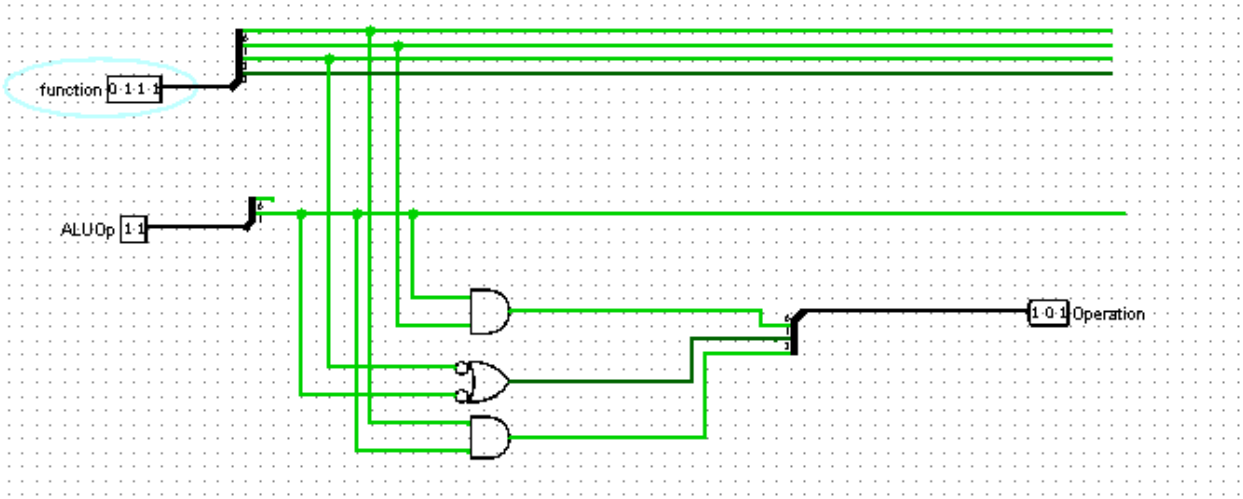


Figure 9.3 Circuit Diagram of ALU Control

9.2.2 Main Control Path\Unit

The main control path in a single-cycle RISC-V processor is responsible for interpreting instructions and generating the necessary control signals to coordinate the datapath components. It consists of several key elements:

9.2.2.1. Instruction Decoder: This component analyzes the opcode of the fetched instruction to determine its type and required operations.

9.2.2.2. Control Signal Generator: Based on the decoded instruction, this unit produces specific control signals. These signals include:

- **RegDst:** Selects the destination register.
- **ALUSrc:** Chooses the ALU's second operand.
- **MemToReg:** Decides if the data to be written to the register comes from memory or the ALU.
- **RegWrite:** Enables writing to the register file.
- **MemRead:** Enables reading from memory.
- **MemWrite:** Enables writing to memory.
- **Branch:** Determines if a branch should be taken.
- **ALUOp:** Defines the operation the ALU should perform.

9.2.2.3. Multiplexers and Control Logic: These elements route data correctly based on control signals. For example, they select the correct source for ALU operations or the destination for data writes.

The control path ensures that all parts of the processor work together harmoniously. For an arithmetic operation, it directs the ALU to perform the correct calculation and stores the result in the appropriate register. For memory operations, it coordinates the flow of data between memory and registers. By managing these actions precisely, the control path enables the processor to execute each instruction accurately within a single cycle, maintaining efficiency and simplicity.

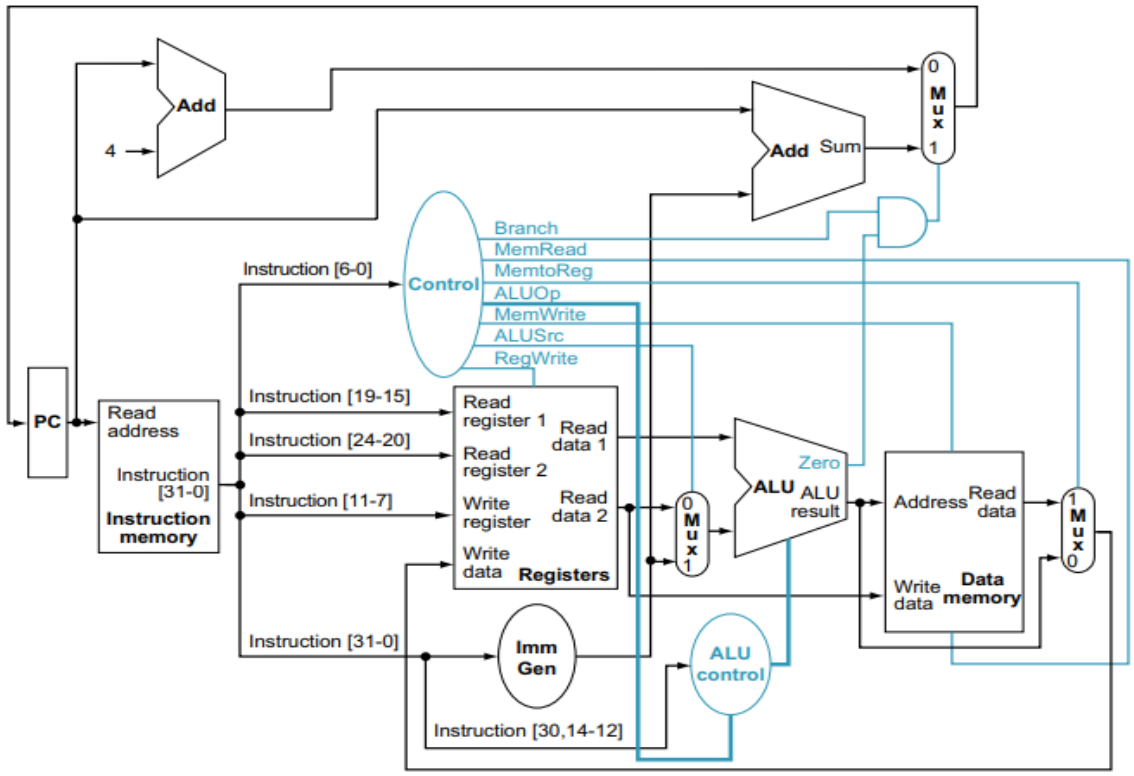


Figure 9.4 Block Diagram with Main Control Path

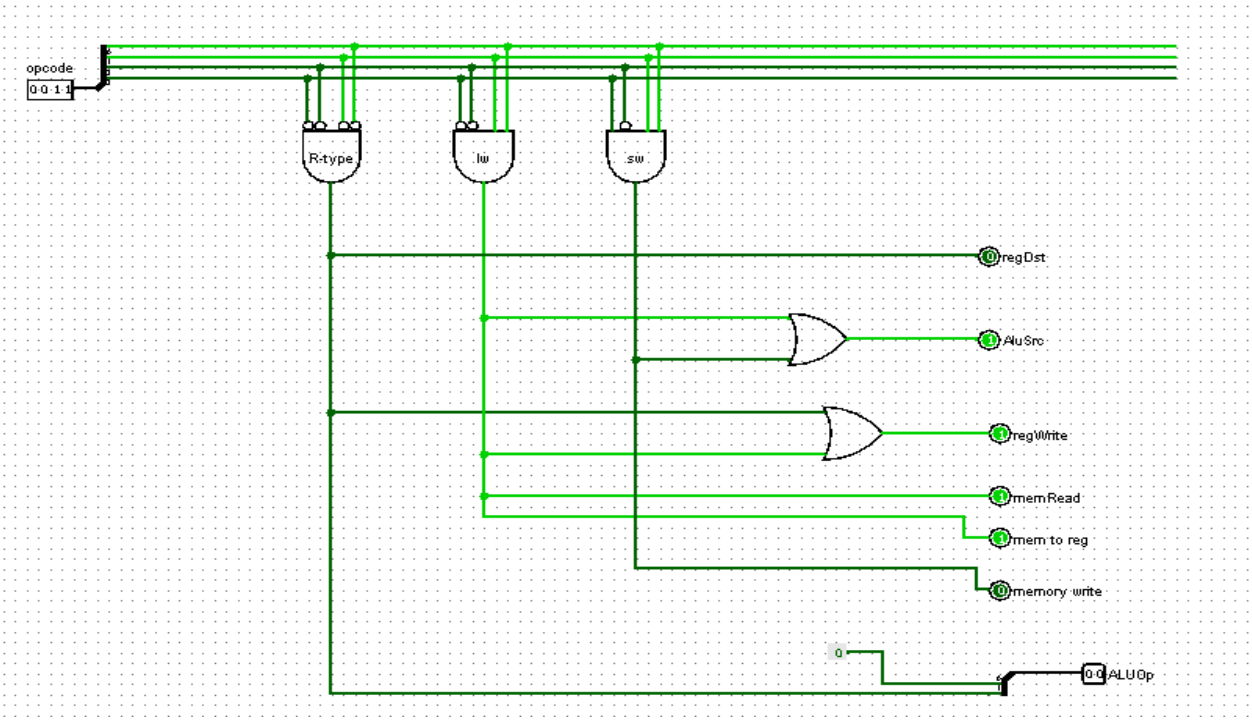


Figure 9.4 Circuit Diagram of Main Control Unit

9.3 Lab Activities

9.3.1 Task-1: Implement ALU Control in LogiSim.

9.3.2 Task-2: Design and Implement Main Control Path with R-Type Instructions in LogiSim.

9.3.3 Task-3: Design and Implement Main Control Path with Branch and Jump Instructions in LogiSim

Lab Exercise and Summary

Summary should cover Introduction, Procedure, Data Analysis and Evaluation.

Student's Signature: _____ **Date: _____**

Cognitive Activity

Q1. For an 'lw' instruction, list each of the following control signals and state whether it should be 0 or 1: RegWrite, MemRead, MemWrite, ALUSrc, MemtoReg, Branch. **(5)**

Ans: _____

_____

_____

_____

_____

Q2. Explain the difference between what the Main Control Unit decides and what the ALU Control block decides, referring to the opcode and funct fields in your answer. **(5)**

Ans: _____

_____

_____

_____

_____

Obtained Marks	**/10**

Psychomotor Activity 1 — Experimental Setup

S. No	Criteria	Performance	Description	Marks
1	Tool Setup (4)	Level 4	Opens and configures LogiSim and selects the required environment correctly and independently.	4
		Level 3	Configures the required tool with minor errors or assistance.	3
		Level 2	Completes the setup with considerable assistance.	2
		Level 1	Setup is incomplete and requires substantial assistance.	1
2	File/Design Setup (4)	Level 4	Correctly creates/selects the required assembly file, Logisim circuit/design and required components for the Logisim control unit design.	4
		Level 3	File/design setup is mostly correct with minor errors or omissions.	3
		Level 2	Setup is partially completed and requires assistance.	2
		Level 1	Major errors or omissions are present in the setup.	1
3	Parameter Setup (2)	Level 2	Correctly defines required data, registers, parameters, components and initial conditions.	2
		Level 1	Most required inputs/components are correctly configured with minor errors.	1
			Total Obtained	/10

If a student is absent:

Criteria	Description	Marks
Absent	Student is absent from the laboratory activity.	0

Psychomotor Activity 2 — Implementation + Results

S. No	Criteria	Performance	Description	Marks
1	**Implementation (4)**	Level 4	Implements the required Logisim control unit design correctly and independently.	**4**
		Level 3	Implements the required program/design with minor errors or assistance.	**3**
		Level 2	Partially implements the program/design and requires considerable assistance.	**2**
		Level 1	Implementation is largely incomplete and requires substantial assistance.	**1**
2	**Execution/Simulation (2)**	Level 2	Executes the RISC-V program or simulates the Logisim design correctly and handles basic errors appropriately.	**2**
		Level 1	Executes/simulates the work with minor errors or assistance.	**1**
3	**Results/Output (4)**	Level 4	Obtains correct results and clearly demonstrates the required program output or circuit behavior.	**4**
		Level 3	Obtains mostly correct results with minor errors.	**3**
		Level 2	Obtains partially correct results with noticeable errors or omissions.	**2**
		Level 1	Results are largely incorrect or incomplete.	**1**
			Total Obtained	**/10**

If a student is absent:

Criteria	Description	Marks
Absent	Student is absent from the laboratory activity.	**0**

Affective Activity — Lab Report + Viva Voce

S. No	Criteria	Performance	Description	Marks
1	**Lab Report Completeness & Organization (4)**	Level 4	Report is complete, well organized and demonstrates full responsibility for the assigned work.	**4**
		Level 3	Report is complete with minor omissions or organizational issues and demonstrates satisfactory responsibility for the assigned work.	**3**
		Level 2	Report is partially complete with several omissions and demonstrates limited responsibility for the assigned work.	**2**
		Level 1	Report has major omissions or poor organization and demonstrates minimal responsibility for the assigned work.	**1**
2	**Code/Design, Results & Conclusion (2)**	Level 2	Logisim control unit design, results, and conclusion are correctly presented and discussed, demonstrating ownership of the assigned work.	**2**
		Level 1	Logisim control unit design, results, or conclusion are presented with minor errors or omissions.	**1**
3	**Viva Voce (4)**	Level 4	Clearly explains the assigned Logisim control unit design, working, and results, demonstrating individual responsibility, participation, and effective communication.	**4**
		Level 3	Explains most of the assigned program/design and results correctly, with minor difficulty, and demonstrates good individual participation.	**3**
		Level 2	Explains basic aspects of the assigned program/design with some prompting and demonstrates basic participation.	**2**
		Level 1	Demonstrates limited understanding or involvement and requires substantial prompting to explain the assigned work.	**1**
			Total Obtained	**/10**

The student has not submitted the report and has not appeared in the Viva Voce.

Criteria	Description	Marks
Lab Report and Results Not Submitted	Student has not submitted the report and results file.	**0**
Absent in Viva Voce	Student is absent from the viva voce.	**0**

Instructor's Signature: _____ Date: _____

Experiment No 10

Integration of data and control path and Simulation of Single Cycle Processor

10.1 Objectives:

After completing this experiment, student will be able to:

- Describe the design and operational principle of RISC-V Single Cycle Processor.
- Design and Simulate Data Path in LogiSim.
- Design and Simulate Control Path in LogiSim.
- Design and Simulate complete RISC-V Single Cycle Processor by Integrating Data and Control Paths.

10.2 Background Theory

A RISC-V single-cycle processor for a lab experiment is a simplified CPU architecture designed to execute each instruction in one clock cycle. This processor type is used in educational settings to help students understand the fundamentals of computer architecture, including instruction fetch, decode, execute, memory access, and write-back processes. In a single-cycle processor, all these stages are completed within a single clock cycle for every instruction, leading to a straightforward design with minimal control logic. However, this simplicity also means the design is less efficient for complex operations compared to multi-cycle or pipelined processors. Building and experimenting with a RISC-V single-cycle processor in the lab provides hands-on experience with instruction set architecture, control path design, and the implementation of arithmetic logic units (ALUs), registers, and memory units, fostering a deep understanding of how basic CPU operations are performed at the hardware level.

10.2.1 Integration of Data Path

Integrating a RISC-V data path using Logisim involves designing and connecting various components such as the program counter (PC), instruction memory, registers, arithmetic logic unit (ALU), and data memory in a graphical circuit simulation tool. Each component represents a part of the CPU's functionality, such as fetching instructions, performing calculations, and accessing memory. In Logisim, these components are visually connected to reflect the data flow and control signals of a RISC-V processor. This integration helps in understanding how instructions move

through the CPU and how different parts of the processor interact, providing a clear and interactive way to learn and experiment with CPU design principles.

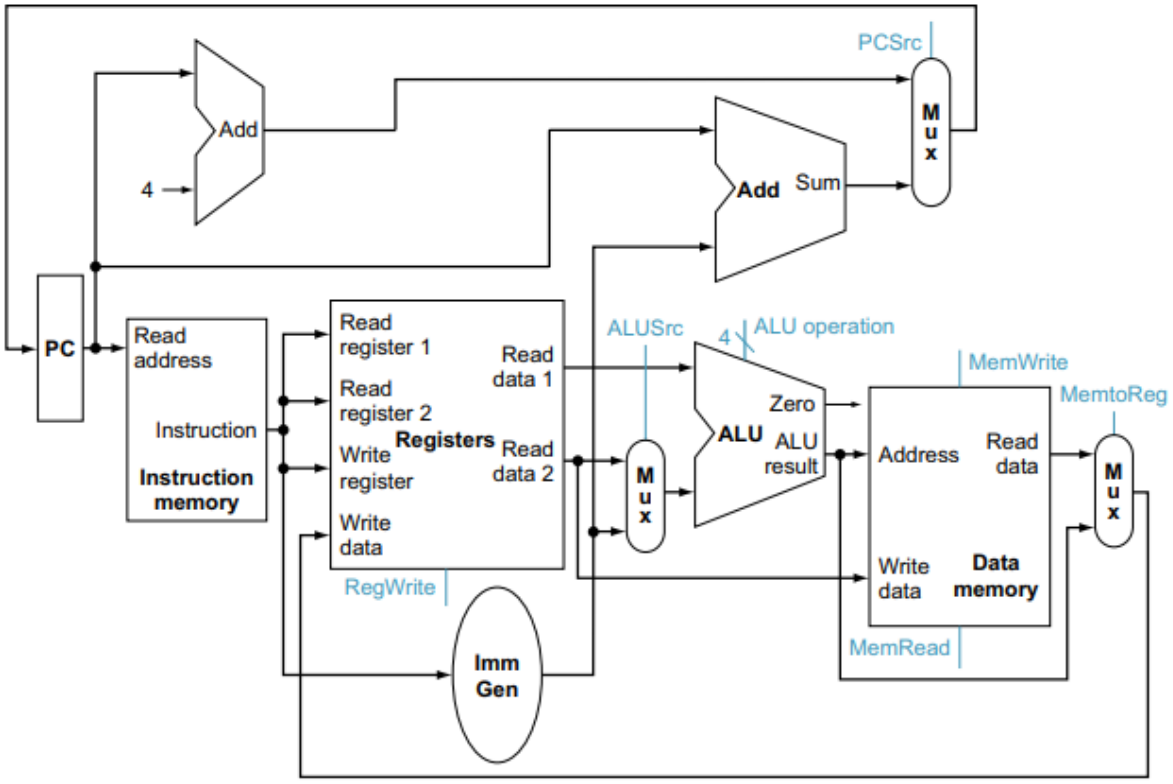


Figure: 10.1 Data Path Integration

10.2.2 Integration of Control Path

Integrating the RISC-V control path using Logisim involves creating a digital circuit that manages the flow of data and execution of instructions within a RISC-V processor. In Logisim, this process begins by designing the main components such as the program counter, instruction memory, register file, ALU, and data memory. The control path is responsible for generating control signals based on the decoded instruction to direct the operation of these components. This includes signals for selecting the ALU operation, controlling the read and write operations of the register file and memory, and managing the program counter updates. Using Logisim's simulation capabilities, each component can be connected and tested to ensure proper functionality. The control unit decodes the opcode and generates the appropriate control signals, which are then routed to the corresponding components to perform operations like arithmetic calculations, data transfers, and branching. By visualizing and simulating the control path in Logisim, students gain practical

insight into the coordination required for executing RISC-V instructions and the interaction between different processor components.

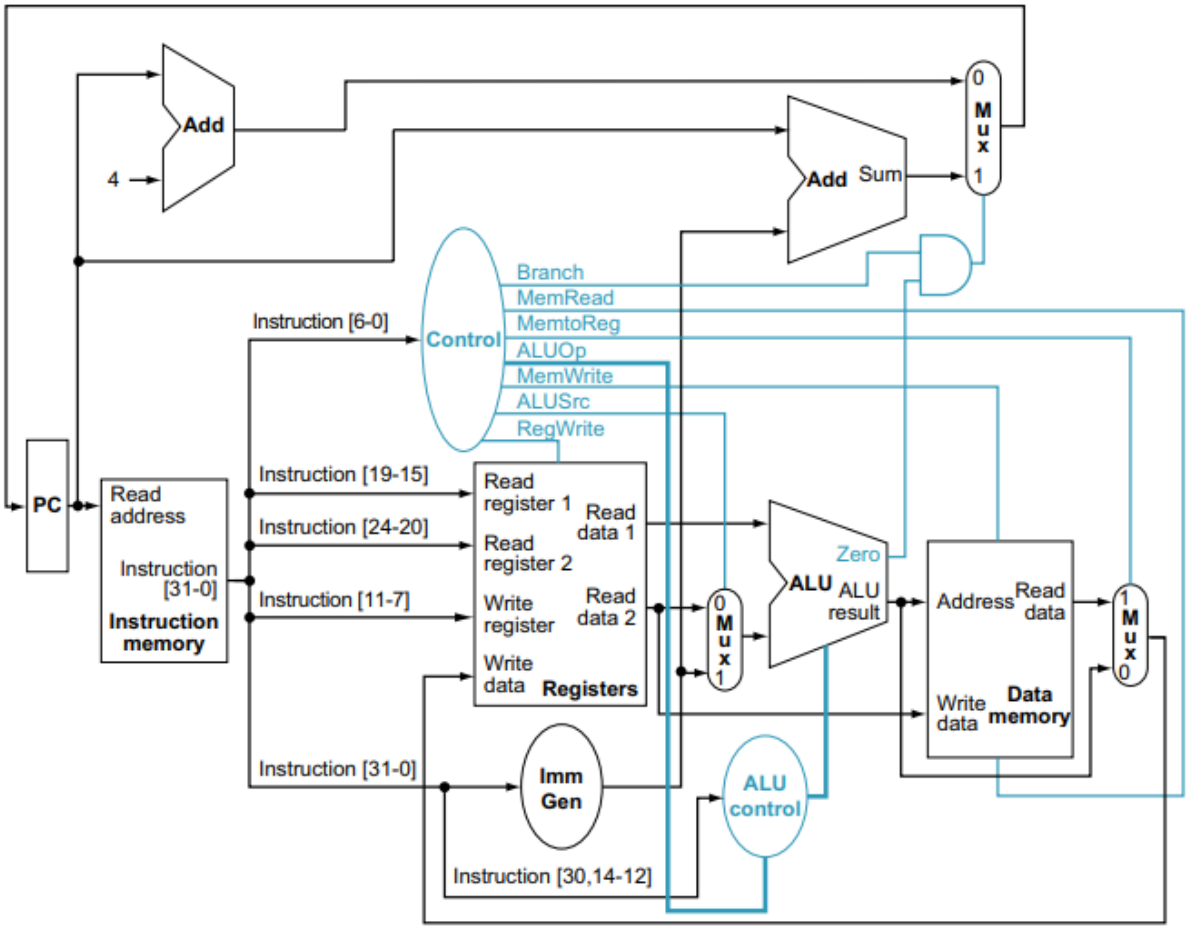


Figure 10.2 Control Path Integration

10.2.3 Integration of Data path and Control Path

Integrating the data path and control path to create a single-cycle RISC-V processor involves combining these two components so they work seamlessly together to execute instructions within one clock cycle. The data path includes the program counter (PC), instruction memory, register file, ALU, data memory, and the write-back mechanism. The control path, on the other hand, comprises a control unit that generates control signals based on the decoded instruction. These control signals direct the operations of the data path components. In Logisim, each component is placed and interconnected: the PC is linked to the instruction memory to fetch instructions, the fetched instruction is sent to both the control unit and the register file, and the control signals manage the flow of data through the ALU and memory for operations like arithmetic calculations,

data transfers, and branching. The control unit ensures the correct control signals are sent to guide the data through the appropriate paths, resulting in the execution of each instruction in a single cycle. This integration process involves careful design and simulation to ensure all components and control signals function correctly, providing students with a practical understanding of processor architecture.

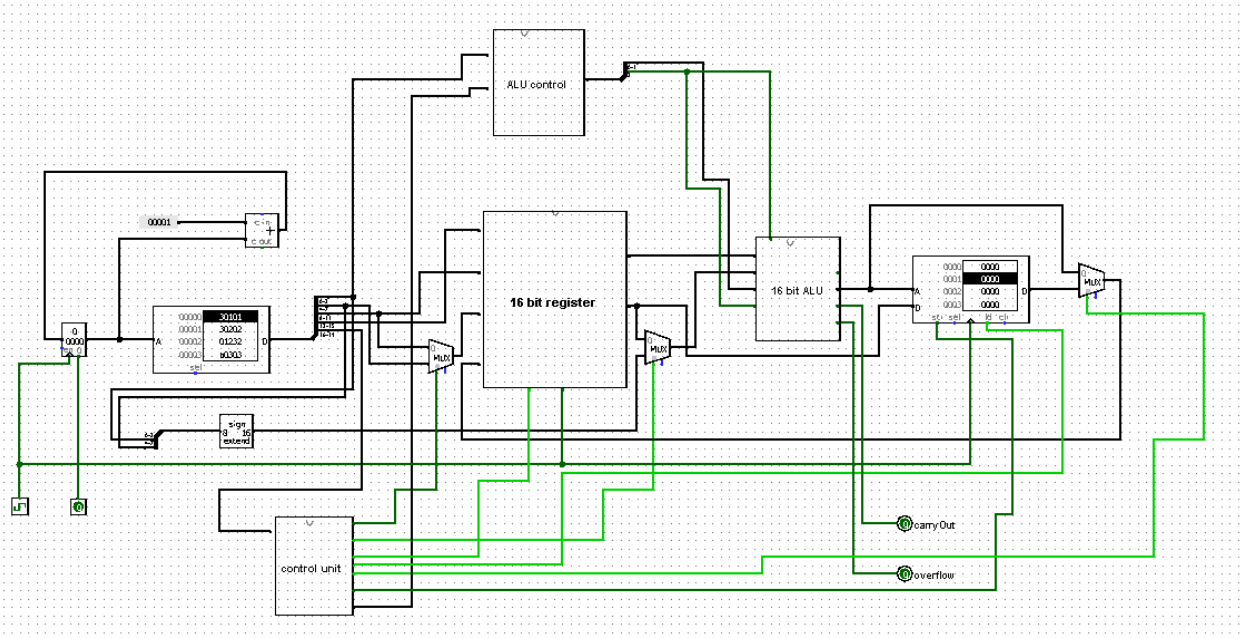


Figure 10.3 Complete 16-bit Single Cycle Processor

10.3 Lab Activities

10.3.1 Task-1: Design and Implement Data Path for 16-bit single cycle processor.

10.3.2 Task-2: Implement a Control Path for 16-bit single cycle RISC-V Processor.

10.3.3 Task-3: Implement a complete Single Cycle RISC-V Processor.

Lab Exercise and Summary

Summary should cover Introduction, Procedure, Data Analysis and Evaluation.

Student's Signature: _____ **Date: _____**

Cognitive Activity

Q1. Walk through, step by step, what happens inside your single-cycle processor during one complete clock cycle when it executes 'sw x5, 0(x6)'. **(5)**

Ans: _____

_____

_____

_____

_____

Q2. Your simulated single-cycle processor executes 100 instructions, and each clock cycle takes 20 ns (set by the slowest instruction). Calculate the total execution time, and briefly explain why a multi-cycle design might complete the same program faster overall. **(5)**

Ans: _____

_____

_____

_____

_____

Obtained Marks	**/10**

Psychomotor Activity 1 — Experimental Setup

S. No	Criteria	Performance	Description	Marks
1	Tool Setup (4)	Level 4	Opens and configures LogiSim and selects the required environment correctly and independently.	4
		Level 3	Configures the required tool with minor errors or assistance.	3
		Level 2	Completes the setup with considerable assistance.	2
		Level 1	Setup is incomplete and requires substantial assistance.	1
2	File/Design Setup (4)	Level 4	Correctly creates/selects the required assembly file, Logisim circuit/design and required components for the Logisim single-cycle processor design.	4
		Level 3	File/design setup is mostly correct with minor errors or omissions.	3
		Level 2	Setup is partially completed and requires assistance.	2
		Level 1	Major errors or omissions are present in the setup.	1
3	Parameter Setup (2)	Level 2	Correctly defines required data, registers, parameters, components and initial conditions.	2
		Level 1	Most required inputs/components are correctly configured with minor errors.	1
			Total Obtained	/10

If a student is absent:

Criteria	Description	Marks
Absent	Student is absent from the laboratory activity.	0

Psychomotor Activity 2 — Implementation + Results

S. No	Criteria	Performance	Description	Marks
1	**Implementation (4)**	Level 4	Implements the required Logisim single-cycle processor design correctly and independently.	**4**
		Level 3	Implements the required program/design with minor errors or assistance.	**3**
		Level 2	Partially implements the program/design and requires considerable assistance.	**2**
		Level 1	Implementation is largely incomplete and requires substantial assistance.	**1**
2	**Execution/Simulation (2)**	Level 2	Executes the RISC-V program or simulates the Logisim design correctly and handles basic errors appropriately.	**2**
		Level 1	Executes/simulates the work with minor errors or assistance.	**1**
3	**Results/Output (4)**	Level 4	Obtains correct results and clearly demonstrates the required program output or circuit behavior.	**4**
		Level 3	Obtains mostly correct results with minor errors.	**3**
		Level 2	Obtains partially correct results with noticeable errors or omissions.	**2**
		Level 1	Results are largely incorrect or incomplete.	**1**
			Total Obtained	**/10**

If a student is absent:

Criteria	Description	Marks
Absent	Student is absent from the laboratory activity.	**0**

Affective Activity — Lab Report + Viva Voce

S. No	Criteria	Performance	Description	Marks
1	**Lab Report Completeness & Organization (4)**	Level 4	Report is complete, well organized and demonstrates full responsibility for the assigned work.	**4**
		Level 3	Report is complete with minor omissions or organizational issues and demonstrates satisfactory responsibility for the assigned work.	**3**
		Level 2	Report is partially complete with several omissions and demonstrates limited responsibility for the assigned work.	**2**
		Level 1	Report has major omissions or poor organization and demonstrates minimal responsibility for the assigned work.	**1**
2	**Code/Design, Results & Conclusion (2)**	Level 2	Logisim single-cycle processor design, results, and conclusion are correctly presented and discussed, demonstrating ownership of the assigned work.	**2**
		Level 1	Logisim single-cycle processor design, results, or conclusion are presented with minor errors or omissions.	**1**
3	**Viva Voce (4)**	Level 4	Clearly explains the assigned Logisim single-cycle processor design, working, and results, demonstrating individual responsibility, participation, and effective communication.	**4**
		Level 3	Explains most of the assigned program/design and results correctly, with minor difficulty, and demonstrates good individual participation.	**3**
		Level 2	Explains basic aspects of the assigned program/design with some prompting and demonstrates basic participation.	**2**
		Level 1	Demonstrates limited understanding or involvement and requires substantial prompting to explain the assigned work.	**1**
			Total Obtained	**/10**

The student has not submitted the report and has not appeared in the Viva Voce.

Criteria	Description	Marks
Lab Report and Results Not Submitted	Student has not submitted the report and results file.	**0**
Absent in Viva Voce	Student is absent from the viva voce.	**0**

Instructor's Signature: _____ **Date: _____**

Experiment No 11

Pipelining of Single Cycle Processor

11.1 Objectives:

After completing this experiment, student will be able to:

- Describe the Design and operational principle of Pipelining in RISC-V Single Cycle Processor.
- Implement and Simulate Pipelining of RISC-V Single Cycle Processor using LogiSim.

11.2 Background Theory

Pipelining is a technique used in computer architecture to increase the instruction throughput by overlapping the execution of multiple instructions. It works by dividing the instruction execution process into several discrete stages, such as Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). Each stage completes a part of the instruction processing, and different instructions are processed simultaneously in different stages. This overlapping allows the processor to start executing a new instruction before the previous one has completed, much like an assembly line in a factory where different stages of production are carried out concurrently on different items. This approach significantly enhances the overall performance and efficiency of the CPU by ensuring that more instructions are completed in a given period.

11.2.1 Stages of a Pipeline

A typical RISC-V pipeline might have the following five stages:

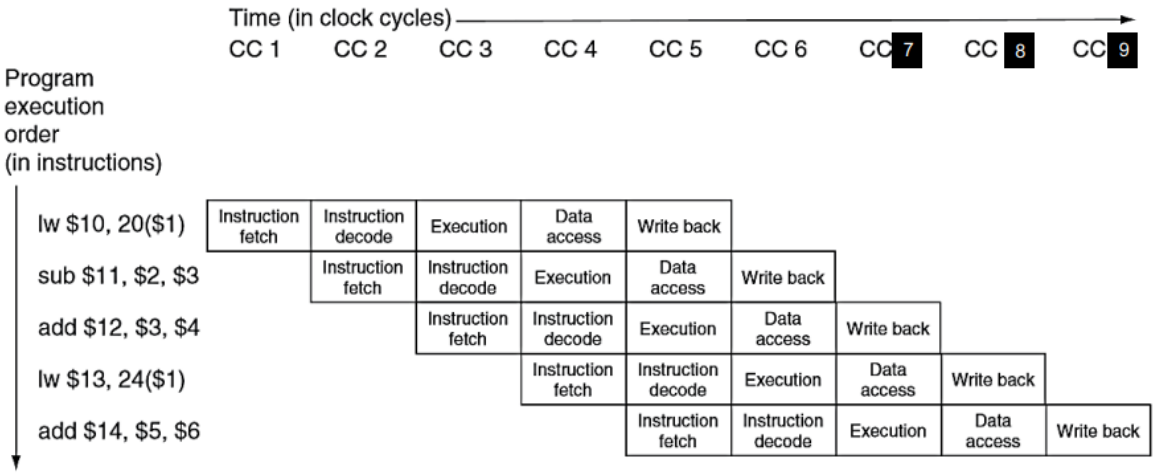
1. **Instruction Fetch (IF):** In this stage, the processor fetches the instruction from memory. The program counter (PC) holds the address of the current instruction, which is used to retrieve the instruction from the instruction memory.

2. **Instruction Decode (ID):** The fetched instruction is then decoded to understand what actions need to be performed. During this stage, the processor reads the necessary operands from the register file.

3. **Execute (EX):** The actual operation specified by the instruction is performed in this stage. For arithmetic and logic instructions, the Arithmetic Logic Unit (ALU) performs the calculation. For branch instructions, the target address is calculated.

4. **Memory Access (MEM):** If the instruction involves memory access (like load or store instructions), the processor reads from or writes to the data memory in this stage.

5. **Write Back (WB):** Finally, the result of the instruction is written back to the register file, updating the appropriate register with the new value.



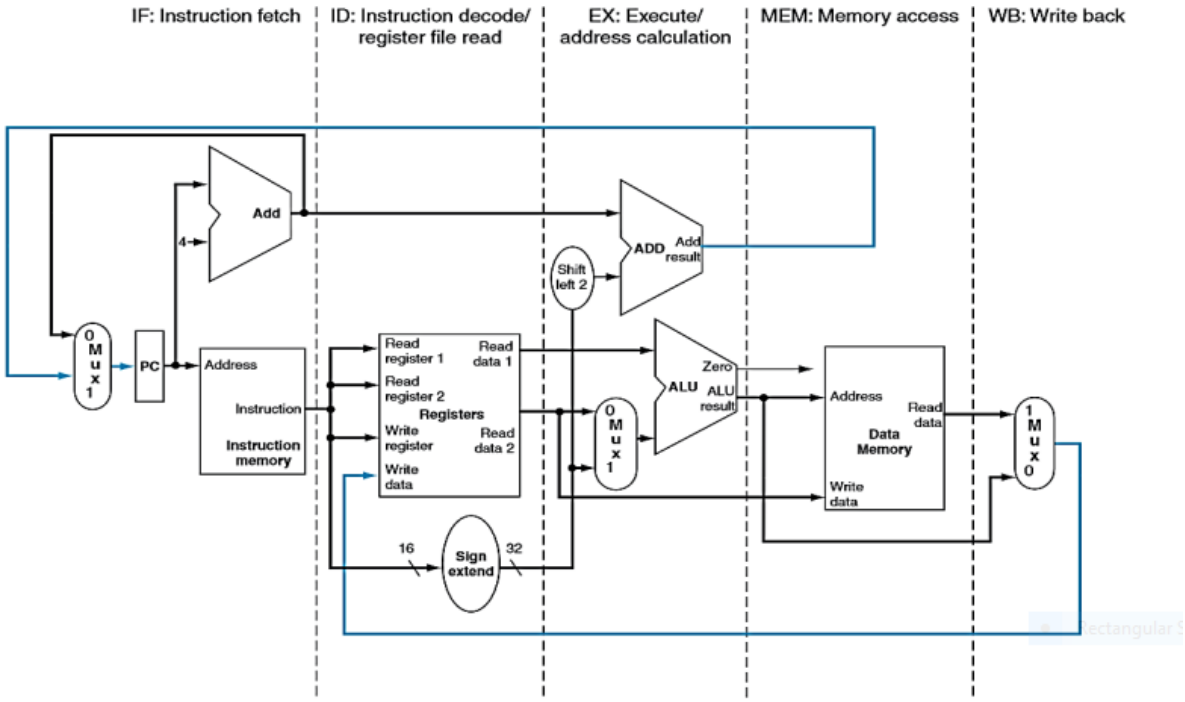


Figure: 11.1 Different Stages of Pipelining

11.2.2 How Pipelining Works

In a pipelined processor, multiple instructions are processed simultaneously, each at a different stage. For example:

1. At clock cycle 1, Instruction 1 is in the IF stage.
2. At clock cycle 2, Instruction 1 moves to the ID stage, and Instruction 2 enters the IF stage.
3. At clock cycle 3, Instruction 1 moves to the EX stage, Instruction 2 moves to the ID stage, and Instruction 3 enters the IF stage.

This continues such that, at any given clock cycle, different instructions are in different stages of execution. This overlapping of instruction execution significantly improves the throughput of the processor, as it can start processing a new instruction before the previous one has completed.

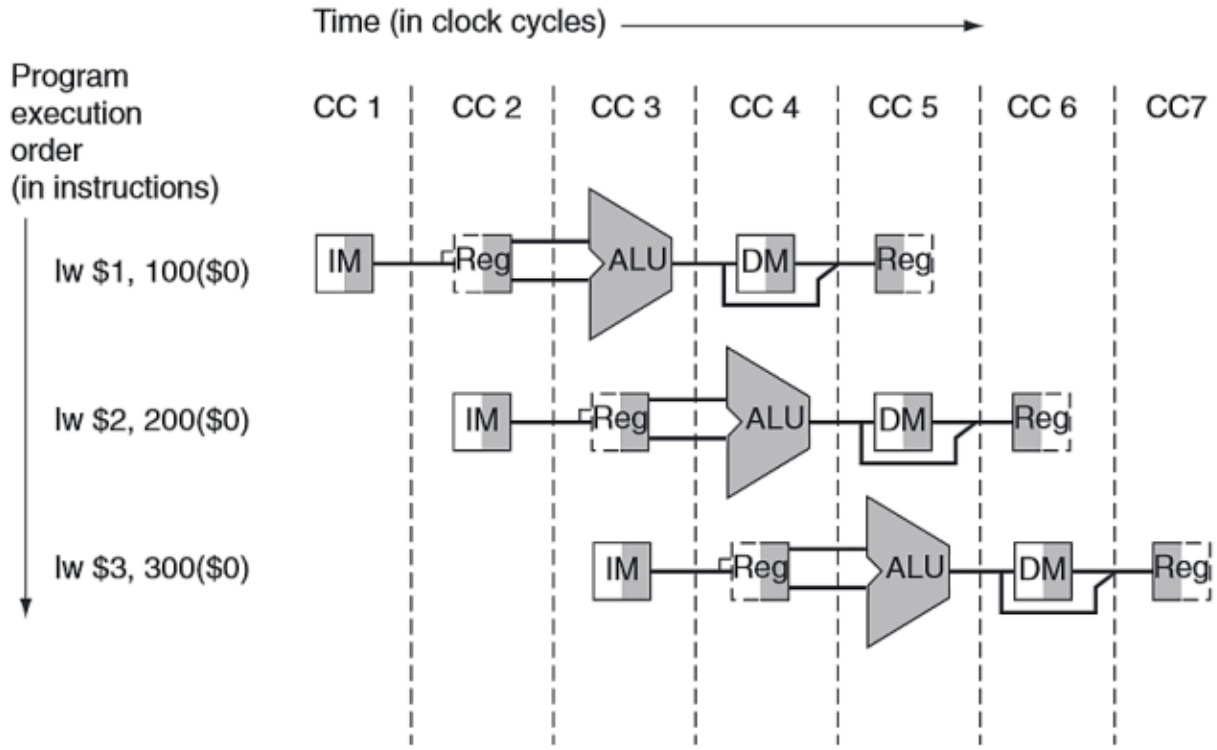
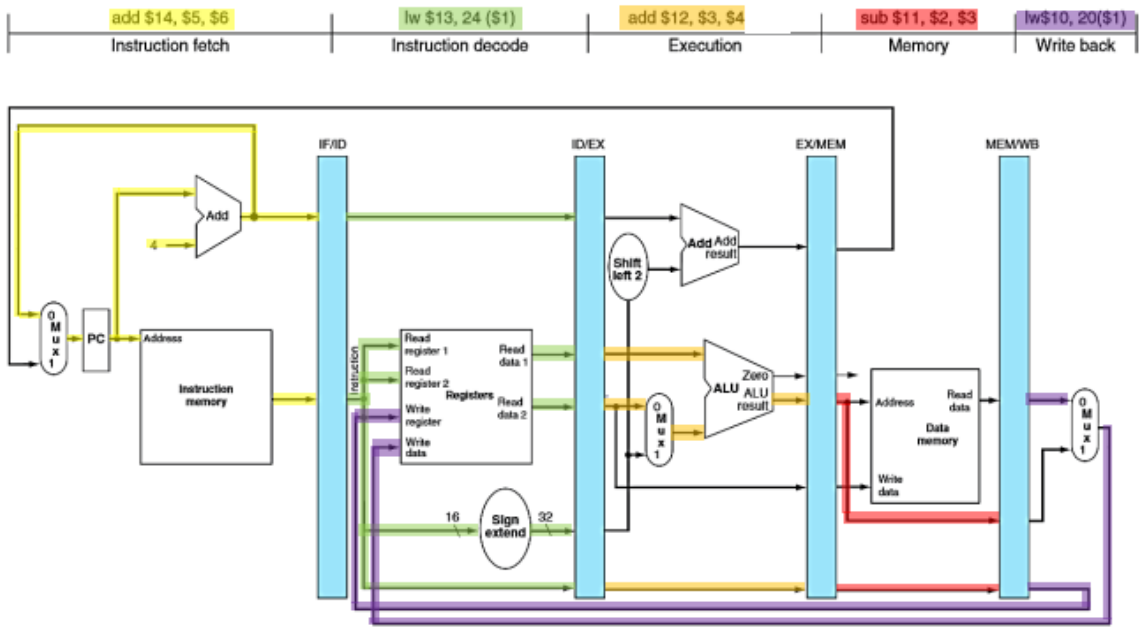


Figure 11.2 Control Path Integration

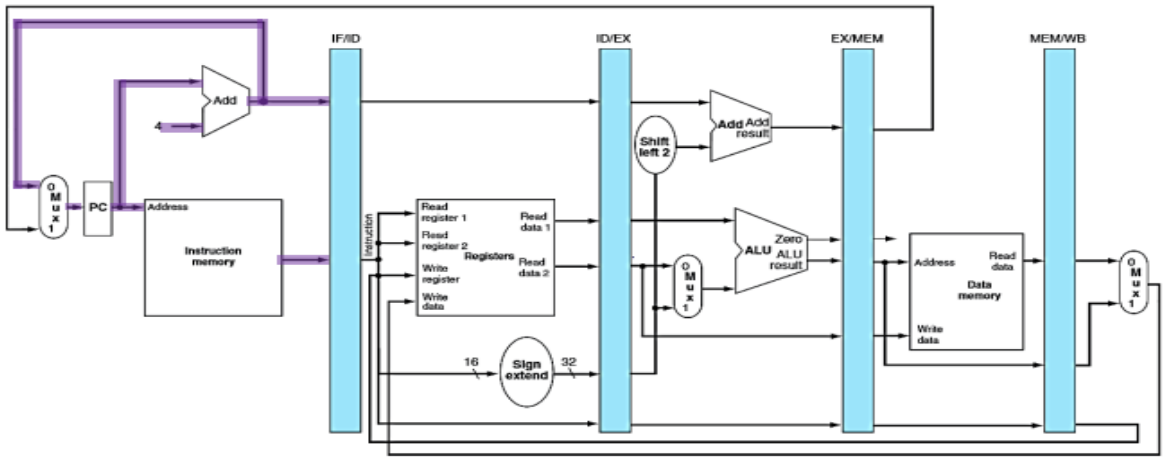
11.2.3 Pipelined Data path

A pipelined data path in a RISC-V single-cycle processor involves breaking down the instruction execution process into multiple stages to improve performance by allowing multiple instructions to be processed simultaneously. These stages typically include instruction fetch (IF), instruction decode/register fetch (ID), execution/address calculation (EX), memory access (MEM), and write-back (WB). Each stage completes a part of the instruction, and while one instruction is in the write-back stage, another can be in memory access, another in execution, and so on. This overlapping of stages, managed by pipeline registers that store intermediate data between stages, enhances throughput by ensuring that each part of the processor is busy with some instruction's task, minimizing idle time and improving overall efficiency.

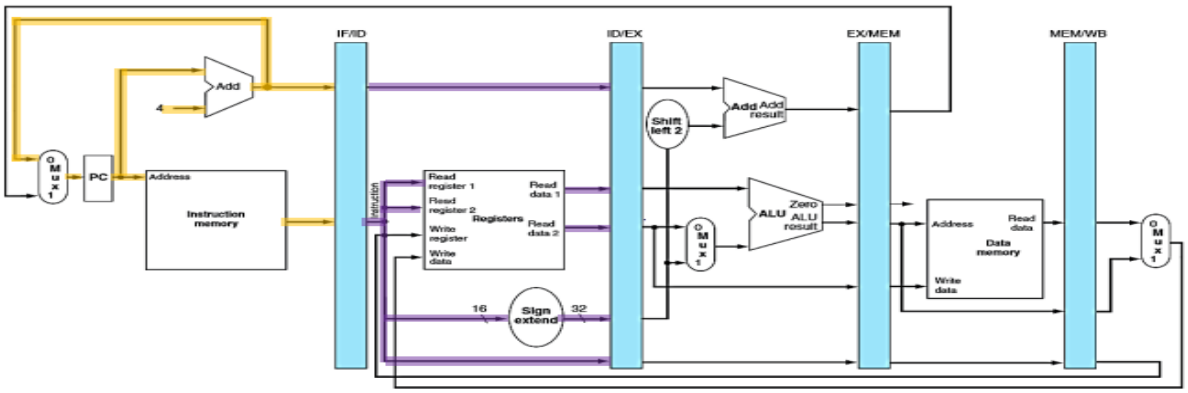
In the following Figures, we walk through all 9 cycles required to fully complete the 5 instructions in our sample sequence. We will start with clock cycle 1 and highlight the relevant data path lines according to the instruction being executed in each particular stage.



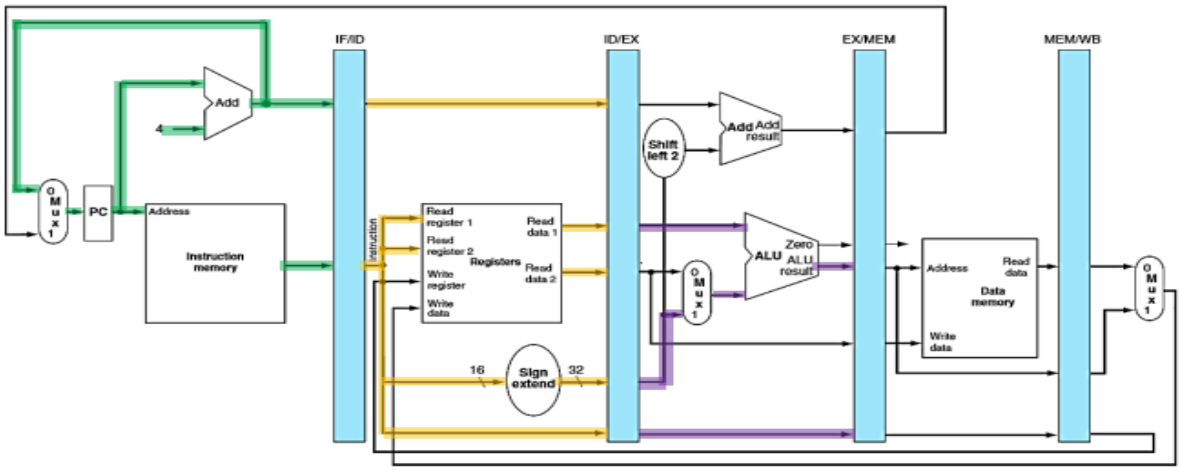
lw $10, 20($1)



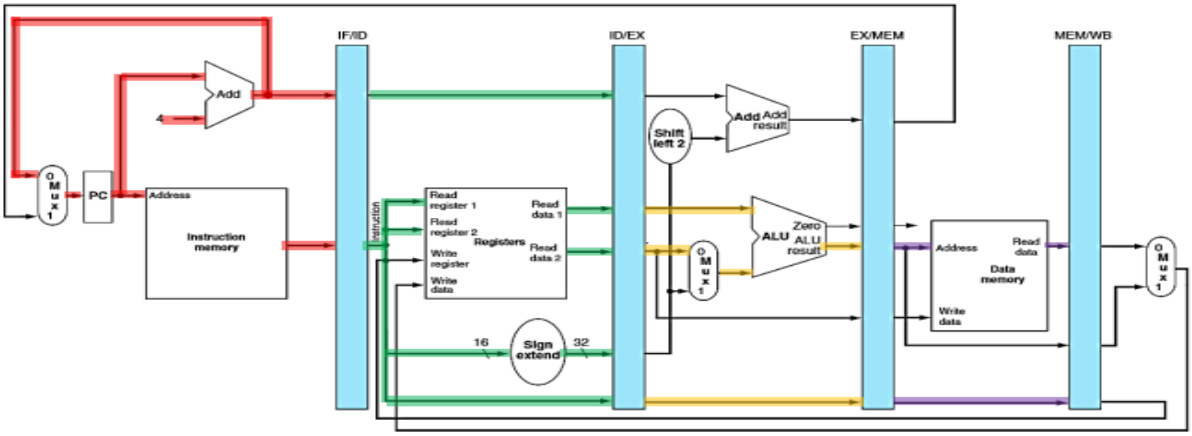
sub $11,$2,$3 lw $10, 20($1)



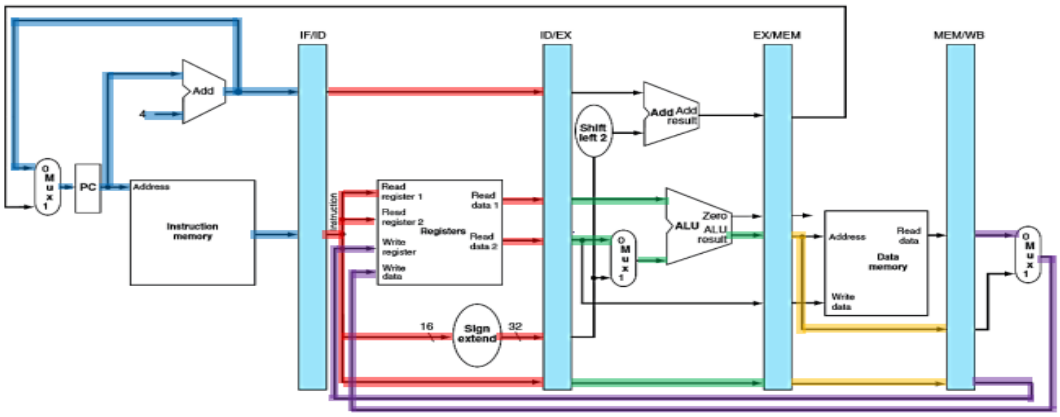
add $12,$3,$4 sub $11,$2,$3 lw $10, 20($1)



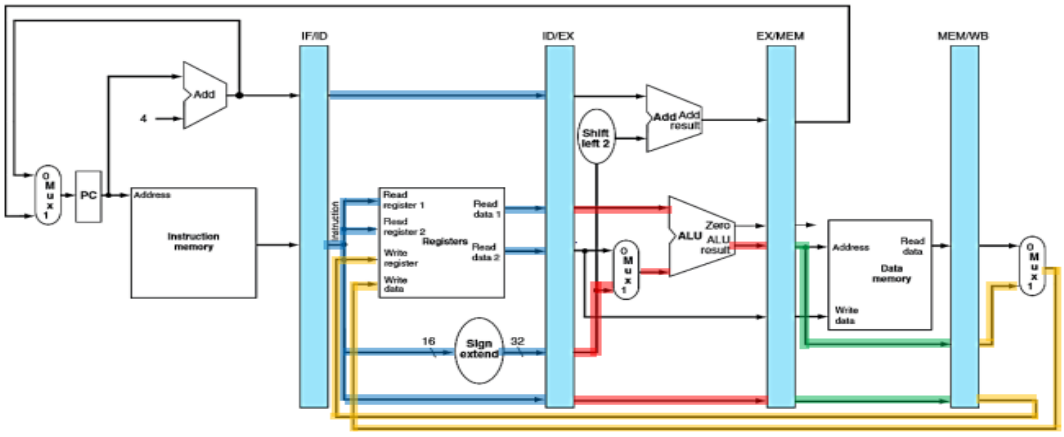
lw $13, 24($1) add $12,$3,$4 sub $11,$2,$3 lw $10, 20($1)



add $14,$5,$6 lw $13, 24($1) add $12,$3,$4 sub $11,$2,$3 lw $10, 20($1)



add $14,$5,$6 lw $13, 24($1) add $12,$3,$4 sub $11,$2,$3



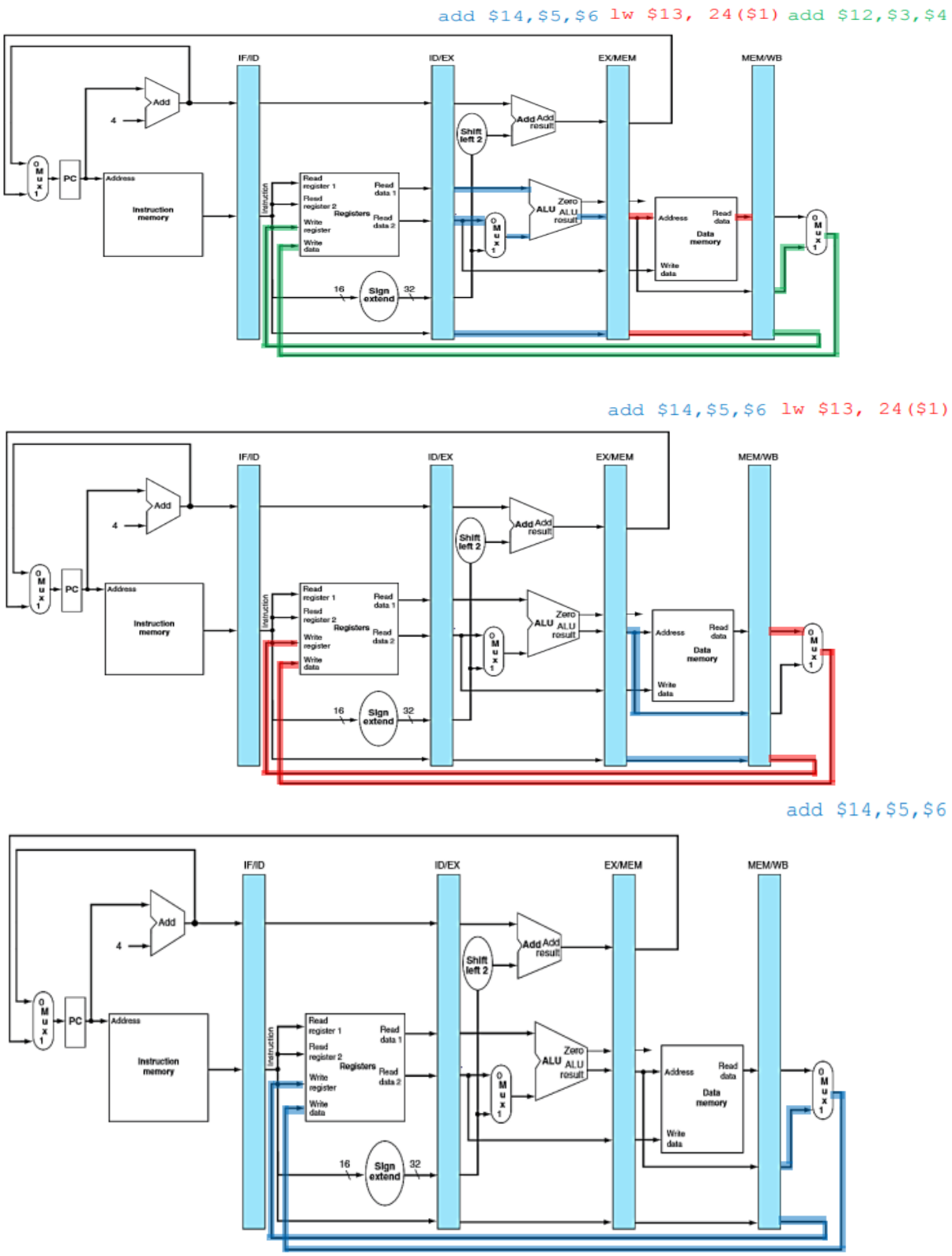


Figure 11.3 Pipelined data path in Single Cycle Risc-V Processor

11.2.3 Pipelined Control path

The pipelined control path of a RISC-V single-cycle processor involves breaking down the instruction execution process into several stages that can be processed concurrently in different pipeline stages. Typically, these stages are Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). In a pipelined design, each stage is handled by separate hardware units, allowing multiple instructions to be in different stages of execution simultaneously, thus improving overall throughput. The control path manages the coordination between these stages, ensuring proper instruction flow, data hazard resolution, and control hazard handling. This involves generating control signals at each stage to manage tasks like instruction decoding, ALU operations, memory reads/writes, and register updates, ensuring that each instruction progresses correctly through the pipeline despite potential hazards.

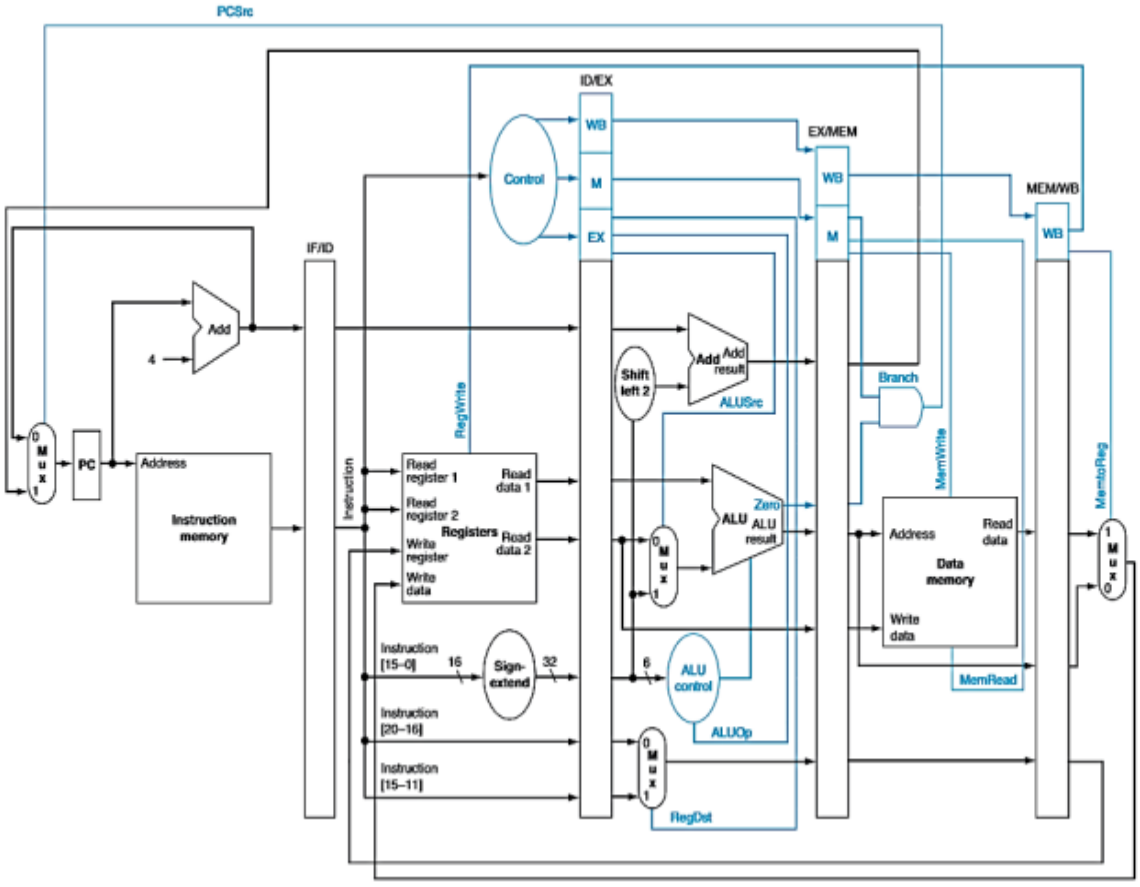
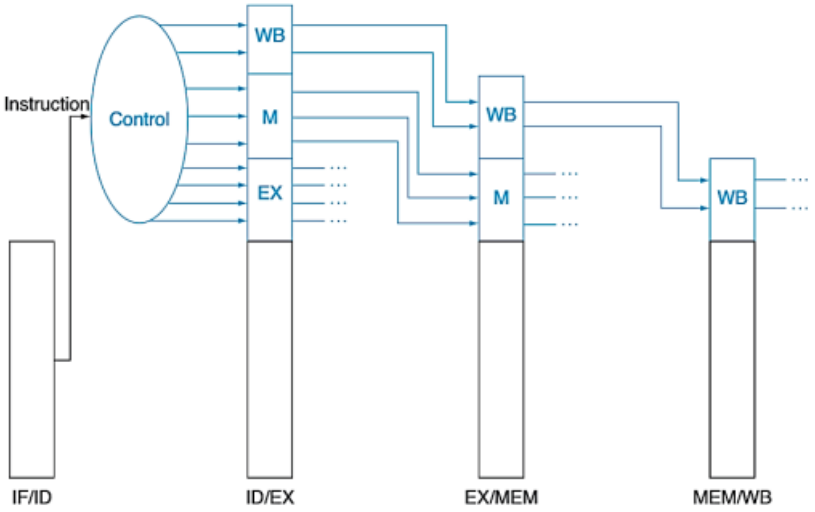


Figure 11.4 Pipelined Control Path in Single Cycle Risc-V Processor

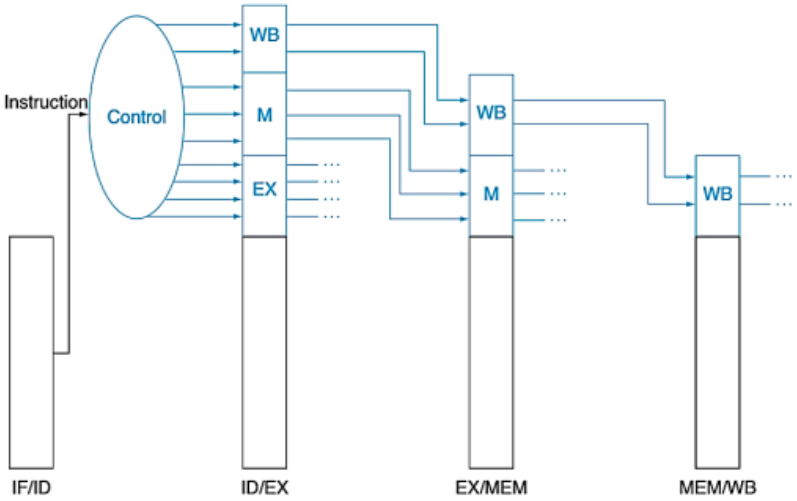
As before, we need to **preserve the context of an instruction as it moves through the datapath.**

We cannot reach back in the datapath for some information about how to execute the instruction – it will have already been overwritten.



We must pass along the control line values for **the last 3 stages.** Remember, the first 2 stages are universal.

For example, the RegWrite and MemtoReg values, decoded after the second stage, must be passed through the EX and MEM stages to the WB stage.



Our pipeline registers have been extended to handle the control information as well.

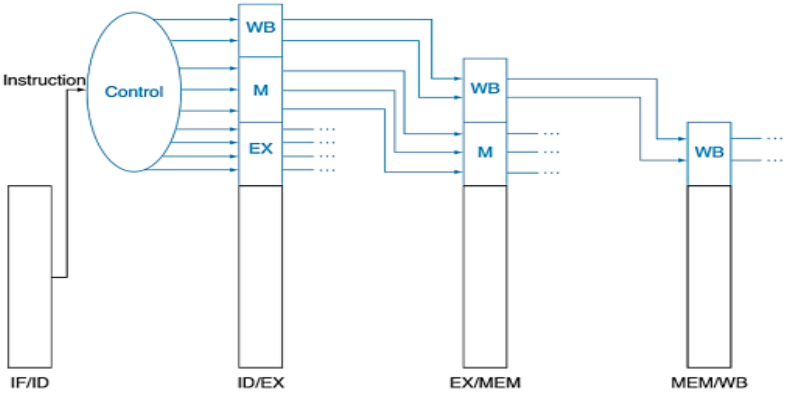


Figure 11.5 Flow of Pipelined Control path in Single Cycle Risc-V Processor

11.3 Lab Activities

11.3.1 Task-1: Design and Implement Pipelined Data Path for 16-bit single cycle processor.

11.3.2 Task-2: Implement a Pipelined Control Path for 16-bit single cycle RISC-V Processor.

11.3.3 Task-3: Implement a complete Pipelined Single Cycle RISC-V Processor.

Lab Exercise and Summary

Summary should cover Introduction, Procedure, Data Analysis and Evaluation.

Student's Signature: _____ **Date: _____**

Cognitive Activity

Q1. For each of the following actions, state which pipeline stage (IF, ID, EX, MEM or WB) it belongs to: (a) reading operands from the register file, (b) computing the ALU result, (c) writing the result back into a register. **(5)**

Ans: _____

_____

_____

_____

_____

Q2. A 'lw' instruction is immediately followed by an 'add' instruction that uses the value just loaded. Explain what type of hazard this creates and name one technique used to resolve it. **(5)**

Ans: _____

_____

_____

_____

_____

Obtained Marks	**/10**

Psychomotor Activity 1 — Experimental Setup

S. No	Criteria	Performance	Description	Marks
1	Tool Setup (4)	Level 4	Opens and configures LogiSim and selects the required environment correctly and independently.	4
		Level 3	Configures the required tool with minor errors or assistance.	3
		Level 2	Completes the setup with considerable assistance.	2
		Level 1	Setup is incomplete and requires substantial assistance.	1
2	File/Design Setup (4)	Level 4	Correctly creates/selects the required assembly file, Logisim circuit/design and required components for the Logisim pipelined processor design.	4
		Level 3	File/design setup is mostly correct with minor errors or omissions.	3
		Level 2	Setup is partially completed and requires assistance.	2
		Level 1	Major errors or omissions are present in the setup.	1
3	Parameter Setup (2)	Level 2	Correctly defines required data, registers, parameters, components and initial conditions.	2
		Level 1	Most required inputs/components are correctly configured with minor errors.	1
			Total Obtained	/10

If a student is absent:

Criteria	Description	Marks
Absent	Student is absent from the laboratory activity.	0

Psychomotor Activity 2 — Implementation + Results

S. No	Criteria	Performance	Description	Marks
1	**Implementation (4)**	Level 4	Implements the required Logisim pipelined processor design correctly and independently.	**4**
		Level 3	Implements the required program/design with minor errors or assistance.	**3**
		Level 2	Partially implements the program/design and requires considerable assistance.	**2**
		Level 1	Implementation is largely incomplete and requires substantial assistance.	**1**
2	**Execution/Simulation (2)**	Level 2	Executes the RISC-V program or simulates the Logisim design correctly and handles basic errors appropriately.	**2**
		Level 1	Executes/simulates the work with minor errors or assistance.	**1**
3	**Results/Output (4)**	Level 4	Obtains correct results and clearly demonstrates the required program output or circuit behavior.	**4**
		Level 3	Obtains mostly correct results with minor errors.	**3**
		Level 2	Obtains partially correct results with noticeable errors or omissions.	**2**
		Level 1	Results are largely incorrect or incomplete.	**1**
			Total Obtained	**/10**

If a student is absent:

Criteria	Description	Marks
Absent	Student is absent from the laboratory activity.	**0**

Affective Activity — Lab Report + Viva Voce

S. No	Criteria	Performance	Description	Marks
1	**Lab Report Completeness & Organization (4)**	Level 4	Report is complete, well organized and demonstrates full responsibility for the assigned work.	**4**
		Level 3	Report is complete with minor omissions or organizational issues and demonstrates satisfactory responsibility for the assigned work.	**3**
		Level 2	Report is partially complete with several omissions and demonstrates limited responsibility for the assigned work.	**2**
		Level 1	Report has major omissions or poor organization and demonstrates minimal responsibility for the assigned work.	**1**
2	**Code/Design, Results & Conclusion (2)**	Level 2	Logisim pipelined processor design, results, and conclusion are correctly presented and discussed, demonstrating ownership of the assigned work.	**2**
		Level 1	Logisim pipelined processor design, results, or conclusion are presented with minor errors or omissions.	**1**
3	**Viva Voce (4)**	Level 4	Clearly explains the assigned Logisim pipelined processor design, working, and results, demonstrating individual responsibility, participation, and effective communication.	**4**
		Level 3	Explains most of the assigned program/design and results correctly, with minor difficulty, and demonstrates good individual participation.	**3**
		Level 2	Explains basic aspects of the assigned program/design with some prompting and demonstrates basic participation.	**2**
		Level 1	Demonstrates limited understanding or involvement and requires substantial prompting to explain the assigned work.	**1**
			Total Obtained	**/10**

The student has not submitted the report and has not appeared in the Viva Voce.

Criteria	Description	Marks
Lab Report and Results Not Submitted	Student has not submitted the report and results file.	**0**
Absent in Viva Voce	Student is absent from the viva voce.	**0**

Instructor's Signature: _____ **Date: _____**

Experiment No 12

Implementation of Cache memory

12.1 Objectives:

After completing this experiment, student will be able to:

- Design and implement cache memory in a RISC-V single-cycle processor using Logisim.

12.2Background Theory

Cache memory is a small, high-speed storage located close to the CPU, used to temporarily hold frequently accessed data and instructions to speed up processing. In a RISC-V processor, which is based on the Reduced Instruction Set Computing architecture, the use of cache memory is crucial for optimizing performance. This is because RISC-V emphasizes executing a large number of simple instructions, which can lead to high memory access rates. By storing these instructions and the corresponding data in cache memory, the processor can minimize the latency associated with accessing the main memory, thereby enhancing overall execution speed. Caches in RISC-V processors typically include multiple levels, such as L1 (closest to the CPU) and L2 caches, each designed to hold data for different purposes and access frequencies. The use of cache memory in RISC-V also helps in reducing power consumption and improving efficiency by decreasing the need for frequent access to slower, energy-intensive main memory. Effective cache management strategies, such as prefetching, replacement policies, and coherency protocols, further optimize the processor's performance, making cache memory an indispensable component in the architecture of RISC-V processors.

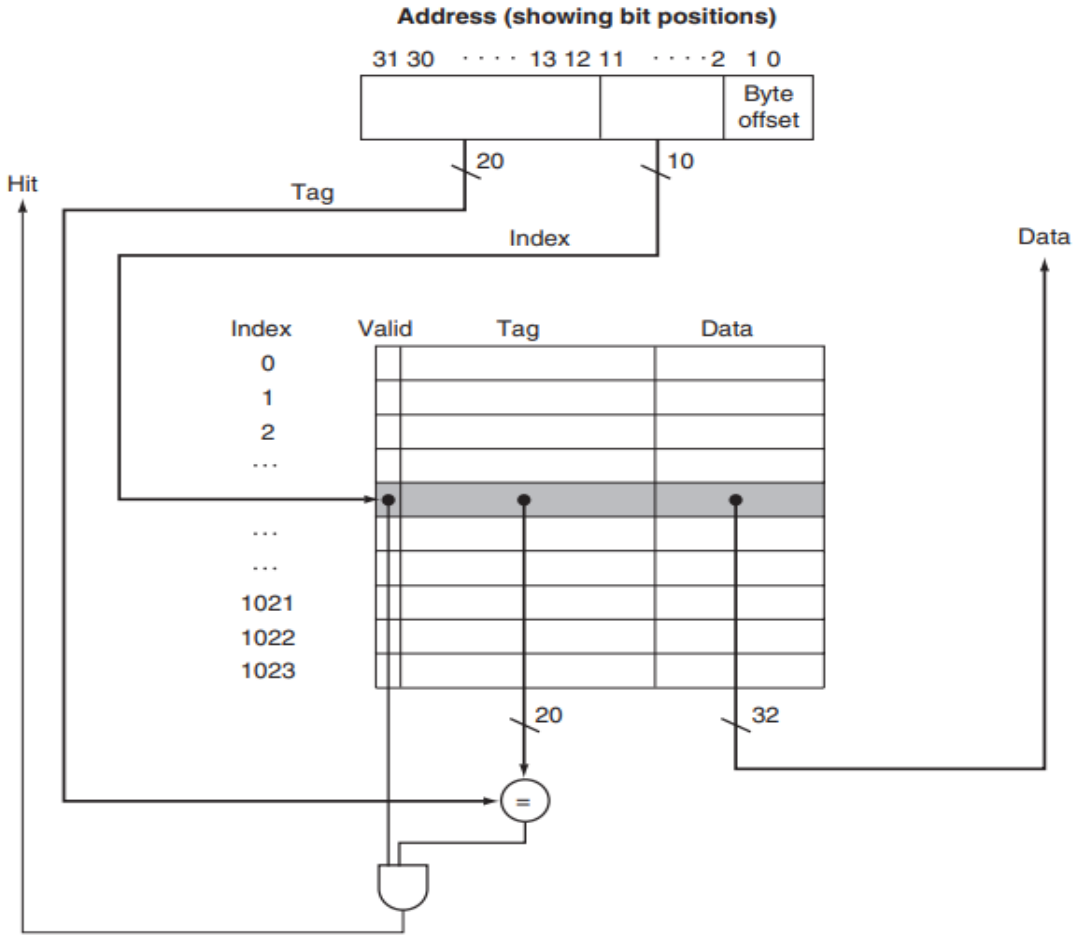


Figure: 12.1 Cache Architecture

12.2.1 Implementation steps in LogiSim

Implementing a direct-mapped cache in a RISC-V single-cycle processor using Logisim involves several steps, including designing the cache structure, integrating it with the processor, and testing its functionality. Here's a detailed guide on how to implement it:

Step 1: Designing the Cache Structure

1. Cache Size Determination:

- Determine the size of the cache (e.g., 64 words).

- Decide the word size (e.g., 32 bits).

2. Cache Organization:

- Choose a direct-mapped cache structure, where each memory address maps to a specific cache line.

- Calculate the number of cache lines based on the cache size and word size.

3. Cache Components:

- **Cache Data Storage:** Use a RAM component to store cache data.

- **Tag Storage:** Use registers or a separate RAM component to store tags for each cache line.

- **Valid Bit:** Use a bit to indicate whether a cache line contains valid data.

- **Hit/Miss Logic:** Implement comparators and logic gates to check for cache hits and misses.

Step 2: Integrating the Cache with the Processor

1. Fetch Stage Integration:

- Connect the Program Counter (PC) output to the cache address input for instruction fetching.

- Use multiplexers (MUXes) to select between cache data and main memory data based on hit/miss signals.

- Update the cache with fetched instructions on cache misses.

2. Memory Stage Integration:

- Connect the data memory address lines to the cache address input for data access.

- Use MUXes to select between cache data and main memory data for data reads and writes.

- Implement cache update logic for write-back or write-through policies.

Step 3: Testing and Debugging

1. Create Test Bench:

- Design test cases to simulate various instruction and data access patterns.

- Ensure that the cache correctly handles hits and misses.

2. Debugging:

- Use Logisim's simulation tools to step through the design.

- Verify that the cache fetches and stores data correctly, and that hit/miss detection works as intended.

12.2.3 Detailed Instructions:

Step1: Cache Structure Design

1. Cache Data Storage (RAM):

- Place a RAM component in the cache circuit.

- Configure the RAM with the determined size and word size.

2. Tag Storage:

- Use registers or a separate RAM component to store tags for each cache line.

- Ensure that the tag storage is appropriately sized to accommodate the memory address space.

3. Valid Bit:

- Include a valid bit for each cache line to indicate whether the data in that line is valid.

- Set the valid bit when data is loaded into the cache and clear it on cache line eviction or invalidation.

4. Hit/Miss Logic:

- Implement comparators to compare the requested memory address tag with the stored tags.

- Use logic gates to combine the comparator outputs and the valid bit to generate hit/miss signals.

Step2: Integration with the Processor

1. Instruction Fetch Integration:

- Connect the PC output to the cache address input.

- Use MUXes to select between cache data and main memory data for instruction fetches.

- Update the cache with fetched instructions on cache misses.

2. Memory Access Integration:

- Connect the data memory address lines to the cache address input.

- Use MUXes to select between cache data and main memory data for data reads and writes.

- Implement cache update logic for write-back or write-through policies.

Step3: Testing and Debugging:

1. Test Bench Creation:

- Design test cases to cover various scenarios, including cache hits, misses, and updates.

- Verify that the cache behaves as expected under different access patterns.

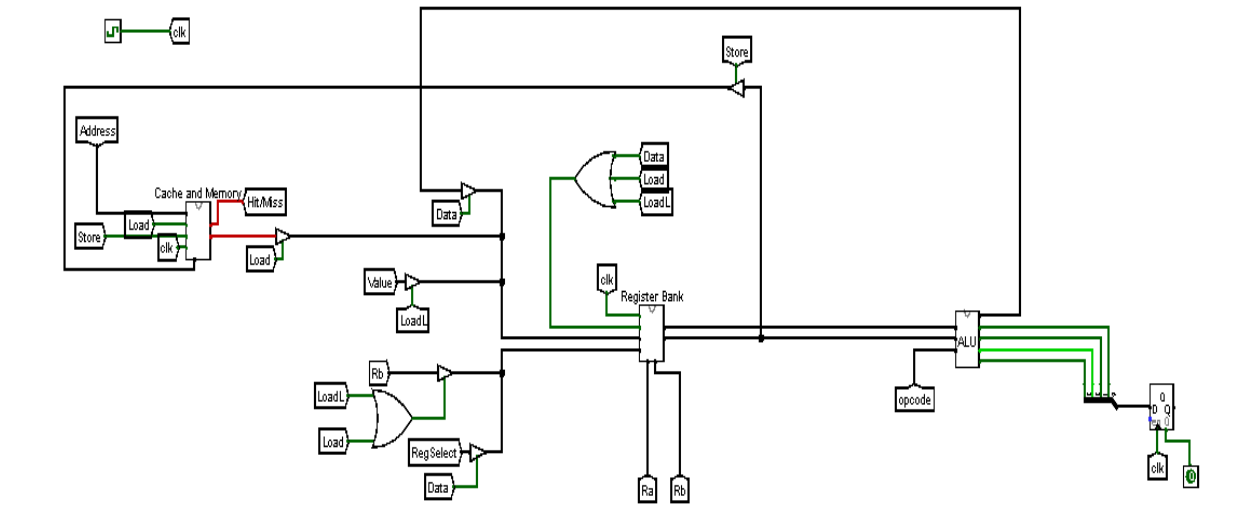
2. Debugging:

- Use Logisim's simulation tools to observe the cache behavior.

- Check for correct data fetching, storage, and hit/miss detection.

- Address any issues or discrepancies in the cache operation.

Implementing a direct-mapped cache in a RISC-V single-cycle processor using Logisim involves designing the cache structure, integrating it with the processor, and thoroughly testing its functionality. By following the detailed steps outlined above, you can create a functional cache system that improves memory access performance in your processor design.



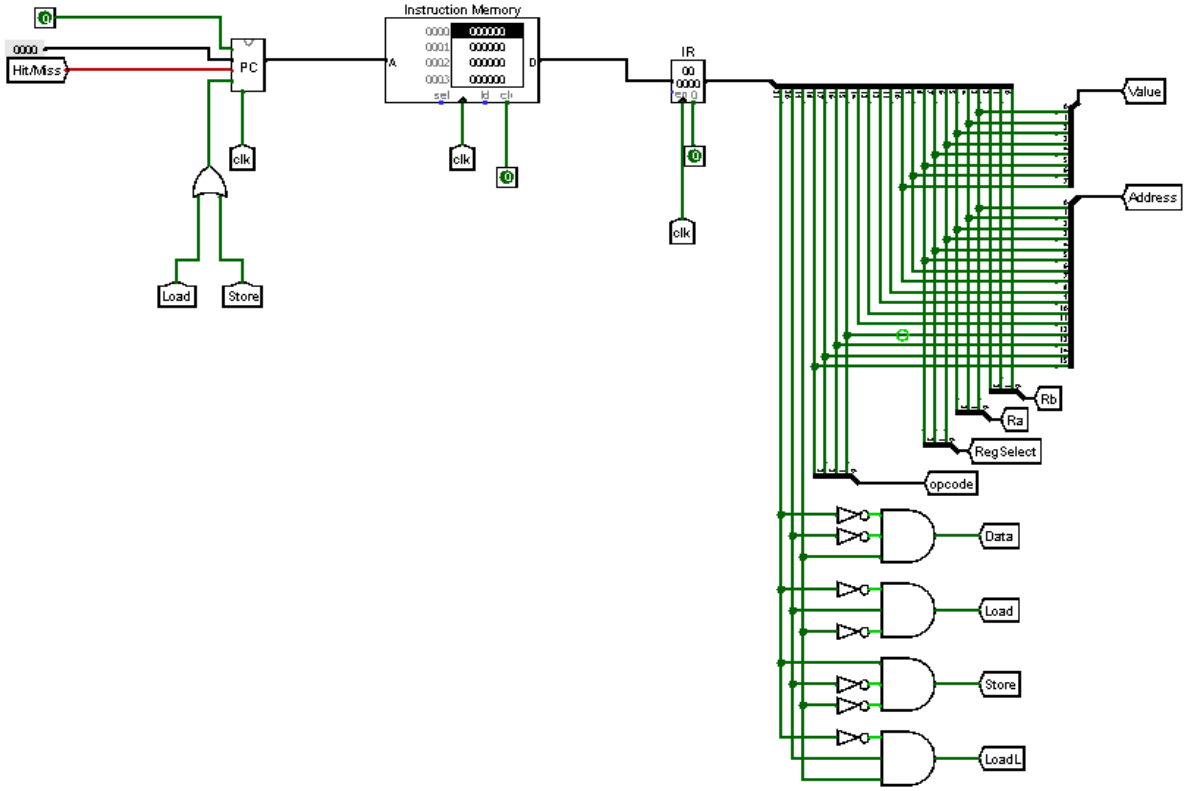


Figure: 12.2 Implemented Circuit of Cache Memory

12.3 Lab Activities

Task-1: Design and Implement 32-bit direct Cache memory in a single cycle RISC-V processor using LogiSim.

Lab Exercise and Summary

Summary should cover Introduction, Procedure, Data Analysis and Evaluation.

Student's Signature: _____ **Date: _____**

Cognitive Activity

Q1. A direct-mapped cache has 8 lines (numbered 0 to 7). Using the rule (block address) mod (number of lines), calculate which cache line block address 21 maps to. Show your calculation. **(5)**

Ans: _____

_____

_____

_____

_____

Q2. Explain, with a short example, the difference between a compulsory (cold) miss and a conflict miss in a direct-mapped cache. **(5)**

Ans: _____

_____

_____

_____

_____

Obtained Marks	**/10**

Psychomotor Activity 1 — Experimental Setup

S. No	Criteria	Performance	Description	Marks
1	Tool Setup (4)	Level 4	Opens and configures LogiSim and selects the required environment correctly and independently.	4
		Level 3	Configures the required tool with minor errors or assistance.	3
		Level 2	Completes the setup with considerable assistance.	2
		Level 1	Setup is incomplete and requires substantial assistance.	1
2	File/Design Setup (4)	Level 4	Correctly creates/selects the required assembly file, Logisim circuit/design and required components for the Logisim cache memory design.	4
		Level 3	File/design setup is mostly correct with minor errors or omissions.	3
		Level 2	Setup is partially completed and requires assistance.	2
		Level 1	Major errors or omissions are present in the setup.	1
3	Parameter Setup (2)	Level 2	Correctly defines required data, registers, parameters, components and initial conditions.	2
		Level 1	Most required inputs/components are correctly configured with minor errors.	1
			Total Obtained	/10

If a student is absent:

Criteria	Description	Marks
Absent	Student is absent from the laboratory activity.	0

Psychomotor Activity 2 — Implementation + Results

S. No	Criteria	Performance	Description	Marks
1	**Implementation (4)**	Level 4	Implements the required Logisim cache memory design correctly and independently.	**4**
		Level 3	Implements the required program/design with minor errors or assistance.	**3**
		Level 2	Partially implements the program/design and requires considerable assistance.	**2**
		Level 1	Implementation is largely incomplete and requires substantial assistance.	**1**
2	**Execution/Simulation (2)**	Level 2	Executes the RISC-V program or simulates the Logisim design correctly and handles basic errors appropriately.	**2**
		Level 1	Executes/simulates the work with minor errors or assistance.	**1**
3	**Results/Output (4)**	Level 4	Obtains correct results and clearly demonstrates the required program output or circuit behavior.	**4**
		Level 3	Obtains mostly correct results with minor errors.	**3**
		Level 2	Obtains partially correct results with noticeable errors or omissions.	**2**
		Level 1	Results are largely incorrect or incomplete.	**1**
			Total Obtained	**/10**

If a student is absent:

Criteria	Description	Marks
Absent	Student is absent from the laboratory activity.	**0**

Affective Activity — Lab Report + Viva Voce

S. No	Criteria	Performance	Description	Marks
1	Lab Report Completeness & Organization (4)	Level 4	Report is complete, well organized and demonstrates full responsibility for the assigned work.	4
		Level 3	Report is complete with minor omissions or organizational issues and demonstrates satisfactory responsibility for the assigned work.	3
		Level 2	Report is partially complete with several omissions and demonstrates limited responsibility for the assigned work.	2
		Level 1	Report has major omissions or poor organization and demonstrates minimal responsibility for the assigned work.	1
2	Code/Design, Results & Conclusion (2)	Level 2	Logisim cache memory design, results, and conclusion are correctly presented and discussed, demonstrating ownership of the assigned work.	2
		Level 1	Logisim cache memory design, results, or conclusion are presented with minor errors or omissions.	1
3	Viva Voce (4)	Level 4	Clearly explains the assigned Logisim cache memory design, working, and results, demonstrating individual responsibility, participation, and effective communication.	4
		Level 3	Explains most of the assigned program/design and results correctly, with minor difficulty, and demonstrates good individual participation.	3
		Level 2	Explains basic aspects of the assigned program/design with some prompting and demonstrates basic participation.	2
		Level 1	Demonstrates limited understanding or involvement and requires substantial prompting to explain the assigned work.	1
			Total Obtained	/10

The student has not submitted the report and has not appeared in the Viva Voce.

Criteria	Description	Marks
Lab Report and Results Not Submitted	Student has not submitted the report and results file.	0
Absent in Viva Voce	Student is absent from the viva voce.	0

Instructor's Signature: _____ **Date: _____**

Experiment No 13-14

Lab Project I-II

Problem Description:

The increasing adoption of open-source processor architectures has made **RISC-V** a widely used platform for learning and developing modern computer systems. Understanding how RISC-V instructions are represented, executed, and supported by processor hardware is essential for designing efficient and reliable computing solutions. Simulation tools such as Logisim and Venus RISC-V Simulator provide an effective environment for analyzing processor components and verifying instruction execution without requiring physical hardware.
As a computer engineering team, you are assigned to **investigate** the RISC-V Architecture and its Instruction Set Architecture (ISA) by designing and simulating relevant processor components in **Logisim** and implementing/testing RISC-V assembly programs in the Venus RISC-V Simulator. Your project should focus on a problem or application of your choice that demonstrates the interaction between the ISA and the underlying hardware.

Project Objectives:

Based on investigation, students are required to:

1. **Justify** the selected architectural decisions and explain how they address the identified problem.
2. **Justify** the choice of RISC-V instructions used in the implementation.
3. Design and simulate the required hardware components using Logisim.
4. Implement and verify the RISC-V assembly program using the Venus RISC-V Simulator.
5. Validate the correctness of the hardware and software implementations using appropriate simulation results and technical evidence.
6. Perform a functional and/or performance evaluation of the proposed solution.
7. Discuss the strengths and limitations of the implemented design.
8. Recommend possible improvements or future enhancements to the proposed solution.
9. Present logical analysis and simulation-based evidence to support all design and implementation decisions.

Note: The project is open-ended; therefore, multiple valid solutions are acceptable, provided they are supported by sound engineering principles, logical analysis, and simulation-based validation.

Project Deliverables:

Students are required to submit the following deliverables for the open-ended RISC-V Architecture and ISA project:

1. **Project Proposal**
    - Problem statement
    - Project objectives
    - Scope of the proposed solution
    - Expected outcomes
2. **System Design**
    - Overall system architecture
    - Block diagram of the proposed solution
    - Description of the selected RISC-V ISA features and hardware components
3. **Logisim Simulation**
    - Design and simulation of the required digital circuits/components
    - Verification of circuit functionality using appropriate test cases
    - Annotated screenshots of the simulation results
4. **Venus RISC-V Implementation**
    - RISC-V assembly source code
    - Successful execution and testing in the Venus simulator
    - Screenshots demonstrating program execution and output
5. **Testing and Evaluation**
    - Test cases and expected results
    - Comparison of expected versus observed outputs
    - Discussion of correctness, limitations, and possible improvements
6. **Final Project Report**
    - Introduction
    - Problem description
    - Design methodology
    - Implementation details
    - Simulation results
    - Analysis and discussion
    - Conclusion
    - References

7. **Project Demonstration**
    - Live demonstration of the Logisim circuit and Venus implementation
    - Explanation of the design decisions and project workflow
    - Question-and-answer session

These deliverables ensure that both the hardware simulation (Logisim) and software implementation (Venus) aspects of the RISC-V project are thoroughly documented and evaluated.

Applications:

Students may choose **any one** of the following application areas for their open-ended project:

1. **Embedded Systems** – Design and simulate processor operations for microcontroller-based applications.
2. **Internet of Things (IoT)** – Implement RISC-V programs for sensor interfacing and IoT edge devices.
3. **Digital Signal Processing (DSP)** – Simulate arithmetic and data-processing operations used in signal processing.
4. **Industrial Automation** – Develop control logic for monitoring and automation systems.
5. **Robotics** – Implement instruction sequences for robot movement and control tasks.
6. **Smart Home Systems** – Simulate processor functionality for home automation applications.
7. **Computer Organization and Processor Design** – Design and verify processor components such as the ALU, register file, control unit, or datapath.
8. **Memory Management** – Simulate memory access instructions, addressing modes, and data transfer operations.
9. **Arithmetic and Logic Processing** – Implement and evaluate arithmetic, logical, and comparison operations using the RISC-V ISA.
10. **Branching and Program Control** – Demonstrate loops, conditional execution, procedure calls, and program flow control.
11. **Data Communication Interfaces** – Simulate processor interaction with memory-mapped input/output devices.
12. **Educational Processor Simulator** – Develop a simplified RISC-V processor model for teaching and learning computer architecture concepts.

13. **Instruction Execution Analysis** – Analyze the execution of different RISC-V instruction formats (R, I, S, B, U, and J types).

14. **Custom RISC-V Extension Study** – Investigate and simulate a simple custom instruction or ISA extension concept.

15. **Performance Analysis** – Compare different implementation approaches based on instruction count, execution cycles, or hardware complexity.

Note: Students may propose another relevant application area, subject to instructor approval, provided it involves hardware simulation in Logisim and software verification using the Venus RISC-V Simulator.